

THƯ VIỆN
ĐẠI HỌC NHA TRANG

Đ

005.74
Ph 561 L



TỦ SÁCH
ĐỂ HỌC

Sổ dữ liệu

giáo trình nhập môn



Phương Lan (Chủ biên)



Tiếp cận cơ sở dữ liệu
(Bảng, cột, kiểu dữ liệu ...)



Chuẩn hóa CSDL, cú pháp
SQL và các hệ quản trị



Phân tích thiết kế CSDL
với mô hình ER



THU VIỆN ĐH NHA TRANG



1000019193

Lược đồ giao dịch, bảo mật ...



NHÀ XUẤT BẢN LAO ĐỘNG XÃ HỘI



Phương Lan (Chủ biên)
Hoàng Đức Hải

giáo trình nhập môn
Cơ sở dữ liệu



cuu duong than cong. com

NHÀ XUẤT BẢN LAO ĐỘNG XÃ HỘI

***Giáo trình nhập môn* Cơ sở dữ liệu** **(TỦ SÁCH DỄ HỌC)**

NHÀ XUẤT BẢN LAO ĐỘNG – XÃ HỘI

Ngõ Hòa Bình 4 - Minh Khai - Hai Bà Trưng - Hà Nội

Tel: (04) 6.246.913 – Fax: (04) 6.246.915

Chịu trách nhiệm xuất bản: HÀ TẮT THẮNG

Biên tập: BAN BIÊN TẬP GIÁO TRÌNH DẠY NGHỀ

Biên soạn: PHƯƠNG LAN

Sửa bản in: NGỌC AN

Trình bày bìa: HUỖNH THẢO

Thực hiện liên doanh: Công ty TNHH Minh Khai S.G

E-mail: mk.book@minhkhai.com.vn – Website: www.minhkhai.com.vn

Tổng phát hành

- ❖ Nhà sách Minh Khai: 249 Nguyễn Thị Minh Khai - Quận 1 - TP.HCM
ĐT: (08) 9.250.590 - 9.250.591 – Fax: (08) 9.257.837
- ❖ Nhà sách Minh Châu: Nhà 30 - Ngõ 22 - Tạ Quang Bửu - Bách Khoa - Hà Nội
ĐT: (04) 8.692.785 – Fax: (04) 8.683.995

Đại lý các khu vực

- ❖ Nhà sách Huy Hoàng: 95 Núi Trúc - Kim Mã - Ba Đình - Hà Nội
ĐT: (04) 7.365.859
 - ❖ Cty cổ phần sách thiết bị trường học Đà Nẵng: 78 Bạch Đằng - Đà Nẵng
ĐT: 0511.837100
 - ❖ Nhà sách Chánh Trí: 116A Nguyễn Chí Thanh - Đà Nẵng
ĐT: 0511.820129
 - ❖ Cty phát hành sách Khánh Hòa:
 - Nhà sách Ponagar: 73 Thống Nhất - Nha Trang - Khánh Hòa
ĐT: 058.822636
 - Siêu thị sách Tân Tiến - 11 Lê Thành Phương - Nha Trang - Khánh Hòa
ĐT: 058.827303
 - ❖ Nhà sách Năm Hiền: 79/6 Xô Viết Nghệ Tĩnh - TP.Cần Thơ
ĐT: 071. 821668
-

In 1.000 cuốn, khổ 16 x 24 cm, tại Xí nghiệp in Machinco

Số 21 Bùi Thị Xuân, Quận 1, Thành phố Hồ Chí Minh

Số đăng ký kế hoạch xuất bản: 204-2008/CXB/21-68/LĐXH

Mã số 21-68
17-3 . Quyết định xuất bản số 167/QĐ-NXBLĐXH ngày 24/3/2008

In xong và nộp lưu chiểu Quý II năm 2008

LỜI GIỚI THIỆU

Trong thời đại thông tin bùng nổ như ngày nay, cơ sở dữ liệu (CSDL) ngày càng đóng vai trò quan trọng trong cuộc sống. Một quyển danh bạ điện thoại, danh mục khách hàng hay hàng trăm địa chỉ email, Web site mà bạn phải nhớ đều cần đến CSDL để lưu trữ và truy xuất. Giáo trình nhập môn CSDL này sẽ cung cấp cho bạn một cái nhìn tổng quát về cơ sở dữ liệu là gì, nó được ứng dụng như thế nào trong cuộc sống.

Giáo trình tập trung vào cả hai nội dung: lý thuyết lẫn thực tiễn. Bạn sẽ tiếp cận các nội dung như:

- *Các khái niệm cơ bản về cơ sở dữ liệu.*
- *Chuẩn hóa cơ sở dữ liệu.*
- *Cú pháp SQL.*
- *Sử dụng MS Access và các hệ quản trị cơ sở dữ liệu.*
- *Phân tích thiết kế cơ sở dữ liệu và mô hình ER.*
- *Đại số quan hệ.*
- *Giao dịch, bảo mật và lập trình kết nối cơ sở dữ liệu.*

và còn nhiều nữa ...

Giáo trình này thích hợp cho các bạn muốn có thêm kiến thức về công nghệ thông tin lẫn các bạn đang học chuyên ngành về tin học. Bạn sẽ có được một nền tảng vững chắc để tự mình xây dựng, phân tích, thiết kế các hệ thống phần mềm từ nhỏ đến lớn. Thông qua giáo trình này bạn có thể lựa chọn cho mình những hệ quản trị cơ sở dữ liệu thích hợp như MS Access, Foxpro hay Oracle, SQL Server, hiểu được cú pháp SQL mà các hệ quản trị này cung cấp để truy xuất và lưu trữ dữ liệu được tốt hơn.

Chúc bạn có một bước khởi đầu đầy thú vị.

MK.PUB

THƯ NGỎ

Kính thưa quý Bạn đọc gần xa!

Trước hết, Ban xuất bản xin bày tỏ lòng biết ơn và niềm vinh hạnh được đồng đảo Bạn đọc nhiệt tình ủng hộ tủ sách MK.PUB.

Trong thời gian qua chúng tôi rất vui và cảm ơn các Bạn đã gửi e-mail đóng góp nhiều ý kiến quý báu cho tủ sách.

Mục tiêu và phương châm phục vụ của chúng tôi là:

- Lao động khoa học nghiêm túc.
- Chất lượng và ngày càng chất lượng hơn.
- Tất cả vì Bạn đọc.

Một lần nữa, Ban xuất bản MK.PUB xin kính mời quý Bạn đọc tiếp tục tham gia cùng chúng tôi để nâng cao chất lượng sách. Cụ thể:

Trong quá trình sử dụng sách, nếu quý Bạn phát hiện thấy bất kỳ sai sót nào (*dù nhỏ*) xin đánh dấu, ghi chú nhận xét ý kiến của Bạn ra bên cạnh rồi gửi cuốn sách này cho chúng tôi theo địa chỉ:

Nhà sách Minh Khai

249 Nguyễn Thị Minh Khai, Q.1, Tp. Hồ Chí Minh.

E-mail: mk.book@minhkhai.com.vn hoặc mk.pub@minhkhai.com.vn

Chúng tôi xin hoàn lại cước phí bưu điện và gửi trả lại Bạn cuốn sách cùng tên. Ngoài ra chúng tôi còn gửi tặng Bạn một cuốn sách khác trong tủ sách MK.PUB. Bạn có thể chọn cuốn sách này theo danh mục thích hợp sẽ gửi tới Bạn.

Với mục đích ngày càng nâng cao chất lượng tủ sách MK.PUB, chúng tôi rất mong nhận được sự hợp tác nhiệt tình của quý Bạn đọc gần xa.

“MK.PUB cùng Bạn đọc đồng hành” để nâng cao chất lượng sách.

Một lần nữa chúng tôi xin chân thành cảm ơn.

MK.PUB

MỤC LỤC

LỜI NÓI ĐẦU	3
LỜI NGỎ.....	3
Chương 1: LÀM QUEN VỚI CƠ SỞ DỮ LIỆU	11
1. Cơ sở dữ liệu là gì	11
2. Thực thể và quan hệ	12
3. Bảng dữ liệu (Tables).....	14
4. Cột (Column) hoặc thuộc tính (Attribute).....	15
5. Hàng (Row), bản ghi (Record), bộ dữ liệu (Tuples).....	15
6. Khóa	15
7. Phụ thuộc hàm	16
8. Lược đồ	17
9. Những nguyên lý thiết kế cơ sở dữ liệu	17
10. Kiểu dữ liệu	23
10.1. Chuỗi (String) và số (Number)	23
10.2. Kiểu ngày tháng (thời gian) và tiền tệ.....	23
10.3. Các kiểu dữ liệu phức hợp	24
10.4. Dữ liệu “nhảy cảm”.....	26
11. Kích thước cột dữ liệu (field size).....	27
12. Tên cột dữ liệu	27
13. Định dạng và kiểm tra tính hợp lệ của dữ liệu	27
Chương 2: CHUẨN HÓA CƠ SỞ DỮ LIỆU	29
1. Phụ thuộc hàm	30
2. Khóa và phụ thuộc hàm.....	31
3. Các dạng chuẩn hóa dữ liệu.....	32
3.1. Chuẩn hóa dạng 1 (1NF)	33
3.2. Chuẩn hóa dạng 2 (2NF)	34
3.3. Chuẩn hóa dạng 3 (3NF)	35
3.4. Quan hệ dữ liệu.....	37
3.5. Chuẩn hóa dạng 4 (4NF)	39

Chương 3: SQL VÀ CƠ SỞ DỮ LIỆU	41
1. SQL là gì?.....	42
2. SQL và các chức năng cơ bản.....	42
2.1. Bảng cơ sở dữ liệu	42
2.2. Truy vấn SQL.....	43
2.3. SQL ngôn ngữ thao tác dữ liệu (DML).....	43
2.4. Ngôn ngữ định nghĩa dữ liệu SQL (DDL)	44
3. Lệnh SELECT.....	44
3.1. Tuyển chọn các cột	44
3.2. Lựa chọn tất cả các cột.....	45
3.3. Tập kết quả.....	45
3.4. SELECT DISTINCT lệnh chọn dữ liệu không trùng.....	46
3.5. Mệnh đề WHERE.....	47
3.6. Điều kiện so khớp LIKE.....	49
4. Lệnh chèn dữ liệu INSERT INTO.....	50
5. Lệnh cập nhật dữ liệu UPDATE	51
6. Lệnh xóa dữ liệu DELETE	52
7. Kiểm tra những kỹ năng SQL vừa học của bạn	53
8. Sắp xếp thứ tự bằng SQL ORDER BY.....	55
Sắp xếp hàng dữ liệu	55
9. SQL AND & OR.....	57
10. SQL IN	59
11. SQL BETWEEN.....	59
12. Bí danh SQL (Alias).....	61
13. SQL JOIN kết nối các bảng	62
Kết nối và khóa.....	62
14. Hợp hai bảng với SQL UNION và UNION ALL.....	66
UNION.....	66
15. SQL tạo cơ sở dữ liệu, bảng và chỉ mục (Index)	69
Tạo chỉ mục (Index)	70
16. SQL DROP xóa bảng, chỉ mục và cơ sở dữ liệu	71
17. SQL thay đổi cấu trúc bảng.....	72

18. Hàm SQL.....	73
19. SQL GROUP BY và HAVING	74
20. SQL và lệnh SELECT INTO	76
21. SQL CREATE VIEW lệnh tạo khung nhìn (view) của dữ liệu.....	77
22. SQL tham khảo nhanh	79
23. Kiểm tra nhanh	81
Chương 4: CƠ SỞ DỮ LIỆU ACCESS	87
1. Tạo cơ sở dữ liệu (Database).....	88
1.1. Cơ sở dữ liệu tự tạo	88
1.2. Tạo cơ sở dữ liệu từ Template	90
Chương 5: LẬP TRÌNH KẾT NỐI CƠ SỞ DỮ LIỆU	97
1. Các hệ ngôn ngữ quản trị cơ sở dữ liệu thế hệ 4.....	97
2. Cơ sở dữ liệu trong những ngôn ngữ lập trình khác	98
Cursors.....	99
3. Các hàm API giao tiếp cơ sở dữ liệu.....	99
4. Liên kết dữ liệu với thành phần trực quan.....	101
5. Sử dụng bảng tính SpreadSheet.....	102
6. Sử dụng ngôn ngữ lập trình Web PHP và cơ sở dữ liệu MySQL	102
7. SQL Embeded	104
8. Ưu điểm của chuẩn API	106
9. ODBC - Open Database Connectivity	106
10. Sử dụng SQLBindCol (ODBC)	107
11. JDBC	109
12. DBI/DBD	110
13. ADO và ASP.....	110
14. Những vấn đề về tính hiệu quả.....	111
Chương 6: METADATA, BẢO MẬT VÀ QUẢN TRỊ (DBA)	113
1. METADATA (Siêu dữ liệu)	113
2. Bảo mật (Security).....	116
3. Bảo vệ mức DBMS	118
4. Bảo vệ mức người dùng – Hạn chế SQL	118

5. Nhà quản trị cơ sở dữ liệu (DBA Database Administrator)	119
Chương 7: PHÂN TÍCH THIẾT KẾ CƠ SỞ DỮ LIỆU.....	121
1. Chu trình sống của phân tích cơ sở dữ liệu	122
2. Mô hình cơ sở dữ liệu ba mức.....	124
3. Mô hình hóa quan hệ thực thể (ER - Entity Relationship)	125
3.1. Thực thể - Entities	125
3.2. Thuộc tính (Attribute).....	126
3.3. Keys (khóa)	126
3.4. Mối quan hệ (relationships).....	127
4. Các cấp độ của quan hệ	127
4.1. Biểu diễn cấp độ quan hệ	127
4.2. Thay thế những mối quan hệ tam phân.....	129
4.3. Quan hệ chính yếu	129
4.4. Quan hệ tùy chọn	130
5. Tập hợp thực thể.....	131
6. Dư thừa quan hệ.....	132
7. Chia cắt quan hệ n:m.....	133
8. Xây dựng mô hình ER	133
Chương 8: XÂY DỰNG MÔ HÌNH QUAN HỆ THỰC THỂ	135
1. Mô hình quản lý Công ty vận tải xe Bus.....	135
1.1. Xác định thực thể.....	136
1.2. Xác định các mối quan hệ.....	136
1.3. Vẽ lược đồ ER.....	137
1.4. Xác định thuộc tính.....	137
2. Các vấn đề phát sinh với mô hình ER.....	137
3. Mở rộng mô hình ER (EER hay Enhanced ER)	138
3.1. Chuyên biệt hóa.....	139
3.2. Tổng quát hóa.....	140
Chương 9: ÁNH XẠ MÔ HÌNH THỰC THỂ ER.....	141
1. Quan hệ là gì?	141
2. Khóa ngoại (Foreign Key)	142
3. Chuẩn bị ánh xạ mô hình ER	142

3.1. Ánh xạ mối quan hệ liên kết 1:1	142
3.2. Ánh xạ mối quan hệ liên kết 1:m	147
3.3. Ánh xạ quan hệ n:m.....	148
4. Tóm lược	148
5. Ánh xạ ER mở rộng	149
5.1. Ánh xạ những mối quan hệ song song	149
5.2. Ánh xạ 1:m trong mối quan hệ liên kết vòng.....	149
5.3. Ánh xạ lớp cha superclass và lớp con subclass	150
Chương 10: ĐẠI SỐ QUAN HỆ.....	153
1. Đại số quan hệ là gì?	153
2. Thuật ngữ	154
3. Toán tử ghi (write).....	154
4. Các toán tử trích rút dữ liệu.....	155
4.1. Quan hệ lựa chọn <i>SELECT</i>	155
4.2. Quan hệ chiếu <i>PROJECT</i>	155
4.3. <i>SELECT</i> và <i>PROJECT</i>	155
5. Toán tử tập hợp - ngữ nghĩa.....	156
6. Các toán tử tập hợp - yêu cầu.....	156
7. Phép tích Decac	157
8. Toán tử kết nối - JOIN	158
9. OUTER JOIN	159
10. Các ví dụ về Đại số quan hệ.....	160
10.1. Toán tử đổi tên.....	162
10.2. Toán tử dẫn xuất.....	162
10.3. Tương đương.....	163
11. So sánh Đại số quan hệ và SQL.....	164
Chương 11: TRANH CHẤP VÀ QUẢN LÝ GIAO DỊCH.....	165
1. Giao dịch (Transaction)	165
2. Lịch biểu cho các giao dịch.....	166
3. Cập nhật mất mát dữ liệu.	167
4. Phụ thuộc không xác nhận	168
5. Sự mâu thuẫn không tương thích (Inconsistency).....	168

6. Tính tuần tự (Serialisability).....	169
7. Đồ thị mức ưu tiên	169
8. Đồng bộ	171
8.1. Khóa (Lock).....	171
8.2. Khóa - phụ thuộc dữ liệu không xác nhận.....	172
8.3. Khóa chết (Deadlock).....	172
8.4. Giải quyết deadlock.....	173
8.5. Các phương pháp tương thích cơ sở dữ liệu khác	173
9. Khôi phục (Recovery).....	174
9.1. Khôi phục từ các file thô (dump)	174
9.2. Khôi phục dựa vào các file log của giao dịch.....	175
9.3. Trì hoãn cập nhật	175
9.4. Cập nhật tức thời.....	177
10. Rollback (Quay ngược)	178
Chương 12: LÝ THUYẾT CÀI ĐẶT VÀ XÂY DỰNG HỆ DBMS ...	179
1. Các thành phần lõi	179
2. Địa và kí ức bộ nhớ.....	182
3. Tổ chức chỉ mục.....	182
3.1. Hash table.....	183
3.2. Cây nhị phân (Binary Tree).....	185
3.3. Chi phí đánh chỉ mục và truy cập trên chỉ mục.....	186

cui duong than cong. com

Chương 1:

LÀM QUEN VỚI CƠ SỞ DỮ LIỆU

Các vấn đề chính sẽ được đề cập:

- ✓ Cơ sở dữ liệu là gì?
- ✓ Các khái niệm cơ bản về thành phần cơ sở dữ liệu.
- ✓ Thực thể và quan hệ.
- ✓ Bảng, Cột, Hàng, Khóa, Kiểu dữ liệu.
- ✓ Phụ thuộc hàm.
- ✓ Lược đồ.
- ✓ Nguyên lý thiết kế cơ sở dữ liệu.

1. CƠ SỞ DỮ LIỆU LÀ GÌ

Để hiểu những nguyên lý của cơ sở dữ liệu trong giáo trình này, trước hết chúng ta cần hiểu qua một vài khái niệm và thuật ngữ tin học cơ bản.

Điều đầu tiên nhất: Cơ sở dữ liệu là gì?

Đơn giản cơ sở dữ liệu là một cấu trúc hay một bộ khung biểu diễn những thông tin liên quan với nhau. Phần mềm dùng quản lý và xử lý thông tin của cấu trúc thông tin này được gọi là hệ DBMS (Hệ thống Quản lý Cơ sở dữ liệu – DataBase Management System). Cơ sở dữ liệu là một thành phần trong hệ DBMS.

Bạn có thể nghĩ và hình dung đơn giản cơ sở dữ liệu là một danh sách thông tin. Như trang niên giám điện thoại chẳng hạn, mỗi trang là một danh sách chứa các mục thông tin – gồm tên, địa chỉ, số điện thoại – mô tả về người thuê bao điện thoại trong một vùng nào đó (thông tin mô tả đối tượng). Tất cả thông tin của người thuê bao dùng chung một mẫu (cấu trúc). Theo thuật ngữ của cơ sở dữ liệu, các trang niên giám tương đương với một bảng (table) dữ liệu mà trong đó thông tin mỗi người thuê bao được đại diện hay biểu diễn bởi một bản ghi (record) hay bạn có thể gọi là “mẫu tin”.

Thông tin bản ghi mô tả về người thuê bao chứa ba mục: tên, địa chỉ và số điện thoại. Các bản ghi được sắp xếp theo thứ tự abc và được gọi là khóa dùng để tìm kiếm khi cần.

Các hình ảnh ví dụ khác về cơ sở dữ liệu còn có thể là danh sách khách hàng, danh mục hay danh sách sinh viên, danh sách hàng hóa, và ngay cả nội dung một trang Web cũng có thể xem là một cơ sở dữ liệu. Nội dung danh sách có thể biểu diễn trong thực tế là vô hạn.

Bạn có thể lập mô hình và thiết kế một cơ sở dữ liệu để cất giữ bất cứ những gì đại diện cho thông tin có cấu trúc.

2. THỰC THỂ VÀ QUAN HỆ

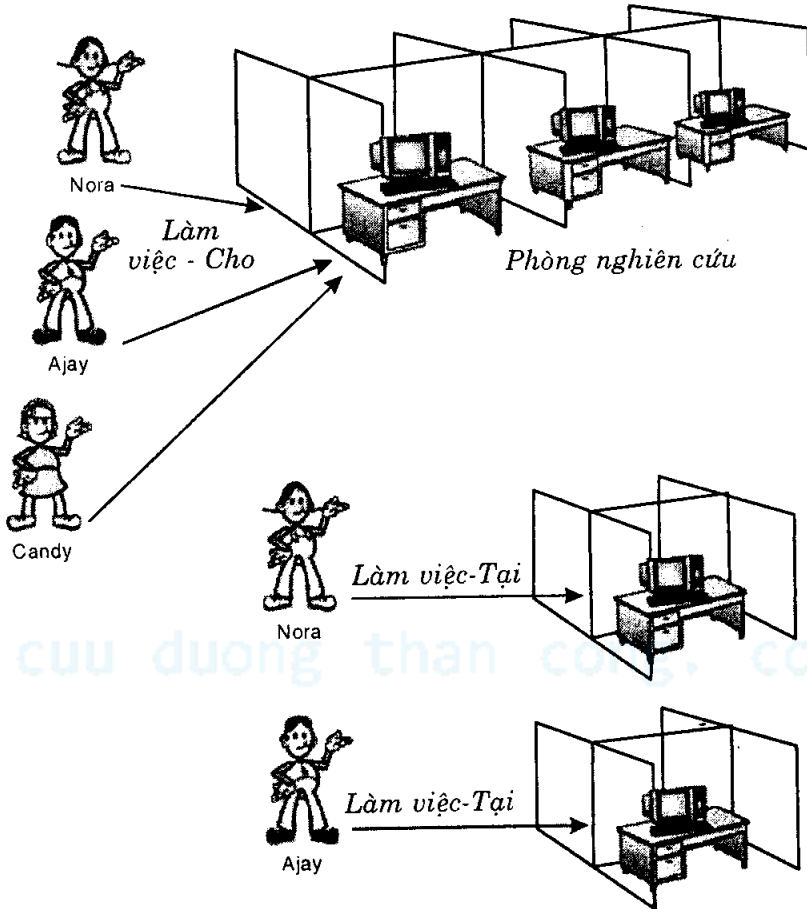
Nền tảng cơ bản nhất mà chúng ta có thể mô hình hóa đó là những *thực thể* (Entity) và các mối *quan hệ* (Relation).

Thực thể là những đối tượng trong thế giới thực mà chúng ta cất giữ thông tin của chúng bên trong cơ sở dữ liệu. Ví dụ, chúng ta có thể chọn cất giữ thông tin về nhân viên và các phòng ban mà nhân viên đó làm việc. Trong trường hợp này, *nhân viên* chính là một thực thể và *phòng ban* là một thực thể khác.

Quan hệ là những mối liên kết giữa các thực thể lại với nhau. Ví dụ, một nhân viên thì *làm việc cho* một phòng ban. *Làm việc-Cho* chính là mối quan hệ giữa thực thể *nhân viên* và thực thể *phòng ban*.

Có nhiều mối quan hệ theo cấp độ khác nhau. Chúng có thể là *một-một* (one-one), hoặc *một-nhiều* (one-many) hay *nhiều-một* (tùy thuộc vào hướng mà bạn đang nhìn vào quan hệ), thậm chí có thể là quan hệ *nhiều-nhiều* (many-many).

Một mối quan hệ một đối một (một-một) chính xác kết nối hai thực thể với nhau. Như bản thân các nhân viên trong một tổ chức công ty có mối quan hệ một-một với vị trí làm việc của mình (*Làm việc - tại*). Quan hệ *Làm việc - Cho* phòng ban nào thường là mối quan hệ một nhiều. Vì một phòng ban có thể có nhiều nhân viên làm việc. Và một nhân viên làm việc thường chỉ cho một phòng ban. Hai mối quan hệ này được biểu diễn như trong hình 1-1.



Hình 1-1: Các mối quan hệ

Chú ý rằng biểu diễn thực thể và các mối quan hệ cũng như loại quan hệ nào là phụ thuộc vào môi trường cùng những quy tắc công việc (business-rule) mà bạn đang hướng đến việc mô hình hóa chúng. Ví dụ, trong một số công ty, các nhân viên có thể làm việc cùng lúc cho nhiều phòng ban. Trong trường hợp đó, quan hệ *Làm việc - Cho* sẽ trở thành mối quan hệ *nhiều-nhiều* thay cho *một-nhiều* trước đây. Hay nếu có nhân viên nào đó chia sẻ chung nơi làm việc với nhân viên khác thì mối quan hệ *làm việc - tại* không còn là một đối một nữa mà sẽ trở thành *một-nhiều*.

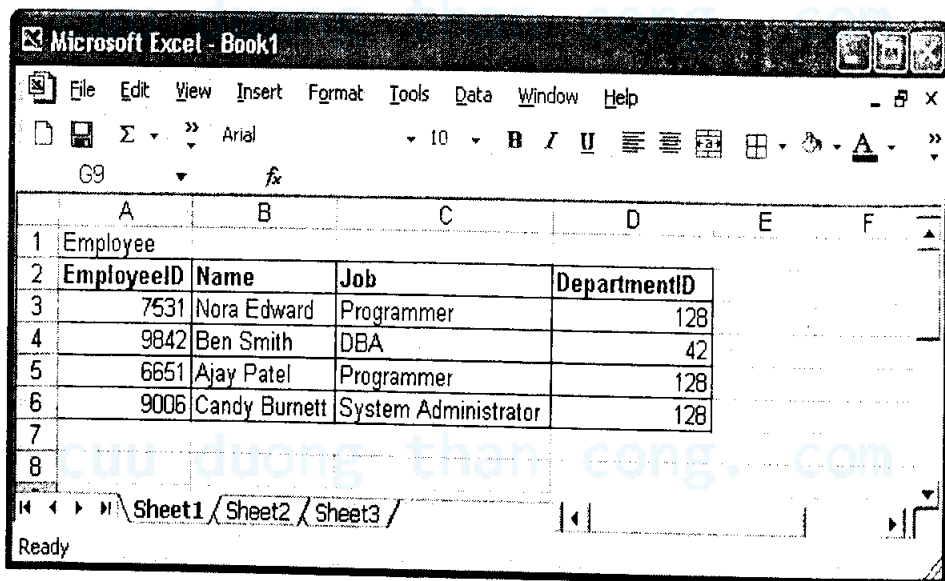
Chú ý rằng sau này bạn sẽ thấy chúng ta không thể biểu diễn quan hệ *nhiều - nhiều* trực tiếp trong một lược đồ quan hệ (Schema) cơ sở dữ liệu, vì hai bảng không thể là con của nhau. Thay vào đó, chúng ta sẽ đặt khóa ngoại trên một bảng trung gian kết hợp khác. Khi chúng ta tiếp cận đến

cách thiết kế cơ sở dữ liệu, bạn sẽ gặp tình huống này. Không có hai hệ thống nào giống nhau tuyệt đối về quan hệ và thực thể.

3. BẢNG DỮ LIỆU (TABLES)

MS Access, SQL Server, Oracle, MySQL ... là những hệ thống quản lý cơ sở dữ liệu quan hệ (RDBMS - Relation Database Management System) - hỗ trợ xây dựng mô hình dữ liệu chứa tập hợp những mối quan hệ lẫn nhau. Thuật ngữ “quan hệ” trong ngữ cảnh này không còn là quan hệ qua lại dạng “thân bằng quyền thuộc” mà là quan hệ chung với nhau qua bảng dữ liệu còn gọi là *table*. Chú ý rằng thuật ngữ bảng dữ liệu “table” và “quan hệ” hay “relation” có cùng nghĩa như nhau. Tuy nhiên trong giáo trình này, chúng ta sẽ sử dụng tên gọi bảng hay table thay cho “quan hệ” vì có thể bạn sẽ nhầm với quan hệ dạng nối kết hai thực thể với nhau.

Nếu bạn đã từng sử dụng một bảng tính của Excel chẳng hạn, thì mỗi bảng tính (Sheet) cũng chính là một bảng dữ liệu. Một bảng dữ liệu mẫu đơn giản được minh họa như trong hình 1-2.



	A	B	C	D	E	F
1	Employee					
2	EmployeeID	Name	Job	DepartmentID		
3	7531	Nora Edward	Programmer	128		
4	9842	Ben Smith	DBA	42		
5	6651	Ajay Patel	Programmer	128		
6	9006	Candy Burnett	System Administrator	128		
7						
8						

Hình 1-2

Bảng nhân viên Employee cất giữ thông tin nhân viên như mã số nhân viên IDs, tên, công việc và phòng ban mà mỗi nhân viên làm việc. Như bạn có thể thấy, bảng này cất giữ dữ liệu hay thông tin của bốn nhân viên trong một công ty.

4. CỘT (COLUMN) HOẶC THUỘC TÍNH (ATTRIBUTE)

Một bảng dữ liệu được biểu diễn bởi một hoặc nhiều cột, mỗi cột (Column) hay còn gọi là thuộc tính (Attribute) mô tả một mẫu thông tin nào đó của dữ liệu lưu trong bảng. Thuật ngữ cột (column) và thuộc tính (attribute) được sử dụng thay thế lẫn nhau và đều có cùng ý nghĩa, nhưng “cột” được sử dụng nhiều hơn khi làm việc trên cấu trúc vật lý của bảng, thuộc tính thường dùng để mô tả và biểu diễn thực thể của thế giới thực mà bảng dữ liệu đang mô hình hóa.

Trong Hình 1-2 mà bạn có thể thấy mỗi nhân viên có một mã số employeeID, một tên, một công việc, và một mã số phòng ban departmentID. Đó cũng chính là các cột của bảng nhân viên mang tên Employee (đôi khi chúng còn được gọi là thuộc tính của bảng nhân viên Employee cũng không sai).

5. HÀNG (ROW), BẢN GHI (RECORD), BỘ DỮ LIỆU (TUPLES)

Chúng ta hãy xem lại bảng nhân viên Employee. Mỗi hàng trong bảng đại diện một bản ghi (hay mẫu tin) cho từng nhân viên. Bạn có thể nghe tên gọi của chúng như hàng, bản ghi, mẫu tin, hoặc bộ dữ liệu đều như nhau cả. Mỗi hàng hay dòng trong bảng chứa các giá trị của các cột trong bảng.

6. KHÓA

Khóa là một khái niệm rất quan trọng trong việc thiết kế và xây dựng cơ sở dữ liệu. Có 5 loại khóa cơ bản:

- Siêu khóa (Super Keys),
- Khóa ứng viên (Candidate Keys)
- Khóa chính (Primary Keys),
- Khóa ngoại (Foreign Keys)
- Khóa nói chung (Keys).

Ghi chú: Ở đây chúng ta chưa đi sâu vào cách sử dụng chúng, bạn sẽ học chi tiết hơn về Khóa trong phần sau.

Siêu khóa là một (hoặc nhiều) cột được dùng để xác định và nhận dạng ra một dòng trong bảng. Một khóa nói chung (Key) là thành phần tối thiểu của Siêu khóa. Ví dụ, hãy xem bảng nhân viên.

Chúng ta có thể sử dụng cột employeeID và tên Name để cùng kết hợp xác định và nhận ra bất kỳ dòng hay hàng dữ liệu nào trong bảng. Chúng ta cũng có thể sử dụng tập hợp của tất cả các cột như sau để dùng làm khóa nhận dạng:

(employeeID, name, job, departmentID).

Chúng chính là những *Siêu khóa*. Tuy nhiên, chúng ta không cần tất cả các cột đó để xác định hay dùng để nhận dạng ra một dòng trong bảng. Chúng ta chỉ cần duy nhất (ví dụ) employeeID. Đây là thành phần tối thiểu hay nhỏ nhất của Siêu khóa. Có thể nói nó là tập hợp nhỏ nhất của các cột có thể sử dụng để xác định ra một dòng dữ liệu trong bảng và như vậy, employeeID là một khóa.

Nào hãy xem lại bảng nhân viên lần nữa. Chúng ta có thể xác định một nhân viên bởi tên (name) hoặc bởi mã số nhân viên employeeID. Cả hai cột này đều có thể dùng làm khóa. Chúng ta gọi chúng là những khóa ứng viên (Candidate key) vì chúng sẽ là những ứng cử viên để ta dùng chọn khóa chính (Primary key) sau này.

Khóa chính (Primary Key) là một cột hoặc tập hợp nhiều cột mà chúng ta sẽ sử dụng để xác định hoặc nhận dạng ra một dòng dữ liệu từ một bảng. Trong trường hợp này chúng ta sẽ chọn employeeID làm khóa chính. Điều này sẽ tốt hơn là chọn cột Name làm khóa chính, đơn giản là có thể có hai người trùng tên với nhau.

Khóa ngoại (Foreign Keys) là cột chứa dữ liệu đại diện cho việc liên kết giữa các bảng. Ví dụ, nếu nhìn lại Hình 1-2, bạn sẽ thấy cột departmentID giữ mã số phòng ban. Đây chính là một khóa ngoại: tập hợp đầy đủ thông tin về mỗi phòng ban sẽ được giữ trong một bảng riêng biệt, với departmentID là khóa chính của bảng đó.

7. PHỤ THUỘC HÀM

Thuật ngữ phụ thuộc hàm này bạn sẽ ít nghe nói đến hơn các thuật ngữ được đề cập trên đây, nhưng bạn cũng cần hiểu nó để nắm được quá trình chuẩn hóa mà chúng ta sẽ tiếp cận trong phần sau.

Nếu có một phụ thuộc hàm giữa cột A và cột B trong một bảng dữ liệu cho trước, chúng ta có thể viết $A \rightarrow B$, nghĩa là nếu có giá trị của cột A ta

sẽ xác định được giá trị của cột B. Ví dụ, trong bảng nhân viên, nếu bạn biết được thông tin cột employeeID sẽ là cột xác định ra cột tên name (cũng như tất cả các cột khác của bảng) hay nói cách khác employeeID → Name.

8. LƯỢC ĐỒ

Thuật ngữ lược đồ (Schema) hoặc lược đồ cơ sở dữ liệu đơn giản có nghĩa là cấu trúc hoặc thiết kế của cơ sở dữ liệu – nó chỉ là bộ khung của cơ sở dữ liệu và không có bất kỳ dữ liệu nào trong đó. Chúng ta có thể mô tả lược đồ cho một bảng đơn theo cách sau:

Bảng: employee(employeeID, name, job, departmentID)

9. NHỮNG NGUYÊN LÝ THIẾT KẾ CƠ SỞ DỮ LIỆU

- Đặt kế hoạch.
- Suy nghĩ trước.

Có lẽ điều quan trọng nhất khi bạn bắt đầu bắt tay vào thiết kế hay xây dựng một cơ sở dữ liệu là cần suy nghĩ cẩn thận tất cả những gì mình sẽ phải làm. Việc mô hình hóa và xây dựng một cơ sở dữ liệu hoàn toàn có thể độc lập với các hệ quản trị dữ liệu, thậm chí bạn có thể xây dựng mô hình dữ liệu mà không cần tiếp cận với máy tính. Hãy suy nghĩ những kiểu thông tin mà bạn muốn thể hiện và những loại câu hỏi hay truy vấn mà bạn sẽ muốn cơ sở dữ liệu của bạn trả lời. Hai câu hỏi chính cụ thể như sau:

- Thông tin nào cần cất giữ hoặc các thực thể nào mà chúng ta cần cất giữ thông tin của chúng?

Những câu hỏi nào mà chúng ta muốn cơ sở dữ liệu trả lời sau khi đã có dữ liệu?

Khi bắt đầu trả lời cho những câu hỏi này, bạn cần phải luôn nhớ các quy tắc hoạt động của thế giới thực mà mình đang muốn mô hình hóa, các mối liên kết giữa thực thể và dữ liệu. Một chu trình thiết kế tiêu biểu được xác lập như sau:

Xác định dữ liệu → Đặt kiểu dữ liệu → Chuẩn hóa các bảng/xây dựng khóa → Tối ưu hóa /lập lại.

Chúng ta hãy quay lại vấn đề quản lý dữ liệu danh bạ điện thoại. Rất đơn giản, tất cả những gì bạn cần là tên (Name), số điện thoại (Phone

Number) và địa chỉ (Address). Bắt đầu bằng cách ta sẽ xây dựng một bảng dữ liệu bao gồm 3 trường thông tin: *tên*, *địa chỉ* và *số điện thoại*. Và như vậy chúng ta có cấu trúc sau:

Name

Address

Phone Number

Gần như đã khá cụ thể. Tuy nhiên, chúng ta cần một tiện ích là tên (Name) có thể sắp xếp theo thứ tự, bạn có thấy đúng như vậy không? Vâng, đúng vậy nhưng bạn muốn sắp xếp thứ tự danh bạ theo tên hay theo họ? Để trả lời, cách đơn giản và dễ hiểu nhất là tách tên tổng quát (Name) ra làm hai phần (hay cột) là Họ (Initial) và Tên chính (Last Name). Tương tự như vậy cho thông tin về địa chỉ. Một địa chỉ chung chung như Address có thể là đã đủ nhưng ta cần chi tiết hóa thông tin địa chỉ để sau này dễ tìm kiếm, về nguyên tắc địa chỉ có thể được cụ thể hóa thành:

Address { *Street (Đường)*
City (Thành phố)
State (Vùng)
Postal Code (Mã vùng)

Cách tốt nhất để xem công việc thiết kế cơ sở dữ liệu làm việc như thế nào là kiểm tra nó với một vài dữ liệu mẫu, dưới đây là một số dòng dữ liệu mà bạn có thể hình dung:

NAME	ADDRESS	PHONE
Jay	T. Apples 100 Megalong Dr Etna	4992122
Beth York	2/53 Alice Lebanon	5050011
Mike. R. Sullivan	9 Jay Lebanon	4893892
Barry J. Anderson	71 Wally Rd Hanover	2298310

Bây giờ nếu sắp xếp dữ liệu trong bảng lại bạn sẽ được danh sách thông tin sau:

NAME	ADDRESS	PHONE
Barry J. Anderson	71 Wally Rd Hanover	2298310
Beth York	2/53 Alice Lebanon	5050011

Jay T. Apples	100 Megalong Dr Etna	4992122
Mike. R. Sullivan	9 Jay Lebanon	4893892

Ngay lập tức, bạn có thể thấy xuất hiện điều mình không mong muốn. Bạn muốn danh sách trong bảng được sắp xếp theo thứ tự abc của tên cuối chứ không phải họ đặt ở phía trước, chính xác sắp theo tên cuối phải là:

NAME	ADDRESS	PHONE
Barry J. Anderson	71 Wally Rd Hanover	2298310
Jay T. Apples	100 Megalong Dr Etna	4992122
Mike. R. Sullivan	9 Jay Lebanon	4893892
Beth York	2/53 Alice Lebanon	5050011

Chúng ta giải quyết như thế nào? Có 2 cách, thứ nhất bạn vẫn có thể giữ nguyên nội dung của cột Name nhưng khi trích rút dữ liệu bạn sẽ sử dụng một số hàm xử lý chuỗi đặc biệt để phân tách tên và họ, sau đó mới sắp xếp (Cách này bạn sẽ tiếp cận khi chúng ta học về ngôn ngữ truy vấn dữ liệu SQL). Cách thứ hai đơn giản hơn là bạn có thể thiết kế lại cấu trúc của bảng dữ liệu bằng cách tách cột tên ra làm 3 phần. Tên (First Name), Họ (Last Name) và chi tiết nữa là tên lót theo họ (Middle name). Bây giờ là cấu trúc bảng của chúng ta sẽ bao gồm các cột:

LastName

FirstName

Mid

Address

Phone

Lần này nếu sắp xếp danh sách theo cột LastName bạn sẽ có kết quả theo thứ tự ABC mong muốn:

LastName	FirstName	Mid	Address	Phone
Anderson	Barry	J.	71 Wally Rd Hanover	2298310
Apples	Jay	T.	100 Megalong Dr Etna	4992122
Sullivan	Mike.	R.	9 Jay Lebanon	4893892
York	Beth		2/53 Alice Lebanon	5050011

Chưa hết, chúng ta còn có thể đi xa hơn nữa. Bảng dữ liệu có thể thậm chí còn hiệu quả hơn nếu bạn biết cách chi tiết cấu trúc của nó sâu hơn nữa. Ví dụ, nếu bạn muốn dễ dàng liệt kê ra chỉ những người nào sống tại thành phố Leichhardt, thì thiết kế này sẽ không giúp được bạn ngay tức thời (trừ khi có sự hỗ trợ của các hàm xử lý chuỗi của các hệ quản trị dữ liệu). Tuy nhiên nếu chịu khó tốn ít công sức nữa, bạn có thể bẻ gãy cấu trúc cơ sở dữ liệu cột Address thành 2 cột: Street (tên đường), city (thành phố) như đã nêu trước đây. Bây giờ cấu trúc bảng của chúng ta sẽ như sau:

FirstName
Mid
LastName
Street
City
Phone

LastName	FirstName	Mid	Street	City	Phone
Anderson	Barry	J.	71 Wally Rd	Hanover	2298310
Apples	Jay	T.	100 Megalong Dr	Etna	4992122
Sullivan	Mike.	R.	9 Jay	Lebanon	4893892
York	Beth		2/53 Alice	Lebanon	5050011

Với cấu trúc này, bạn có thể sắp xếp cơ sở dữ liệu của mình theo thứ tự abc bằng tên họ hoặc số điện thoại và bạn cũng sẽ có thể biết hoặc truy tìm ra nhanh chóng tất cả người nào đó sống trong một thành phố hay con đường bất kỳ.

Hy vọng ví dụ trên có thể cho bạn thấy khi tạo ra một cơ sở dữ liệu nào đó điều đầu tiên chúng ta cần làm là có những bước phân tích và lập kế hoạch trước.

Bạn cần xem xét các thông tin mình muốn cất giữ và những cách thức mà mình muốn truy tìm lại thông tin đó. Công việc này không đòi hỏi bạn phải tiếp xúc với bất kỳ chương trình quản trị dữ liệu nào cả. Nó không đòi hỏi bạn làm việc trên máy tính mà là làm việc chỉ với cây bút và tờ giấy.

Cách bạn xây dựng cấu trúc dữ liệu sẽ ảnh hưởng đến tất cả những tương tác sau này của bạn với cơ sở dữ liệu đó. Nó cũng sẽ xác định việc đưa thông tin vào cơ sở dữ liệu có dễ dàng hay không; loại bỏ được dữ liệu trùng lặp và bảo đảm rằng buộc toàn vẹn dữ liệu; và sau cùng việc trích rút thông

tin cần thiết cho bạn có thể thực hiện đơn giản hay sẽ trở nên phức tạp. Một cấu trúc cơ sở dữ liệu không tốt sẽ gây rất nhiều khó khăn khi bạn muốn trích rút lại thông tin lưu trong đó để sử dụng. Bảng dữ liệu trên có thể được cất giữ trong một bảng đơn duy nhất (đôi khi thuật ngữ còn gọi là database phẳng - flat database), nhưng sẽ có nhiều thuận lợi cũng như tiện dụng hơn nếu bạn tiếp tục phân tích cấu trúc dữ liệu và tách chúng ra thành nhiều bảng thay vì một bảng, chúng ta bắt đầu tiếp cận với loại cơ sở dữ liệu quan hệ từ đây. Cơ sở dữ liệu quan hệ như bạn sẽ tiếp tục nghiên cứu trong phần dưới đây, có thể cung cấp một sức mạnh và tính linh hoạt vượt bậc trong việc cất giữ và trích rút thông tin.

Bạn hãy xem một ví dụ về cơ sở dữ liệu chứa các thông tin đĩa CD dưới đây để bắt đầu có một khái niệm về cơ sở dữ liệu quan hệ (Relational Database). Khó khăn và tính không đúng đắn thường phát sinh từ cơ sở dữ liệu phẳng chỉ có một bảng. Cơ sở dữ liệu quan hệ chính là một cứu cánh cũng như hướng đi làm thay đổi cả thế giới!

Khi bạn lần đầu tiên tổ chức dữ liệu, file dữ liệu đơn chỉ có một bảng là cấu trúc đơn gian và dễ hiểu nhất. Chúng được gọi là cơ sở dữ liệu file đơn chiếc (Single-database file hay flat database) vì chỉ có một bảng dữ liệu lưu trên một file. Khi sử dụng lâu dài, loại cơ sở dữ liệu đơn phẳng này sẽ gây không ít phiền phức vì phải thực hiện nhiều thao tác xử lý phức tạp. Cũng không phải cơ sở dữ liệu đơn sẽ không làm việc tốt nhưng trong nhiều trường hợp, sử dụng cơ sở dữ liệu quan hệ tổ chức làm nhiều bảng sẽ giúp bạn giải quyết nhiều vấn đề dễ dàng hơn. Ví dụ, nếu bạn tạo ra một file dữ liệu đơn để lưu danh sách các đĩa CDs nhạc. Bạn phải đặt tất cả các thông tin chi tiết, như thông tin ca sĩ, tác giả vào trong một bảng. Và tiếp theo là những thông tin như tựa (title) của CD, tên nhóm nhạc, ban nhạc, nơi sản xuất, các ghi chú ... Khi đó cấu trúc dữ liệu của bạn có thể sẽ như sau:

cd_name
cd_date
genre
tracknumber
trackname
artist_or_band_name
band_members
recording_label
discography
notes

Với mỗi CD của ban nhạc Beatles mà bạn sở hữu, bạn sẽ phải nhập tất cả các thông tin trên! Điều này có nghĩa bạn sẽ phải nhập vào tất cả tên các thành viên của ban nhạc Beatles nhiều lần (tương ứng với từng CD). Quả thật phiền phức!

Khi tổ chức thành nhiều bảng dữ liệu mọi chuyện sẽ dễ chịu hơn. Nếu bạn sử dụng cơ sở dữ liệu quan hệ và tổ chức cấu trúc dữ liệu thành nhiều bảng, bạn có thể cất giữ thông tin chi tiết của CD (tên, ngày tháng, các bài hát trong một bảng CD Table) và cất giữ thông tin chi tiết về ca sĩ riêng trong bảng Artist khác. Bảng CD của bạn sẽ có cấu trúc như sau:

```
cd_name  
cd_date  
genre  
tracks  
artist_or_band_name
```

Bảng danh sách các ca sĩ sẽ cấu trúc gồm các thông tin:

```
artist_or_band_name  
band_members  
recording_label  
discography  
notes
```

Sau đó bạn liên kết hai bảng thông qua tên nghệ sĩ /hoặc tên ban nhạc (artist_or_band_name), đó là lý do vì sao chúng ta gọi cơ sở dữ liệu này là cơ sở dữ liệu quan hệ – bạn định nghĩa mối quan hệ giữa các bảng đóng vai trò là khóa ngoại (Foreign Key) và nhập vào tên các ca sĩ của từng ban nhạc chỉ một lần duy nhất. Mỗi khi bạn thêm thông tin của đĩa nhạc Beatles CD vào bộ sưu tập của mình, bạn chỉ cần nhập tên ban nhạc Beatles trong cột dữ liệu artist và cơ sở dữ liệu sẽ truy tìm chi tiết của ban nhạc Beatles trong bảng Artist giúp bạn. Nó không chỉ giảm thiểu công sức nhập liệu của bạn, mà còn bảo đảm sự toàn vẹn thông tin và hạn chế khả năng nhập vào dữ liệu thừa cũng như không đúng.

Để tiện hơn nữa bạn có thể tiếp tục tạo ra thêm bảng chứa danh mục bài hát Songs như cấu trúc sau:

```
cd_name  
song_title
```


duration
track_number
writer
vocals

và liên kết bảng này với bảng CD dựa vào cột `cd_name` đóng vai trò khóa ngoại. Tốt hơn nữa bạn có thể dùng `CD_ID` làm khóa chính (Primary Key) cho bảng CD và `CD_ID` làm khóa ngoại cho bảng Songs thay vì `cd_name`, vì nếu tình ý bạn có thể nhận ra 2 bộ sưu tập khác nhau có thể có 2 đĩa CD trùng tên. Giờ đây bạn đã có thể lưu đầy đủ thông tin về bộ sưu tập CD của mình.

Bạn sẽ học cách thiết kế dữ liệu tốt hơn trong chương sau khi chúng ta học về quy trình chuẩn hóa dữ liệu. Trong chương mở đầu này có lẽ như vậy là đủ.

10. KIỂU DỮ LIỆU

Kiểu dữ liệu (Type) thể hiện tính chất loại dữ liệu mà bạn muốn lưu, như dữ liệu là số, chuỗi, ngày tháng ...

Chuẩn SQL định nghĩa một số kiểu dữ liệu chuẩn và đa số các nhà cung cấp phần mềm quản trị cơ sở dữ liệu đều hỗ trợ những kiểu dữ liệu này trong sản phẩm của mình.

10.1. Chuỗi (String) và số (Number)

Nói chung kiểu số dùng biểu diễn số (nguyên, thập phân, tiền tệ) – bạn chỉ cần lựa chọn một phạm vi cần thiết đủ lớn để chứa giá trị tối đa có thể có của thông tin đang nhắm đến là đủ.

Chuỗi là dạng dữ liệu văn bản hay ký tự như nội dung một thông tin mô tả hay tên họ, tựa bài hát ... Với chuỗi đơn giản bạn chỉ cần xác định độ dài tối đa cần có của nội dung nhập vào. Ví dụ như tên thì chỉ dài tối đa 20 ký tự, tựa bài hát thì không nên dài hơn 80 ký tự.

10.2. Kiểu ngày tháng (thời gian) và tiền tệ

Ngày tháng (date/time) là kiểu dữ liệu dùng biểu diễn ngày tháng và thời gian (gồm cả giờ phút giây). Hầu hết các hệ quản trị dữ liệu đều hỗ trợ kiểu ngày tháng thống nhất theo chuẩn SQL. Ví dụ bạn có thể nhập vào thông tin thể hiện ngày sinh của nhân viên, ngày nhập hàng vào kho, ngày kết thúc hợp đồng... Trong thế giới thực, bất kỳ thông tin gì lưu trữ lại đều

có khả năng liên quan đến mốc thời gian và ngày tháng là kiểu dữ liệu rất có ích để bạn biểu diễn chúng.

Kiểu tiền tệ thật ra chỉ là kiểu số thập phân thông thường nhưng đòi hỏi phải có độ chính xác cao. Trong các chương trình liên quan đến xử lý số thập phân thì công việc làm tròn số đôi khi dẫn đến một sai số khá lớn, ví dụ nếu kiểu dữ liệu lưu trữ tiền tệ bạn chỉ gồm 2 số thập phân (0.01), khi số tiền thanh toán là lẻ ở hàng 3 hay 4 số thập phân (0.0001 chẳng hạn) nếu làm tròn và bỏ đi phần dư của hai số thập phân cuối thì khi số tiền tổng cộng khá lớn sẽ khiến con số làm tròn mất đi rất nhiều (như $50,000,000đ \times 0.0001 = 5,000đ$). Chính vì vậy kiểu tiền tệ giúp bạn thể hiện số lưu trữ với độ chính xác rất cao.

10.3. Các kiểu dữ liệu phức hợp

Cuối cùng, trong thế giới thực có những kiểu dữ liệu phức hợp như số điện thoại, địa chỉ, thông tin liên lạc, mã số thẻ tín dụng... Những kiểu dữ liệu này xuất hiện rất nhiều trong hầu hết các lược đồ cơ sở dữ liệu. Thường những mẫu thông tin này được tổ chức truy nhập từ nhiều bảng. Chẳng hạn trong một hệ thống thương mại điện tử eCommerce, cơ sở dữ liệu cùng một thông tin liên lạc có thể cất giữ và phân loại khác nhau theo nhóm người dùng, nhà cung cấp, kho hàng, hay nhà quản trị admin.

Thay vì lưu thành từng bảng riêng địa chỉ tương ứng người dùng, nhà cung cấp, hoặc nhà kho... (dẫn đến lặp lại rất nhiều những cột địa chỉ trong toàn bộ cơ sở dữ liệu) như trên, chúng ta có thể thiết lập một bảng đơn duy nhất chứa thông tin liên lạc và tạo một khóa ngoại cho bảng. Sau đó thông tin của bảng sẽ được các bảng khác tham chiếu đến thông qua khóa ngoại. Điều này sẽ tạo nên hai lợi ích tức thời:

- Dễ dàng thay đổi những thông tin chính của quan hệ.
- Dễ dàng thay đổi những kiểu cấu trúc dữ liệu phức tạp sẽ xảy ra trong tương lai.

Đoán trước tập những thuộc tính hay cột dữ liệu nào (hình thành nên cấu trúc phức hợp) sẽ thay đổi trong tương lai khi thiết kế cơ sở dữ liệu đôi khi là cả một nghệ thuật. Cấu trúc địa chỉ Address có thể được tách ra chi tiết như sau:

Department

Company

MailStop

AddressLine1

AddressLine2

AddressLine3

City

State

PostalCode

Country

Thông tin liên lạc đầy đủ còn có thể bao gồm thêm những thuộc tính sau:

Title

FirstName

MiddleName

LastName

Suffix

HomeAddress

WorkAddress

HomePhone

WorkPhone

CellPhone

Fax

Pager

Email

Cuối cùng, ngay cả số điện thoại bạn cũng có thể phân thành cấu trúc sau:

CountryCode

AreaCode

ExchangeCode

LineNumber

Extension

Chẳng hạn bạn có thể có một số điện thoại như 84-08-8501232-123-7 trong đó 84 là mã quốc gia (Việt nam), 08 là mã vùng (Thành phố HCM), 8501232 là số điện thoại công ty, 123 là số phòng kế toán, và cuối cùng 7 là số máy ngay bàn làm việc của bạn. Bạn có thể lưu số này thành một chuỗi nhưng cũng có thể lưu số này tách rời ra thành nhiều cột.

10.4. Dữ liệu “nhạy cảm”

Bất kỳ dữ liệu “nhạy cảm” (sensitive data) nào lưu trong cơ sở dữ liệu đều cần phải được mã hóa. Dữ liệu nhạy cảm là những dữ liệu mang tính riêng tư cao, ví dụ như số tài khoản ngân hàng, số thẻ tín dụng, mật mã truy nhập hệ thống... Ngay cả khi hệ thống cơ sở dữ liệu của bạn được xem là đảm bảo cơ chế bảo mật nhưng bạn cũng cần có cơ chế mã hóa cho riêng mình. Ví dụ nổi tiếng nhất về quản lý loại dữ liệu này là hệ thống mật khẩu của hệ điều hành Unix. Nội dung mật khẩu được lưu ngay trong file văn bản nhưng nội dung đã mã hóa khiến cho dù có, bạn mở file và đọc được nội dung này thì cũng không hiểu được ý nghĩa của nó.

Có nhiều cách mã hóa nhưng nhìn chung là có hai hướng chính, mã hóa hai chiều và mã hóa dữ liệu một chiều. Một số dữ liệu như số thẻ tín dụng cần mã hóa theo kiểu 2 chiều tức là có thể khôi phục lại nội dung từ dữ liệu mã hóa; Mã hóa một chiều là sử dụng những thuật toán khiến dữ liệu không thể giải mã trở lại nội dung ban đầu được. Ví dụ như thông tin về mật khẩu thường được mã hóa theo kiểu một chiều.

Tóm lại, mỗi cột trong bảng dữ liệu sẽ có một kiểu dữ liệu riêng. Kiểu dữ liệu định nghĩa kiểu thông tin mà cột có thể cất giữ. Phần lớn các cột có kiểu dữ liệu văn bản (text). Các cột kiểu văn bản có thể cất giữ những thông tin số, chữ cái, các ký tự đặc biệt hay thậm chí nội dung đầy đủ của một tài liệu. Những kiểu dữ liệu thông dụng khác gồm có kiểu số (còn gọi là number), tiền tệ (một dạng kiểu số có độ chính xác thập phân cao), kiểu ngày tháng/thời gian, và kiểu luận lý Boolean (còn gọi là kiểu Yes/No), kiểu memo (để cất nội dung văn bản) và kiểu picture hay binary chứa dữ liệu nhị phân (dữ liệu dạng số hóa như hình ảnh...)

Không phải hệ quản trị cơ sở dữ liệu nào cũng hỗ trợ đầy đủ những kiểu dữ liệu này. Tuy nhiên bốn kiểu cấu trúc dữ liệu đơn giản: văn bản, số, Boolean và ngày tháng thì hầu như luôn luôn có.

Kiểu Boolean còn gọi là kiểu lôgic có thể chứa giá trị 0 hoặc 1, hoặc cặp giá trị như True/False hay Yes/No. Kiểu này hữu ích khi bạn muốn biểu diễn các thông tin chỉ có hai trạng thái như Nam hay Nữ, Được phép hay Không...

Chú ý, chúng ta sử dụng kiểu dữ liệu văn bản để biểu diễn số điện thoại lẫn mã vùng. Tại sao không sử dụng kiểu dữ liệu số? Với số điện thoại, câu trả lời hiển nhiên là: số điện thoại thường chứa những ký tự đặc biệt như dấu ngoặc hay dấu nối. Ví dụ: (02) 4782-03021. Sử dụng kiểu dữ liệu văn bản chúng ta có thể cho phép lưu những ký tự này còn nếu dùng kiểu số thì không thể. Với trường hợp mã số vùng, mặc dù thông tin này chỉ chứa các con số, nhưng chúng ta cũng không xem chúng là số mà nên lưu dạng văn bản, đơn giản là vì chúng không dùng để đem ra tính toán.

11. KÍCH THƯỚC CỘT DỮ LIỆU (FIELD SIZE)

Điều quan trọng nhất khi định kích thước cho cột dữ liệu đó là xác định phạm vi đủ lớn để có thể cất giữ mọi trường hợp xảy ra của thông tin. Ví dụ như trường tên Name và địa chỉ Address, có thể đoán ra tên không quá 50 ký tự còn địa chỉ cỡ 100 chữ là tối đa. Tuy nhiên sẽ có những trường hợp thật sự bạn không biết chắc chắn chiều dài cột dữ liệu bao nhiêu là đủ. Khi gặp trường hợp này tốt nhất là hãy để kích thước cột dữ liệu bằng với số tối đa mà một hệ quản trị dữ liệu cho phép (thường là 255 hoặc 254 ký tự). Một số hệ quản trị như SQL Server hay Oracle còn cho phép bạn sử dụng các kiểu như NVARCHAR hay NTEXT không cần chỉ định kích thước. Nội dung lưu trữ sẽ được các hệ quản trị dữ liệu tự động tùy chỉnh.

12. TÊN CỘT DỮ LIỆU

Bạn sẽ thấy tên các cột dữ liệu thường đặt ghép liền với nhau như FirstName hay MembershipType. Đây là điều nên làm. Tại sao không viết thêm khoảng trắng để chúng dễ đọc hơn?

Mặc dầu hiện nay hầu hết các chương trình quản trị cơ sở dữ liệu đều cho phép bạn tạo ra tên trường hay cột dữ liệu được phép có khoảng trắng nhưng còn rất nhiều thư viện cũng như ngôn ngữ lập trình chưa hỗ trợ việc truy xuất tên cột có khoảng trắng. Do vậy tốt nhất là dùng nên dùng trừ khi bạn biết rõ mình sẽ sử dụng thư viện truy xuất hỗ trợ cơ chế đặt tên trường có khoảng trắng.

13. ĐỊNH DẠNG VÀ KIỂM TRA TÍNH HỢP LỆ CỦA DỮ LIỆU

Một số định dạng và quy tắc kiểm tra dữ liệu được các hệ quản trị áp đặt mặc định trên kiểu dữ liệu mà bạn chọn. Ví dụ kiểu số thì không được

có những ký tự chữ cái, chuỗi nhập vào không vượt quá kích thước định trước. Bằng cách này dữ liệu của bạn đảm bảo tính đúng đắn khi đưa vào lưu trữ. Có nhiều cấp độ kiểm tra dữ liệu, một số hệ quản trị cho phép bạn tạo các quy tắc kiểm tra (Rule) ngay khi dữ liệu đưa vào. Một số ứng dụng có thể tự kiểm tra và bảo đảm dữ liệu đúng khuôn dạng trước khi đưa vào lưu trữ trong các bảng của cơ sở dữ liệu.

cuu duong than cong. com

cuu duong than cong. com

Chương 2:

CHUẨN HÓA CƠ SỞ DỮ LIỆU

Các vấn đề chính sẽ được đề cập:

- ✓ Phụ thuộc hàm và quan hệ giữa các thuộc tính, khóa.
- ✓ Chuẩn hóa dạng 1 (1NF).
- ✓ Chuẩn hóa dạng 2 (2NF).
- ✓ Chuẩn hóa dạng 3 (3NF).
- ✓ Chuẩn hóa dạng 4 (4NF).

Một trong những nhân tố quan trọng nhất trong thiết kế cơ sở dữ liệu là việc định nghĩa. Nếu cấu trúc các bảng của bạn không được thiết lập đúng hoặc hợp lý thì có thể khiến bạn nhức đầu khi xử lý và truy vấn dữ liệu từ các bảng. Khi hiểu rõ về các mối quan hệ dữ liệu và quy tắc chuẩn hóa dữ liệu, bạn sẽ thiết kế dữ liệu tốt hơn, tạo cơ sở cho việc chuẩn bị và phát triển ứng dụng tiếp theo sau này.

Một cơ sở dữ liệu được thiết kế tốt là cơ sở dữ liệu hạn chế tối đa việc dư thừa dữ liệu nhưng vẫn không làm mất đi bất kỳ dữ liệu nào. Có nghĩa là chúng ta sử dụng không gian lưu trữ trong cơ sở dữ liệu ít nhất nhưng vẫn bảo đảm tất cả mối liên kết cùng với nội dung dữ liệu. Hơn nữa, chuẩn hóa lược đồ cơ sở dữ liệu sẽ tránh được những dị thường trong thao tác chèn, cập nhật, hoặc xóa dữ liệu sau này và do đó bảo đảm tính toàn vẹn trong cơ sở dữ liệu.

Cho dù bạn làm việc với hệ quản trị cơ sở dữ liệu nào đi chăng nữa (MySQL, SQL Server, Access, Oracle...), bạn đều cần phải biết rõ phương pháp chuẩn hóa các bảng trong hệ thống cơ sở dữ liệu quan hệ của mình. Nó sẽ giúp bạn có được một cơ sở dữ liệu dễ hiểu, dễ truy nhập và mở rộng hơn. Trong một số trường hợp, nó còn làm tăng tốc độ của ứng dụng khi truy xuất cơ sở dữ liệu.

Về cơ bản, các quy tắc chuẩn hóa buộc loại bỏ các dư thừa dữ liệu và những quan hệ phụ thuộc mâu thuẫn nhau giữa các bảng. Trong ví dụ sau chúng ta sẽ khảo sát các bước chuẩn hóa để tạo ra một cơ sở dữ liệu hiệu quả và đầy đủ chức năng.

Chúng tôi sẽ trình bày chi tiết những kiểu quan hệ mà cấu trúc dữ liệu của bạn có thể sử dụng.

1. PHỤ THUỘC HÀM

Trước khi chúng ta bước vào quá trình chuẩn hóa, bạn cần biết rằng chuẩn hóa không phải là đặc thù với một kiểu cơ sở dữ liệu nào cả. Những quy tắc chuẩn hóa có thể áp dụng cho hầu hết các hệ thống quản trị cơ sở dữ liệu như MySQL, SQL Server, Oracle, Access...

Trước hết, chúng ta quay lại một chút về phụ thuộc hàm đã nêu ở chương trước, đây là yếu tố quan trọng nhất trong quá trình xử lý chuẩn hóa. Thuật ngữ “phụ thuộc hàm” tuy khó hiểu nhưng nó lại dùng diễn đạt cho ý tưởng hết sức đơn giản. Để minh họa, bạn hãy xem một bảng dữ liệu mẫu dưới đây:

Name	Pay_Class	Rate
Ward	1	.05
Maxim	1	.05
Cane	1	.05
Beechum	2	.07
Collins	1	.05
Cannery	3	.09

Trong đó Name là tên nhân viên, Pay_Class là loại xác định giá thanh toán và Rate là tỉ giá cần thanh toán.

Định nghĩa: cột A phụ thuộc hàm vào cột B, nếu có một giá trị bất kỳ của A ta sẽ xác định ra giá trị duy nhất khác của cột B.

Trong ví dụ trên, trường Rate là phụ thuộc hàm vào trường Pay_Class. Nói cách khác trường Pay_Class xác định Rate. Để xác định phụ thuộc hàm, bạn có thể nghĩ đơn giản như sau: cho một giá trị của cột A, hãy xác định giá trị đơn duy nhất tương ứng của cột B? Nếu B suy ra từ A, ta nói A là phụ thuộc hàm xác định B. Với bảng dữ liệu trên chúng ta thêm vào các cột như sau:

Name	Sales_Rep_Number	Pay_Class	Rate	Soc.Sec. no.
Ward	001	1	.05	133-45-6789
Maxim	002	1	.05	122-46-6889
Cane	003	1	.05	123-45-6999
Beechum	004	2	.07	113-75-6889
Collins	005	1	.05	121-44-6789
Cannery	006	3	.09	111-45-9339

Trong đó Sales_Rep_Number là mã số nhân viên bán hàng, Soc.Sec.no. là số bảo hiểm an ninh xã hội (hay số CMND) của nhân viên bán hàng. Bây giờ, hãy xem bảng trên để tìm ra một số phụ thuộc hàm khác. Bạn đã biết Pay_Class xác định Rate. Chúng ta cũng có thể nói rằng Sales_Rep_Number xác định cột Name. Một mã số Sales_Rep_Number có tương ứng duy nhất một giá trị Name duy nhất. Nó hoàn toàn phù hợp với định nghĩa của phụ thuộc hàm. Tuy nhiên cột Name có thể dùng xác định được giá trị của cột nào khác hay không? Thoạt nhìn bạn có thể nói là có, tuy nhiên sự thật là không. Thường, bạn có thể nói với tên Ward sẽ chỉ ra được một giá trị tương ứng của cột Sales_Rep_Number, tuy nhiên nếu có một người khác trùng tên Ward luôn thì sao? Khi đó cùng một tên Ward bạn xác định hai giá trị Sales_Rep_Number khác nhau. Và do đó Name không có chức năng dùng xác định được cột nào cả.

2. KHÓA VÀ PHỤ THUỘC HÀM

Bạn đã biết phụ thuộc hàm là gì, chúng ta có thể tìm hiểu thêm chi tiết về khóa (Keys) đã được giới thiệu qua ở chương trước. Nếu bạn đã từng làm việc với các hệ cơ sở dữ liệu, ắt hẳn bạn có lẽ đã biết đến thuật ngữ khóa chính (Primary Key). Thế nhưng, bạn có thể định nghĩa Primary Key là gì không?

Định nghĩa: Cột A là khóa chính cho bảng T nếu:

Thuộc tính 1: Tất cả các cột trong T phụ thuộc hàm vào A.

Thuộc tính 2: Không có một cột hay tập hợp cột nào trong bảng T cùng có *Thuộc tính 1*.

Đễ hiểu, nếu tất cả các trường hay cột trong một bảng cơ sở dữ liệu phụ thuộc vào một và chỉ một cột (hay tập hợp cột) A trong bảng, thì A chính là khóa của bảng.

Đôi khi *Thuộc tính 2* trong định nghĩa trên bị vi phạm, và tồn tại hai cột đều có khả năng làm ứng cử viên cho khóa chính. Những khóa này được gọi là khóa ứng viên hay Candidates Key. Từ những khóa ứng viên này, ta chọn ra một khóa làm khóa chính (Primary Key), cái còn lại gọi là khóa thay thế (Alternate Key). Ví dụ, trong cùng bảng dữ liệu ở trên:

Name	Sales_Rep_Number	Pay_Class	Rate
Ward	001	1	.05
Maxim	002	1	.05
Cane	003	1	.05

Beechum	004	2	.07
Collins	005	1	.05
Cannery	006	3	.09

Khóa chính của chúng ta trong trường hợp này là mã số nhân viên bán hàng `Sales_Rep_Number`, nó phù hợp với định nghĩa của một khóa chính. Tất cả các cột khác trong bảng đều phụ thuộc vào cột `Sales_Rep_Number`, không gì phải bàn cãi.

Bây giờ, đi xa thêm một bước nữa giả thiết rằng chúng ta có thêm số an ninh xã hội (hay số CMND) của nhân viên:

Name	Sales_Rep_Number	Pay_Class	Rate	Soc.Sec. no.
Ward	001	1	.05	133-45-6789
Maxim	002	1	.05	122-46-6889
Cane	003	1	.05	123-45-6999
Beechum	004	2	.07	113-75-6889
Collins	005	1	.05	121-44-6789
Cannery	006	3	.09	111-45-9339

Giờ đây bạn có hai khóa ứng viên, `Sales_Rep_Number` và `So. Sec. no.` Vậy, chúng ta phải quyết định là sử dụng cột nào trong hai cột khóa ứng viên làm khóa chính, vì cả hai đều xác định tính duy nhất? Tốt nhất là chọn `Sales_Rep_Number` làm khóa chính vì nhiều lý do mà bạn sẽ thấy hiển nhiên trong các bước về chuẩn hóa mà chúng ta bắt đầu tiếp cận dưới đây.

3. CÁC DẠNG CHUẨN HÓA DỮ LIỆU

Giả sử chúng ta muốn tạo ra một bảng lưu thông tin người dùng, và muốn cất giữ những thông tin như `Name`, `Company`, `Company Address`, `URL`, một số địa chỉ liên lạc khác...

URL là địa chỉ Web trên Internet, ví dụ như `http://www.mycompany.com`. Ngày nay địa chỉ Web đã trở nên thông dụng như số điện thoại, số Fax, địa chỉ công ty. Một người có thể sử dụng địa chỉ Web để giới thiệu về cá nhân mình, một công ty có thể sử dụng địa chỉ Web để quảng bá hình ảnh công ty...

Bạn có thể bắt đầu bằng việc định nghĩa một cấu trúc bảng như sau:

Dạng chuẩn hóa 0 (Zero Form)

Table: users

name	company	company_address	url1	url2
Joe	ABC	1 Work Lane	abc.com	xyz.com
Jill	XYZ	1 Job Street	abc.com	xyz.com

Chúng ta nói bảng này thuộc dạng đầu tiên hay dạng 0 (Zero Form) vì nó chưa áp dụng quy tắc chuẩn hóa nào cả. Lưu ý *url1*, *url2* là các địa chỉ Website có thể có của một công ty hay cá nhân – Nếu người dùng có thêm một địa chỉ *url3* nữa thì sao? Bạn tạo một cột *url3* nữa chứ? Và nếu nhu cầu của người dùng muốn tạo *n* địa chỉ *url* khác thì có lẽ bạn phải liên tục chỉnh sửa bảng dữ liệu của mình để thêm cột mới. Chưa kể trong các chương trình xử lý bạn cũng phải thay đổi mã cứng nhắc để truy xuất đến các cột. Rõ ràng chúng ta muốn tạo một hệ thống các bảng có thể mở rộng ra theo những yêu cầu phát triển mới. Chúng ta hãy xem quy tắc chuẩn hóa dạng 1 (First Normal Form) và áp dụng quy tắc này cho bảng dữ liệu trên như thế nào.

3.1. Chuẩn hóa dạng 1 (1NF)

Dạng chuẩn hóa 1, đôi khi còn gọi là 1NF theo tên viết tắt của nó (First Normal Form), *yêu cầu thuộc tính hoặc giá trị cột phải là đơn nguyên nhỏ nhất không thể chia cắt thông tin được nữa*. Điều này có nghĩa là thuộc tính phải chứa một giá trị đơn duy nhất, nó không được chứa một tập giá trị hay tập hợp các giá trị từ những dòng dữ liệu của bảng khác.

Quy tắc dạng chuẩn hóa 1:

- 1. Loại bỏ những nhóm dữ liệu lặp lại trong từng bảng riêng lẻ.
- 2. Tạo ra một bảng riêng biệt cho tập dữ liệu liên hệ với nhau.
- 3. Xác định khóa chính cho bảng.

Xét lại bảng dữ liệu trước đây, quy tắc chuẩn hóa 1 bị vi phạm với hai thuộc tính *url1* và *url2* vì như bạn thấy chúng có những nhóm dữ liệu lặp lại cho cùng một tên Joe. Khi áp dụng các quy tắc chuẩn hóa dạng 1, bạn sẽ có được bảng biến đổi sau:

userId	name	company	company_address	url
1	Joe	ABC	1 Work Lane	abc.com
1	Joe	ABC	1 Work Lane	xyz.com
2	Jill	XYZ	1 Job Street	abc.com
2	Jill	XYZ	1 Job Street	xyz.com

Ở đây *userID* là khóa chính, khóa này được thiết lập là mã số duy nhất dành cho từng nhân viên. *url1* và *url2* được đưa về chỉ còn một cột *url*. Bây giờ bảng của chúng ta được gọi là tuân theo dạng chuẩn hóa 1. Chúng ta đã giải quyết vấn đề trùng lặp giữa các cột *url* khi mở rộng. Tuy nhiên, hãy xem một vấn đề khác chúng ta đã tự mình làm rắc rối thêm.

Mỗi khi nhập vào một mẫu tin hay dòng mới cho một url của người dùng khác vào bảng Users, chúng ta tạo nên sự trùng lặp tên công ty Company và tên người dùng Name. Khi danh sách lưu trữ lớn dần lên, dữ liệu ngày càng trở nên dư thừa và hãy hình dung nếu chúng ta muốn đổi tên cho Joe hay tên Company nơi Joe làm việc ta phải đổi hai lần. Nếu số dòng trùng lặp lớn hơn thì việc cập nhật dữ liệu thiếu có thể gây ra vấn đề nghiêm trọng về tính không đúng đắn của dữ liệu. Chúng ta hãy áp dụng thêm một quy tắc chuẩn hóa dạng 2.

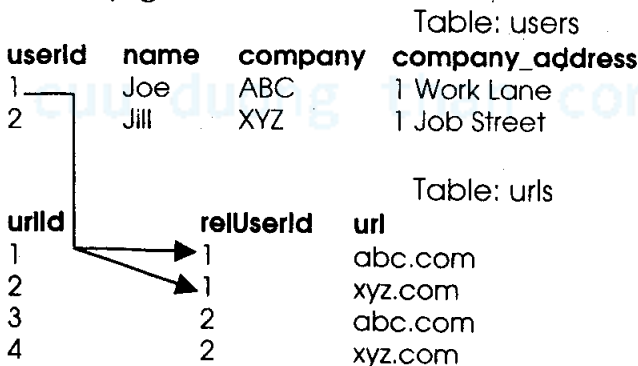
3.2. Chuẩn hóa dạng 2 (2NF)

Một lược đồ được xem là tuân theo dạng chuẩn hóa thứ hai (2NF) nếu tất cả các thuộc tính hay cột của nó (không nằm trong bộ phận của khóa chính) hoàn toàn phụ thuộc hàm vào khóa chính, và lược đồ đã tuân theo dạng chuẩn hóa 1 trước đó. Điều này có nghĩa là gì? Nó có nghĩa rằng mỗi thuộc tính (hay cột) không khóa đều phải là phụ thuộc hàm vào tất cả các bộ phận của khóa. Nếu khóa chính được tạo ra bởi nhiều cột, thì mỗi cột khác trong bảng phải phụ thuộc vào sự kết hợp của những cột của khóa chính. Quy tắc ở đây là:

1. Tạo ra bảng riêng biệt cho các tập hợp giá trị trùng lặp nhiều mẫu tin trên bảng chính.
2. Liên hệ bảng chính với bảng này bằng một khóa ngoại.

Theo chuẩn hóa dạng 2, chúng ta tách các giá trị url thành một bảng riêng biệt sao cho có thể thêm nhiều địa chỉ url khác mà không trùng lặp dữ liệu. Chúng ta cũng sẽ sử dụng giá trị khóa chính của bảng ban đầu để liên kết với các cột khóa ngoại của bảng mới:

Dạng chuẩn hóa 2



Chúng ta đã tạo xong hai bảng riêng biệt và khóa chính trong bảng Users là `userId`, kết nối tới khóa ngoại trong bảng `urls`, là `relUserId`. Dạng này đã tốt hơn. Bạn có thể thêm bao nhiêu thông tin `Url` cho `UserID` vào bảng `Urls` vẫn không có dữ liệu nào được thêm vào bảng `Users` và do đó thêm `Url` mới cho `User` không làm trùng lặp dữ liệu.

Thế nhưng điều gì xảy ra nếu ta muốn thêm một nhân viên (hoặc 200 nhân viên) mới của công ty ABC vào bảng `Users`?

Như bạn thấy mỗi nhân viên thêm vào sẽ làm trùng lặp lại địa chỉ và tên công ty. Chúng ta cần chuẩn hóa dữ liệu theo một dạng thứ 3 để khắc phục điều này.

3.3. Chuẩn hóa dạng 3 (3NF)

Dạng này yêu cầu loại bỏ các cột không phụ thuộc vào khóa. Chúng ta phải loại bỏ tất cả các phần phụ thuộc bắc cầu, và lược đồ phải ở dạng chuẩn hóa thứ hai trước đó. Vậy phụ thuộc bắc cầu là gì? Hãy xem lược đồ sau:

Users (userId name company company_address)

Lược đồ này chứa các phụ thuộc hàm sau:

userId $\rightarrow$ *name, company, company_address*

company $\rightarrow$ *company_address*

Khóa chính là `userId`, và tất cả các cột khác của bảng đều hoàn toàn phụ thuộc vào khóa chính, điều này dễ thấy vì khóa chính chỉ có một cột. Tuy nhiên, bạn có thể thấy:

userId $\rightarrow$ *name*

userId $\rightarrow$ *company*

userId $\rightarrow$ *company_address*

và do:

company $\rightarrow$ *company_address*

nên bạn có thể bắt cầu như sau:

userId $\rightarrow$ *company* $\rightarrow$ *company_address*

phụ thuộc hàm *userId* $\rightarrow$ *company* được gọi là phụ thuộc hàm bắc cầu vì nó có thể dùng làm bước trung gian để tìm ra phụ thuộc *company* $\rightarrow$ *company_address*.

Để chuyển về dạng chuẩn hóa thứ ba, chúng ta cần loại bỏ phần phụ thuộc bắc cầu này. Trường tên Company và địa chỉ công ty Company_Address của chúng ta không liên quan gì đến UserId, vì vậy chúng cần phải có một bộ khóa CompanyId khác. Các bảng được tách ra như sau:

Dạng chuẩn hóa thứ 3

Table: users

userId	name	companyId
1	Joe	1
2	Jill	2

Table: companies

companyId	company	company_address
1	ABC	1 Work Lane
2	XYZ	1 Job Street

Table: urls

urlId	relUserId	url
1	1	abc.com
2	1	xyz.com
3	2	abc.com
4	2	xyz.com

Bây giờ chúng ta đã có khóa chính CompanyId cho bảng Companies, khóa này sẽ liên kết với khóa ngoại cùng tên CompanyID trong bảng Users. Giờ đây bạn có thể chèn thêm 200 người dùng mới vào bảng Users mà chỉ cần chèn tên công ty "ABC" vào bảng Companies một lần. Bảng Users và Urls của chúng ta có thể mở rộng thêm bao nhiêu mẫu tin cũng được mà không gây ra dư thừa dữ liệu hoặc làm phát sinh vấn đề khi cập nhật thông tin về công ty.

Đa số chúng ta đều dừng lại ở dạng chuẩn hóa thứ ba này và nó hầu như thích hợp với hầu hết các mô hình ứng dụng quản lý. Nhưng hãy xem lại dữ liệu lưu trong bảng Urls, bạn có phát hiện ra khả năng dữ liệu bị trùng lặp hay không? Nếu cả hai người Joe và Jill đều thích tham chiếu đến cùng một địa chỉ Url (abc.com) thì sao?

Để giải quyết vấn đề này chúng ta cần đến một dạng chuẩn thứ 4 (4NF). Tuy nhiên dạng chuẩn này liên quan đến một quan hệ khác, đó là quan hệ nhiều-nhiều (many-many).

3.4. Quan hệ dữ liệu

Trước khi chúng ta định nghĩa và tiếp cận cách chuẩn hóa thứ 4 hãy xem lại ba loại quan hệ dữ liệu cơ bản: một-một (one-one), một-nhiều (one-many hay đôi khi còn gọi là master/detail), và nhiều-nhiều (many-many). Bạn hãy xem lại bảng người dùng Users trong ví dụ dạng chuẩn hóa 1 ở trên.

Dạng chuẩn hóa 1

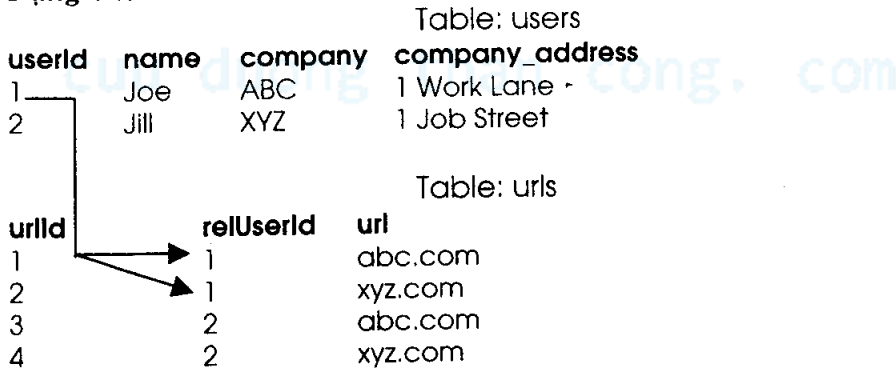
Table Users

userid	name	company	company_address	url
1	Joe	ABC	1 Work Lane	abc.com
1	Joe	ABC	1 Work Lane	xyz.com
2	Jill	XYZ	1 Job Street	abc.com
2	Jill	XYZ	1 Job Street	xyz.com

Hãy hình dung giả sử chúng ta tách cột url ra một bảng riêng biệt. Mỗi khi nhập vào một dòng mới trong bảng người dùng Users chúng ta lại nhập một dòng thông tin tương ứng url vào bảng urls. Khi đó bạn có mối quan hệ *một-một*: mỗi hàng trong bảng Users có chính xác một dòng tương ứng trong bảng urls.

Bây giờ hãy xem bảng trong ví dụ của dạng chuẩn hóa thứ 2. Bảng của chúng ta cho phép một người dùng có thể có nhiều địa chỉ url kết hợp với dòng dữ liệu.

Dạng chuẩn hóa 2



như bạn thấy UserID 1 có thể có 2 địa chỉ relUserID tương ứng. Đây là mối quan hệ một - nhiều, dạng chung và thường sử dụng nhất. Đôi khi dạng này còn gọi là master-detail (cha-con) vì một dòng thông tin của bảng này liên hệ với nhiều dòng thông tin của bảng khác như kiểu quan hệ một cha có nhiều con.

Mối quan hệ *nhiều-nhiều* (many-many) có phần phức tạp hơn. Chú ý trong ví dụ dạng chuẩn hóa thứ 3 trước đây, chúng ta có một người dùng liên quan đến nhiều urls. Như đã đề cập, chúng ta muốn thay đổi cấu trúc để cho phép nhiều người dùng được sử dụng hay liên kết đến nhiều url khác nhau, đây chính là lúc chúng ta muốn tạo một mối quan hệ nhiều-nhiều.

Hãy xem điều chúng ta muốn thực hiện trên cấu trúc bảng trước khi đi vào chi tiết:

Dạng chuẩn hóa thứ 4

Table: users

userid	name	companyID
--------	------	-----------

1	Joe	1
2	Jill	2

Table: companies

companyID	company	company_address
-----------	---------	-----------------

1	ABC	1 Work Lane
2	XYZ	1 Job Street

Table: urls

urlID	url
-------	-----

1	abc.com
2	xyz.com
3	abc.com
4	xyz.com

Table: urls_relations

urlId	relUrlId	relUserId
1	1	1
2	1	2
3	2	1
4	2	2

Để giảm bớt trùng lặp dữ liệu (và để chuẩn bị cho dạng chuẩn hóa thứ 4), chúng ta tạo ra thêm một bảng chỉ có một khóa chính và hai khóa ngoại, bảng url_relations. Chúng ta đã có thể loại bỏ những mục thông tin trùng lặp trong bảng urls bằng cách tạo bảng url_relations khác. Bây giờ chúng ta đã có thể diễn đạt đúng đắn mối quan hệ mà cả Joe lẫn Jill liên kết đến một, hoặc hai địa chỉ url. Như vậy hãy xem chính xác dạng chuẩn hóa thứ 4 yêu cầu những gì.

3.5. Chuẩn hóa dạng 4 (4NF)

Trong một quan hệ nhiều-nhiều, thông tin của những thực thể độc lập không cất giữ trong cùng một bảng. Do nó chỉ áp dụng cho quan hệ nhiều-nhiều, đa số các nhà phát triển ứng dụng có thể bỏ qua quy tắc này. Tuy nhiên trong một số tình huống nhất định, như trường hợp này của chúng ta chẳng hạn, ta cần phải quan tâm đến nó. Chúng ta đã sắp xếp hợp lý bảng urls để loại bỏ những mục thông tin trùng lặp và di chuyển các mối quan hệ vào trong một bảng khác của riêng chúng. Khi học về ngôn ngữ SQL trong phần sau bạn sẽ thấy việc truy xuất toàn bộ danh sách các URL của Joe có thể thực hiện như sau:

```
SELECT name, url
FROM users, urls, url_relations
WHERE url_relations.relatedUserId = 1
AND users.userId = 1
AND urls.urlId = url_relations.relatedUrlId
```

Và nếu chúng ta muốn duyệt qua danh sách của tất cả người dùng User cùng với thông tin về Url của họ, bạn có thể truy vấn dữ liệu như sau:

```
SELECT name, url
FROM users, urls, url_relations
WHERE users.userId = url_relations.relatedUserId
AND urls.urlId = url_relations.relatedUrlId
```

Tóm lại quá trình chuẩn hóa dữ liệu sẽ giúp bạn chia nhỏ, tổ chức thông tin gọn, hiệu quả loại bỏ dữ liệu dư thừa, dữ liệu chứa trong các bảng là cơ bản nhất, tốn ít không gian lưu trữ nhất. Đồng thời chuẩn hóa có thể giúp quá trình truy xuất dữ liệu nhanh hơn. Dạng chuẩn hóa 2 và 3 là thường dùng nhất. Quan hệ nhiều-nhiều về nguyên tắc có thể biến đổi thành quan hệ một-nhiều theo hai hướng.

Đôi lúc, việc tạo ra một cơ sở dữ liệu hiệu quả và đơn giản là cả một quá trình trực giác. Thường xuyên làm việc với cơ sở dữ liệu là cách tốt nhất để bạn có được kinh nghiệm và trực giác khi thiết kế. Trong phần sau chúng ta sẽ bắt đầu học những bài học ứng dụng thực tế hơn về cơ sở dữ liệu, đó là sử dụng ngôn ngữ tương tác SQL. Tuy nhiên tất cả những khái niệm và dạng chuẩn hóa trong chương này là điều kiện cơ bản bạn cần phải nắm vững, nó là nền tảng để chúng ta tìm hiểu các ứng dụng thực tế về cơ sở dữ liệu trong những chương sau.

cuu duong than cong. com

cuu duong than cong. com

Chương 3:

SQL VÀ CƠ SỞ DỮ LIỆU

Các vấn đề chính sẽ được đề cập:

- ✓ *SQL là gì?*
- ✓ *Các chức năng cơ bản của SQL.*
- ✓ *Lệnh SELECT.*
- ✓ *Lệnh INSERT.*
- ✓ *Lệnh UPDATE.*
- ✓ *Lệnh DELETE.*
- ✓ *Sắp xếp với ORDER BY.*
- ✓ *Lọc dữ liệu với mệnh đề WHERE.*
- ✓ *Toán tử AND, OR, BETWEEN, LIKE, IN.*
- ✓ *Kết nối bảng với JOIN.*
- ✓ *Hợp hai bảng với UNION.*
- ✓ *Nhóm dữ liệu với GROUP BY.*
- ✓ *Xử lý tổng gộp SUM, COUNT.*
- ✓ *Thay đổi cấu trúc bảng với lệnh SQL ALTER.*
- ✓ *Tạo View.*
- ✓ *Xóa bảng, xóa cơ sở dữ liệu.*
- ✓ *Tham khảo nhanh SQL.*
- ✓ *Kiểm tra.*

SQL là một ngôn ngữ máy tính chuẩn để truy nhập và thao tác xử lý hầu hết mọi hệ cơ sở dữ liệu. Trong chương này bạn sẽ học cách sử dụng SQL để truy nhập và thao tác trên các bảng dữ liệu. Khi tiếp cận với các hệ quản trị như Oracle, Sybase, SQL Server, DB2, Access... bạn đều có thể sử dụng cú pháp này.

1. SQL LÀ GÌ?

SQL ý nghĩa của nó là ngôn ngữ truy vấn cấu trúc (Structured Query Language).

- SQL cho phép bạn truy nhập xử lý cơ sở dữ liệu.
- SQL là một ngôn ngữ máy tính theo chuẩn ANSI.
- SQL có thể thực thi những truy vấn trên cơ sở dữ liệu.
- SQL có thể dùng trích rút dữ liệu từ các bảng.
- SQL có thể chèn những mẫu tin hay dòng dữ liệu mới vào trong một bảng của cơ sở dữ liệu.
- SQL có thể xóa những dòng mẫu tin từ một bảng cơ sở dữ liệu.
- SQL có thể cập nhật những dòng mẫu tin trong một cơ sở dữ liệu.
- SQL rất dễ học.

SQL là một tiêu chuẩn – Thế nhưng...

SQL tuân theo chuẩn ANSI (American National Standards Institute - Viện những tiêu chuẩn Quốc gia Mỹ) được quy định là chuẩn cho ngôn ngữ máy tính để truy nhập và thao tác trên những hệ thống cơ sở dữ liệu. Những câu lệnh SQL được sử dụng để trích rút và cập nhật dữ liệu trong một hoặc nhiều bảng của cơ sở dữ liệu. SQL làm việc với tất cả những chương trình quản trị cơ sở dữ liệu như MS Access, DB2, Informix, FoxPro, Oracle, SQL Server, MySQL. Không may, có khá nhiều phiên bản khác nhau của ngôn ngữ SQL, nhưng theo chuẩn ANSI chúng hầu hết phải hỗ trợ những từ khóa chính và các lệnh cơ bản gần tương tự nhau (như SELECT, UPDATE, DELETE, INSERT, WHERE...).

Ghi chú: Đa số các chương trình quản trị cơ sở dữ liệu SQL đều có những mở rộng về ngôn ngữ của riêng mình ngoài những lệnh và từ khóa tuân theo chuẩn SQL ANSI!

2. SQL VÀ CÁC CHỨC NĂNG CƠ BẢN

2.1. Bảng cơ sở dữ liệu

Như bạn đã biết, một hệ cơ sở dữ liệu thường chứa một hoặc nhiều bảng dữ liệu. Mỗi bảng được xác định bởi một tên (ví dụ "Customers" hoặc

“Orders”). Bảng chứa các dòng mẫu tin (hàng) với dữ liệu là các cột. Dưới đây là một bảng ví dụ mang tên Persons:

LastName	FirstName	Address	City
Hansen	Ola	Timoteivn 10	Sandnes
Svendson	Tove	Borgvn 23	Sandnes
Pettersen	Kari	Storgt 20	Stavanger

Bảng trên chứa ba dòng mẫu tin (mỗi dòng là thông tin cá nhân của một người) và bốn cột (LastName, FirstName, Address, và City).

2.2. Truy vấn SQL

Với SQL, bạn có thể thực hiện các truy vấn (query) trên cơ sở dữ liệu và nhận về câu trả lời là một tập kết quả tương ứng. Một câu truy vấn nội dung của bảng dữ liệu cụ thể sẽ như sau:

```
SELECT LastName FROM Persons
```

Kết quả trả về là tập hợp dữ liệu sau:

LastName
Hansen
Svendson
Pettersen

Ghi chú: Một số hệ thống cơ sở dữ liệu như Oracle, MySQL... yêu cầu dấu chấm phẩy cuối câu lệnh SQL để làm dấu hiệu nhận dạng nơi kết thúc lệnh. Chẳng hạn:

```
SELECT LastName FROM Persons;
```

trong các ví dụ của chương này, để đơn giản hóa chúng ta không sử dụng dấu chấm phẩy.

2.3. SQL ngôn ngữ thao tác dữ liệu (DML)

Tuy cú pháp SQL tập trung vào thực thi các câu truy vấn dữ liệu nhưng nó cũng bao gồm một số cú pháp để cập nhật, chèn, và xóa các dòng mẫu tin.

Những lệnh cập nhật và tác động để dữ liệu này nhóm lại thành một phần của ngôn ngữ thường được gọi là phần ngôn ngữ tương tác với dữ liệu (DML - Data Manipulation Language) vì nó trực tiếp thao tác làm thay đổi dữ liệu lưu trữ.

- **SELECT** - Trích rút dữ liệu từ một bảng cơ sở dữ liệu.
- **UPDATE** - Cập nhật các dòng trong một bảng cơ sở dữ liệu.
- **DELETE** - Xóa các dòng dữ liệu khỏi một bảng cơ sở dữ liệu.
- **INSERT INTO** - Chèn các dòng dữ liệu mới vào một bảng cơ sở dữ liệu.

2.4. Ngôn ngữ định nghĩa dữ liệu SQL (DDL)

Một bộ phận khác của ngôn ngữ SQL là nhóm lệnh định nghĩa cấu trúc dữ liệu, cho phép tạo, xóa các bảng, cột trong cơ sở dữ liệu. Nhóm lệnh này được gọi là nhóm lệnh định nghĩa dữ liệu (DDL – Data Definition Language). Bạn cũng có thể định nghĩa cách đánh chỉ mục, khóa, xác định mối liên kết và các ràng buộc giữa các bảng cơ sở dữ liệu. Các lệnh DDL quan trọng nhất trong SQL là:

- **CREATE TABLE** - Tạo ra một bảng cơ sở dữ liệu mới.
- **ALTER TABLE** - Thay đổi cấu trúc một bảng cơ sở dữ liệu.
- **DROP TABLE** - Xóa một bảng cơ sở dữ liệu.
- **CREATE INDEX** - Tạo chỉ mục (khóa tìm kiếm).
- **DROP INDEX** - Xóa một chỉ mục.

3. LỆNH SELECT

Lệnh **SELECT** được sử dụng để chọn lọc dữ liệu từ một bảng. Kết quả các dòng trích lọc do **SELECT** trả về cũng ở dạng bảng dữ liệu và thường gọi là tập bảng kết quả (result-set).

Cú pháp:

```
SELECT column_name(s) FROM table_name
```

3.1. Tuyển chọn các cột

Để lựa chọn những cột có tên **LastName** và **FirstName**, bạn sử dụng lệnh **SELECT** như sau:

SELECT LastName,FirstName FROM Persons

Persons Table

LastName	FirstName	Address	City
Hansen	Ola	Timoteivn 10	Sandnes
Svendson	Tove	Borgvn 23	Sandnes
Pettersen	Kari	Storgt 20	Stavanger

Kết quả:

LastName	FirstName
Hansen	Ola
Svendson	Tove
Pettersen	Kari

3.2. Lựa chọn tất cả các cột

Để lựa chọn tất cả các cột trong bảng “Persons” bạn sử dụng ký tự đại diện * thay vì liệt kê tên từng cột đặt như sau:

SELECT * FROM Persons

Kết quả

LastName	FirstName	Address	City
Hansen	Ola	Timoteivn 10	Sandnes
Svendson	Tove	Borgvn 23	Sandnes
Pettersen	Kari	Storgt 20	Stavanger

3.3. Tập kết quả

Kết quả từ một truy vấn SQL được cất giữ trong một bảng kết quả gọi là tập kết quả result-set. Hầu hết hệ phần mềm và ngôn ngữ lập trình cơ sở dữ liệu đều cho phép bạn duyệt các mẫu tin trong tập kết quả bằng những hàm như di chuyển tới trước, lùi về, nhảy đến mẫu tin đầu tiên, cuối cùng: MoveFirst, GetContent, MoveNext.

Để học về việc truy nhập dữ liệu bằng các ngôn ngữ lập trình, bạn hãy tham khảo các sách lập trình VB hay VB.NET của nhà sách Minh Khai đã phát hành. Thường thì với VB chẳng hạn, các hàm này được đóng gói trong thư viện truy xuất dữ liệu ADO hay ADO.NET (với VB.NET).

3.4. SELECT DISTINCT lệnh chọn dữ liệu không trùng

Từ khóa DISTINCT được sử dụng để trả lại các giá trị phân biệt (khác nhau), nghĩa là cho phép chọn ra tập dữ liệu không bị trùng lặp.

Cú pháp:

SELECT DISTINCT column_name(s) **FROM** table_name

Sử dụng từ khóa DISTINCT

Để lựa chọn tất cả giá trị từ cột Company, chúng ta sử dụng lệnh SELECT như sau:

SELECT Company FROM Orders

Orders Table

Company	OrderNumber
Sega	3412
W3Schools	2312
Trio	4678
W3Schools	6798

Kết quả:

Company
Sega
W3Schools
Trio
W3Schools

Bạn chú ý W3Schools được liệt kê hai lần trong tập kết quả. Để lựa chọn những giá trị khác nhau từ cột Company bạn sử dụng lệnh SELECT DISTINCT như sau:

SELECT DISTINCT Company FROM Orders

Kết quả:

Company
Sega
W3Schools
Trio

Bây giờ W3Schools được liệt kê chỉ một lần trong tập kết quả.

SQL cung cấp mệnh đề WHERE để xác định tiêu chuẩn chọn lọc dữ liệu.

3.5. Mệnh đề WHERE

Để lựa chọn dữ liệu có điều kiện từ một bảng, mệnh đề WHERE có thể được thêm vào kết hợp với lệnh SELECT.

Cú pháp:

SELECT column FROM table
WHERE column operator value

Các toán tử (operator) sau có thể được sử dụng với mệnh đề WHERE:

Toán tử	Mô tả
=	Bằng nhau
<>	Không bằng nhau
>	Lớn hơn
<	Nhỏ hơn
>=	Lớn hơn hoặc bằng
<=	Nhỏ hơn hoặc bằng
BETWEEN	Giữa một phạm vi giá trị
LIKE	Tìm kiếm so khớp theo mẫu

Ghi chú: Trong một số phiên bản SQL, toán tử <> có thể được viết như sau: !=

Sử dụng mệnh đề WHERE

Để lựa chọn ra duy nhất chỉ những người sống trong thành phố “Sandnes”, chúng ta thêm một mệnh đề WHERE trong lệnh SELECT như sau:

```
SELECT * FROM Persons WHERE City='Sandnes'
```

Persons Table

LastName	FirstName	Address	City	Year
Hansen	Ola	Timoteivn 10	Sandnes	1951
Svendson	Tove	Borgvn 23	Sandnes	1978
Svendson	Stale	Kaivn 18	Sandnes	1980
Pettersen	Kari	Storgt 20	Stavanger	1960

Kết quả:

LastName	FirstName	Address	City	Year
Hansen	Ola	Timoteivn 10	Sandnes	1951
Svendson	Tove	Borgvn 23	Sandnes	1978
Svendson	Stale	Kaivn 18	Sandnes	1980

Sử dụng dấu nháy bội chuỗi

Chú ý rằng trong ví dụ trên chúng ta đã được sử dụng dấu nháy đơn bao xung quanh chuỗi điều kiện. SQL sử dụng dấu nháy đơn (') bao xung quanh giá trị chuỗi hay văn bản (tuy nhiên hầu hết các hệ thống cơ sở dữ liệu cũng sẽ chấp nhận dấu nháy kép (") thay cho dấu nháy đơn). Giá trị số không cần được bao trong cặp dấu nháy. Đối với giá trị văn bản, cú pháp sau đây là đúng:

```
SELECT * FROM Persons WHERE FirstName='Tove'
```

Nhưng cú pháp sau là sai:

```
SELECT * FROM Persons WHERE FirstName=Tove
```

Với giá trị số, cú pháp sau là đúng:

```
SELECT * FROM Persons WHERE Year>1965
```

Nhưng cú pháp sau là sai:

```
SELECT * FROM Persons WHERE Year>'1965'
```

3.6. Điều kiện so khớp LIKE

Điều kiện LIKE được sử dụng để chỉ rõ điều kiện tìm kiếm giá trị trong một cột theo mẫu (pattern) nào đó.

Cú pháp:

```
SELECT column FROM table
WHERE column LIKE pattern
```

Ký hiệu phần trăm “%” có thể sử dụng định nghĩa những ký tự đại diện cho mẫu cần tìm (thay thế các ký tự không có trong mẫu), bạn có thể đặt % cả phía trước và phía sau mẫu.

Sử dụng LIKE

Câu lệnh SQL sau sẽ trả về kết quả là danh sách những người có tên bắt đầu bằng chữ 'O':

```
SELECT * FROM Persons
WHERE FirstName LIKE 'O%'
```

Câu lệnh SQL sau sẽ trả về kết quả là danh sách những người có tên kết thúc bằng chữ 'a':

```
SELECT * FROM Persons
WHERE FirstName LIKE '%a'
```

Câu lệnh SQL sau sẽ trả về kết quả là danh sách những người có tên mang 2 chữ 'la' ở bất kỳ vị trí nào trong chuỗi tên:

```
SELECT * FROM Persons
WHERE FirstName LIKE '%la%'
```

4. LỆNH CHÈN DỮ LIỆU INSERT INTO

Lệnh chèn dữ liệu **INSERT INTO** được sử dụng để chèn một dòng hay hàng dữ liệu mới vào trong một bảng.

Cú pháp:

INSERT INTO table_name **VALUES** (value1, value2,...)

Bạn có thể chỉ rõ những cột cụ thể muốn chèn dữ liệu như cú pháp sau:

INSERT INTO table_name (column1, column2,...)

VALUES (value1, value2,...)

Chèn một dòng mới

Persons Table

LastName	FirstName	Address	City
Pettersen	Kari	Storgt 20	Stavanger

khi thực thi lệnh SQL:

INSERT INTO Persons

VALUES ('Hetland', 'Camilla', 'Hagabakka 24', 'Sandnes')

sẽ cho kết quả là bảng dữ liệu Persons được thêm vào một dòng mới như sau:

Persons Table

LastName	FirstName	Address	City
Pettersen	Kari	Storgt 20	Stavanger
Hetland	Camilla	Hagabakka 24	Sandnes

Chèn dữ liệu vào những cột chỉ định cụ thể

Persons Table

LastName	FirstName	Address	City
Pettersen	Kari	Storgt 20	Stavanger
Hetland	Camilla	Hagabakka 24	Sandnes

và khi thực thi câu lệnh SQL sau:

```
INSERT INTO Persons (LastName, Address)
VALUES ('Rasmussen', 'Storgt 67')
```

sẽ cho kết quả là bảng dữ liệu Persons được thêm vào một dòng mới như sau:

Persons Table

LastName	FirstName	Address	City
Pettersen	Kari	Storgt 20	Stavanger
Hetland	Camilla	Hagabakka 24	Sandnes
Rasmussen		Storgt 67	

5. LỆNH CẬP NHẬT DỮ LIỆU UPDATE

Lệnh cập nhật UPDATE được sử dụng để sửa đổi dữ liệu trong một bảng.

Cú pháp:

```
UPDATE table_name
SET column_name = new_value
WHERE column_name = some_value
```

Persons Table

LastName	FirstName	Address	City
Nilsen	Fred	Kirkegt 56	Stavanger
Rasmussen		Storgt 67	

Cập nhật giá trị cột trong một dòng dữ liệu

Giả sử chúng ta muốn đổi tên FirstName cho một người đã có tên LastName là “Rasmussen”, lệnh cập nhật thực hiện như sau:

```
UPDATE Person SET FirstName = 'Nina'
WHERE LastName = 'Rasmussen'
```

Kết quả:

LastName	FirstName	Address	City
Nilsen	Fred	Kirkegt 56	Stavanger
Rasmussen	Nina	Storgt 67	

Cập nhật nhiều cột trong một dòng dữ liệu

Chúng ta muốn thay đổi địa chỉ và thêm vào tên thành phố:

```
UPDATE Person
```

```
SET Address = 'Stien 12', City = 'Stavanger'
```

```
WHERE LastName = 'Rasmussen'
```

Kết quả:

LastName	FirstName	Address	City
Nilsen	Fred	Kirkegt 56	Stavanger
Rasmussen	Nina	Stien 12	Stavanger

6. LỆNH XÓA DỮ LIỆU DELETE

Lệnh DELETE được sử dụng để xóa các dòng dữ liệu một bảng.

Cú pháp:

```
DELETE FROM table_name WHERE column_name = some_value
```

Persons Table

LastName	FirstName	Address	City
Nilsen	Fred	Kirkegt 56	Stavanger
Rasmussen	Nina	Stien 12	Stavanger

Xóa một hàng

Dòng "Nina Rasmussen" sẽ bị xóa:

```
DELETE FROM Person WHERE LastName = 'Rasmussen'
```

Kết quả:

LastName	FirstName	Address	City
Nilsen	Fred	Kirkegt 56	Stavanger

Xóa tất cả các hàng

Bạn có thể xóa tất cả các dòng trong một bảng mà không cần phải xóa bảng. Điều này có nghĩa rằng cấu trúc bảng, cột và chỉ mục sẽ không bị thay đổi và mất đi:

```
DELETE FROM table_name
```

hoặc:

```
DELETE * FROM table_name
```

7. KIỂM TRA NHỮNG KỸ NĂNG SQL VỪA HỌC CỦA BẠN

Trong phần này bạn có thể kiểm tra những kỹ năng SQL vừa học của mình qua những ví dụ đơn giản. Chúng ta sẽ sử dụng bảng khách hàng trong cơ sở dữ liệu Northwind (cơ sở dữ liệu mẫu bạn có thể thấy trong hệ quản trị cơ sở dữ liệu của MS Access hay SQL Server của Microsoft):

CompanyName	ContactName	Address	City
Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin
Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå
Centro comercial Moctezuma	Francisco Chang	Sierras de Granada 9993	México D.F.
Ernst Handel	Roland Mendel	Kirchgasse 6	Graz
FISSA Fabrica Inter. Salchichas S.A.	Diego Roel	C/ Moralzarzal, 86	Madrid
Galería del gastrónomo	Eduardo Saavedra	Rambla de Cataluña, 23	Barcelona
Island Trading	Helen Bennett	Garden House Crowther Way	Cowes

Königlich Essen	Philip Cramer	Maubelstr. 90	Brandenburg
Laughing Bacchus Wine Cellars	Yoshi Tannamuri	1900 Oak St.	Vancouver
Magazzini Alimentari Riuniti	Giovanni Rovelli	Via Ludovico il Moro 22	Bergamo
North/South	Simon Crowther	South House 300 Queensbridge	London
Paris spécialités	Marie Bertrand	265, boulevard Charonne	Paris
Rattlesnake Canyon Grocery	Paula Wilson	2817 Milton Dr.	Albuquerque
Simons bistro	Jytte Petersen	Vinbæltet 34	København
The Big Cheese	Liz Nixon	89 Jefferson Way Suite 2	Portland
Vaffeljernet	Palle Ibsen	Smagsløget 45	Århus
Wolski Zajazd	Zbyszek Piestrzeniewicz	ul. Filtrowa 68	Warszawa

Để đỡ tốn không gian, chúng tôi chỉ trích ra trong bảng trên một phần dữ liệu khách hàng, dữ liệu thật sẽ nhiều hơn

Bạn hãy thử:

Xem danh sách tất cả thông tin khách hàng:

```
SELECT * FROM customers
```

Xem danh sách công ty và địa chỉ của khách hàng:

```
SELECT CompanyName, ContactName FROM customers
```

Xem danh sách các khách hàng có tên bắt đầu bằng chữ 'a':

```
SELECT * FROM customers
WHERE companyname LIKE 'a%'
```


Xem danh sách tên CompanyName, ContactName tất cả các khách hàng có tên chữ cái sau 'g':

```
SELECT CompanyName, ContactName
FROM customers
WHERE CompanyName > 'g'
AND ContactName > 'g'
```

8. SẮP XẾP THỨ TỰ BẰNG SQL ORDER BY

Từ khóa ORDER BY được sử dụng để phân loại và sắp xếp kết quả.

Sắp xếp hàng dữ liệu

Mệnh đề ORDER BY được sử dụng để sắp xếp các hàng dữ liệu kết xuất. Dưới đây là bảng danh sách đơn hàng của các công ty:

Orders Table

Company	OrderNumber
Sega	3412
ABC Shop	5678
W3Schools	2312
W3Schools	6798

Ví dụ, để liệt kê danh sách công ty theo thứ tự chữ cái:

```
SELECT Company, OrderNumber FROM Orders ORDER BY Company
```

Kết quả:

Company	OrderNumber
ABC Shop	5678
Sega	3412
W3Schools	6798
W3Schools	2312

Để liệt kê danh sách những công ty trong thứ tự vần chữ cái và đơn hàng theo thứ tự số từ nhỏ đến lớn:

```
SELECT Company, OrderNumber FROM Orders  
ORDER BY Company, OrderNumber
```

Kết quả:

Company	OrderNumber
ABC Shop	5678
Sega	3412
W3Schools	2312
W3Schools	6798

Để hiển thị những công ty sắp xếp theo ngược thứ tự chữ cái bạn thêm từ khóa DESC:

```
SELECT Company, OrderNumber FROM Orders  
ORDER BY Company DESC
```

Kết quả:

Company	OrderNumber
W3Schools	6798
W3Schools	2312
Sega	3412
ABC Shop	5678

Để hiển thị những công ty sắp xếp theo ngược thứ tự chữ cái và đơn hàng theo thứ tự số nhỏ đến lớn:

```
SELECT Company, OrderNumber FROM Orders  
ORDER BY Company DESC,  
OrderNumber ASC
```

Kết quả:

Company	OrderNumber
W3Schools	2312
W3Schools	6798
Sega	3412
ABC Shop	5678

Lưu ý, mặc định nếu bạn không chỉ định DESC (giảm dần) hay ASC (tăng dần) thì SQL sử dụng ASC cho mệnh đề ORDER, nghĩa là:

```
SELECT Company, OrderNumber FROM Orders
ORDER BY Company
```

tương đương với:

```
SELECT Company, OrderNumber FROM Orders
ORDER BY Company ASC
```

9. SQL AND & OR

Toán tử AND (“và”) và OR (“hoặc”) nối hai hoặc nhiều điều kiện với nhau trong mệnh đề WHERE. Toán tử AND hiển thị dòng hay mẫu tin nếu tất cả các điều kiện liệt kê mà nó kết nối là TRUE (đúng). Toán tử OR hiển thị dòng hay mẫu tin khi có ít nhất một điều kiện mà nó kết nối là TRUE (đúng).

Ghi chú: TRUE hay FALSE là giá trị xác định chân trị đúng hoặc sai. Ví dụ 1>0 hay ‘A’=‘A’ là các biểu thức cho giá trị TRUE (đúng) còn 1=2 hay ‘A’=‘B’ sẽ cho giá trị FALSE (sai)

Dưới đây là các ví dụ để bạn có thể hình dung:

Persons Table

LastName	FirstName	Address	City
Hansen	Ola	Timoteivn 10	Sandnes
Svendson	Tove	Borgvn 23	Sandnes
Svendson	Stephen	Kaivn 18	Sandnes

Ví dụ, hãy sử dụng AND để hiển thị những người có họ bằng "Tove" và tên là "Svendson", lệnh SELECT được viết như sau:

```
SELECT * FROM Persons
      WHERE FirstName='Tove' AND LastName='Svendson'
```

Kết quả:

LastName	FirstName	Address	City
Svendson	Tove	Borgvn 23	Sandnes

Sử dụng OR để hiển thị danh sách các cá nhân có họ bằng "Tove" hoặc tên bằng "Svendson":

```
SELECT * FROM Persons
      WHERE firstname='Tove' OR lastname='Svendson'
```

Kết quả:

LastName	FirstName	Address	City
Svendson	Tove	Borgvn 23	Sandnes
Svendson	Stephen	Kaivn 18	Sandnes

Bạn cũng có thể kết hợp AND và OR (sử dụng dấu ngoặc đơn để tách riêng những biểu thức phức tạp):

```
SELECT * FROM Persons WHERE
      (FirstName='Tove' OR FirstName='Stephen')
      AND LastName='Svendson'
```

Kết quả:

LastName	FirstName	Address	City
Svendson	Tove	Borgvn 23	Sandnes
Svendson	Stephen	Kaivn 18	Sandnes

10. SQL IN

Toán tử IN có thể được sử dụng để xác định một giá trị có nằm trong danh sách những giá trị biết trước hay không.

```
SELECT column_name FROM table_name
WHERE column_name IN (value1,value2,...)
```

Persons Table

LastName	FirstName	Address	City
Hansen	Ola	Timoteivn 10	Sandnes
Nordmann	Anna	Neset 18	Sandnes
Pettersen	Kari	Storgt 20	Stavanger
Svendson	Tove	Borgvn 23	Sandnes

Ví dụ 1, để hiển thị những cá nhân có tên LastName bằng “Hansen” hoặc “Pettersen”, bạn sử dụng lệnh SQL sau:

```
SELECT * FROM Persons
WHERE LastName IN ('Hansen','Pettersen')
```

Kết quả:

LastName	FirstName	Address	City
Hansen	Ola	Timoteivn 10	Sandnes
Pettersen	Kari	Storgt 20	Stavanger

11. SQL BETWEEN

Toán tử BETWEEN ... AND cho phép chọn lọc dữ liệu trong một phạm vi các giá trị đầu và cuối. Những giá trị này có thể là số, chuỗi văn bản hoặc ngày tháng.

```
SELECT column_name FROM table_name
WHERE column_name BETWEEN value1 AND value2
```

Persons Table

LastName	FirstName	Address	City
Hansen	Ola	Timoteivn 10	Sandnes
Nordmann	Anna	Neset 18	Sandnes
Pettersen	Kari	Storgt 20	Stavanger
Svendson	Tove	Borgvn 23	Sandnes

Ví dụ 1, để hiển thị danh sách các cá nhân theo thứ tự abc giữa (bao gồm cả) "Hansen" và "Pettersen", bạn sử dụng lệnh SQL:

```
SELECT * FROM Persons WHERE LastName
        BETWEEN 'Hansen' AND 'Pettersen'
```

Kết quả:

LastName	FirstName	Address	City
Hansen	Ola	Timoteivn 10	Sandnes
Nordmann	Anna	Neset 18	Sandnes
Pettersen	Kari	Storgt 20	Stavanger

Lưu ý quan trọng! Toán tử *BETWEEN...AND* có thể được cài đặt khác nhau trong từng cơ sở dữ liệu. Với một số cơ sở dữ liệu mệnh đề *X BETWEEN a AND b* tương đương $a \leq X \leq b$, một số khác lại tương đương $a \leq X < b$. Do đó hãy đọc tài liệu hướng dẫn của các hệ quản trị cơ sở dữ liệu mà bạn đang sử dụng để biết chi tiết cách cài đặt và sử dụng *BETWEEN ... AND*.

Ví dụ 2, để hiển thị những cá nhân nằm ngoài phạm vi của *BETWEEN ... AND* bạn sử dụng thêm toán tử *NOT*:

```
SELECT * FROM Persons WHERE LastName
        NOT BETWEEN 'Hansen' AND 'Pettersen'
```

Kết quả:

LastName	FirstName	Address	City
Svendson	Tove	Borgvn 23	Sandnes

12. BÍ DANH SQL (ALIAS)

Với SQL, bạn có thể dùng bí danh thay thế để đặt cho tên cột và tên bảng.

Bí danh tên Cột

Cú pháp:

SELECT column **AS** column_alias **FROM** table

Bí danh tên Bảng

Cú pháp:

SELECT column **FROM** table **AS** table_alias

Persons Table

LastName	FirstName	Address	City
Hansen	Ola	Timoteivn 10	Sandnes
Svendson	Tove	Borgvn 23	Sandnes
Pettersen	Kari	Storgt 20	Stavanger

Ví dụ: Sử dụng bí danh cột

SELECT LastName **AS** Family, FirstName **AS** Name **FROM** Persons

Kết quả:

Family	Name
Hansen	Ola
Svendson	Tove
Pettersen	Kari

Ví dụ: Sử dụng một bí danh bảng.

Persons Table

LastName	FirstName	Address	City
Hansen	Ola	Timoteivn 10	Sandnes
Svendson	Tove	Borgvn 23	Sandnes
Pettersen	Kari	Storgt 20	Stavanger

```
SELECT LastName, FirstName FROM Persons AS Employees
```

Kết quả:

Employees Table

LastName	FirstName
Hansen	Ola
Svendson	Tove
Pettersen	Kari

13. SQL JOIN KẾT NỐI CÁC BẢNG

Kết nối và khóa

Trong các chương trước bạn đã học về cách xây dựng và chuẩn hóa các bảng. SQL cung cấp cho bạn cú pháp kết nối và trích rút dữ liệu từ các bảng quan hệ với nhau thông qua khóa. Bạn có thể trích lọc cùng lúc dữ liệu từ hai hoặc nhiều bảng lại với nhau để tạo nên kết quả là một bảng tổng hợp đầy đủ. Chúng ta phải sử dụng mệnh đề kết nối JOIN.

Như bạn đã biết, trong một cơ sở dữ liệu các bảng liên quan với nhau thông qua khóa. Khóa chính (Primary Key) là cột chứa các giá trị duy nhất có thể dùng nhận dạng ra mỗi hàng dữ liệu trong bảng.

Trong bảng Employees, cột Employee_ID là khóa chính, có nghĩa là không có hai hàng nào có thể có cùng mã số Employee_ID. Employee_ID phân biệt hai người khác nhau cho dù hai người có trùng tên đi chăng nữa. Khi bạn xem ví dụ trong các bảng dưới đây, lưu ý rằng: cột Employee_ID là khóa chính của "Employees", cột Prod_ID là khóa chính của bảng Orders. Cột Employee_ID trong bảng Orders được sử dụng để tham chiếu tới các cá nhân trong bảng Employees mà không cần sử dụng đến tên của nhân viên.

Employees Table

Employee_ID	Name
01	Hansen, Ola
02	Svendson, Tove

03	Svendson, Stephen
04	Pettersen, Kari

Orders Table

Prod_ID	Product	Employee_ID
234	Printer	01
657	Table	03
865	Chair	03

Kết hợp hai bảng

Chúng ta có thể trích lọc dữ liệu từ hai bảng bằng cách tham chiếu đến cả hai bảng trong lệnh **SELECT** như sau:

Ví dụ: Để trả lời ai đã đặt hàng sản phẩm, và họ đặt hàng những sản phẩm gì?

```
SELECT Employees.Name, Orders.Product
FROM Employees, Orders
WHERE Employees.Employee_ID=Orders.Employee_ID
```

Kết quả:

Name	Product
Hansen, Ola	Printer
Svendson, Stephen	Table
Svendson, Stephen	Chair

Ví dụ: Muốn biết ai đã đặt hàng máy in Printer?

```
SELECT Employees.Name
FROM Employees, Orders
WHERE Employees.Employee_ID=Orders.Employee_ID
AND Orders.Product='Printer'
```

Kết quả:

Name
Hansen, Ola

Sử dụng JOIN

Thay vì kết nối hai bảng dựa vào so sánh hai khóa như trên, bạn có thể dùng mệnh đề INNER JOIN, LEFT JOIN hay RIGHT JOIN như sau:

INNER JOIN

Cú pháp:

```
SELECT field1, field2, field3 FROM first_table
    INNER JOIN second_table
    ON first_table.keyfield = second_table.foreign_keyfield
```

Bạn có thể trích rút thông tin ai đã đặt hàng sản phẩm gì bằng lệnh SQL sau:

```
SELECT Employees.Name, Orders.Product
    FROM Employees
    INNER JOIN Orders
    ON Employees.Employee_ID=Orders.Employee_ID
```

Kết nối INNER JOIN sẽ trả về tất cả các dòng từ hai bảng thỏa điều kiện so khớp khóa của nhau. Nếu có các dòng trong bảng Employees không bằng khóa với nhau, những dòng đó sẽ không được liệt kê ra.

Kết quả

Name	Product
Hansen, Ola	Printer
Svendson, Stephen	Table
Svendson, Stephen	Chair

LEFT JOIN

Cú pháp:

```
SELECT field1, field2, field3
FROM first_table
LEFT JOIN second_table
ON first_table.keyfield = second_table.foreign_keyfield
```

Ví dụ: Để liệt kê tất cả các nhân viên và những sản phẩm của khách hàng mà họ theo dõi:

```
SELECT Employees.Name, Orders.Product
FROM Employees
LEFT JOIN Orders
ON Employees.Employee_ID=Orders.Employee_ID
```

Mệnh đề **LEFT JOIN** trả về tất cả các dòng trong bảng đầu tiên (Employees), ngay cả khi không có khóa nào của Employee_ID tìm thấy trong bảng Orders.

Kết quả:

Name	Product
Hansen, Ola	Printer
Svendson, Tove	
Svendson, Stephen	Table
Svendson, Stephen	Chair
Pettersen, Kari	

RIGHT JOIN

Cú pháp:

```
SELECT field1, field2, field3
FROM first_table
RIGHT JOIN second_table
ON first_table.keyfield = second_table.foreign_keyfield
```

Ví dụ, liệt kê tất cả các đơn hàng và nhân viên quản lý đơn hàng đó nếu có:

```
SELECT Employees.Name, Orders.Product
FROM Employees
RIGHT JOIN Orders
ON Employees.Employee_ID=Orders.Employee_ID
```

RIGHT JOIN trả về tất cả các hàng từ bảng thứ hai (bên phải) ngay cả khi các khóa của **Employee_ID** của bảng **Orders** không tìm thấy trong bảng **Employees**.

Kết quả:

Name	Product
Hansen, Ola	Printer
Svendson, Stephen	Table
Svendson, Stephen	Chair

Ví dụ, muốn biết ai đã đặt hàng máy in?

```
SELECT Employees.Name
FROM Employees
INNER JOIN Orders
ON Employees.Employee_ID=Orders.Employee_ID
WHERE Orders.Product = 'Printer'
```

Kết quả:

Name
Hansen, Ola

14. HỢP HAI BẢNG VỚI SQL UNION VÀ UNION ALL

UNION

Lệnh hợp **UNION** được sử dụng để lựa chọn thông tin liên quan đến hai bảng, khá giống với lệnh **JOIN**. Tuy nhiên, khi sử dụng lệnh **UNION** tất cả các cột được lựa chọn phải cùng kiểu dữ liệu tương ứng với nhau.

Ghi chú: Với *UNION*, chỉ có những giá trị khác nhau (*DISTINCT*) được chọn ra.

Cú pháp

Câu lệnh SQL 1 UNION Câu lệnh SQL 2
--

Ví dụ, ta có bảng *Employees_Norway Table*

Employee_ID	E_Name
01	Hansen, Ola
02	Svendson, Tove
03	Svendson, Stephen
04	Pettersen, Kari

và bảng *Employees_USA*

Employee_ID	E_Name
01	Turner, Sally
02	Kent, Clark
03	Svendson, Stephen
04	Scott, Stephen

Sử dụng lệnh *UNION*

Ví dụ: Hãy liệt kê tất cả tên nhân viên khác nhau trong bảng *Norway* và *USA*:

```
SELECT E_Name FROM Employees_Norway
UNION
SELECT E_Name FROM Employees_USA
```

Kết quả:

Name
Hansen, Ola
Svendson, Tove

Svendson, Stephen
Pettersen, Kari
Turner, Sally
Kent, Clark
Scott, Stephen

Ghi chú: Lệnh này không thể sử dụng để liệt kê tất cả các nhân viên trong Norway và USA. Trong ví dụ ở trên chúng ta có hai nhân viên trùng tên nhưng chỉ có một trong số đó là được liệt kê ra. Lệnh UNION như đã nêu chỉ lựa chọn những giá trị phân biệt (khác nhau).

UNION ALL

UNION ALL gần giống lệnh UNION, chỉ có điều UNION ALL lựa chọn tất cả các giá trị.

Câu lệnh SQL 1 UNION ALL Câu lệnh SQL 2

Sử dụng UNION ALL

Liệt kê tất cả các nhân viên trong bảng Norway và USA:

```
SELECT E_Name FROM Employees_Norway
```

```
UNION ALL
```

```
SELECT E_Name FROM Employees_USA
```

Kết quả:

Name
Hansen, Ola
Svendson, Tove
Svendson, Stephen
Pettersen, Kari
Turner, Sally
Kent, Clark
Svendson, Stephen
Scott, Stephen

15. SQL TẠO CƠ SỞ DỮ LIỆU, BẢNG VÀ CHỈ MỤC (INDEX)

Tạo cơ sở dữ liệu

Để tạo ra một cơ sở dữ liệu bạn dùng lệnh:

```
CREATE DATABASE database_name
```

Bạn lưu ý, một cơ sở dữ liệu có thể chứa nhiều bảng.

Tạo bảng dữ liệu

Để tạo ra một bảng trong cơ sở dữ liệu bạn dùng lệnh sau:

```
CREATE TABLE table_name  
    (column_name1 data_type,  
    column_name2 data_type,  
    .....  
    )
```

Ví dụ dưới đây sẽ minh họa cách bạn tạo ra bảng Persons với bốn cột. Tên các cột sẽ là LastName, FirstName, Address, Age:

```
CREATE TABLE Persons  
    (LastName varchar,  
    FirstName varchar,  
    Address varchar,  
    Age int  
    )
```

bạn có thể chỉ rõ kích thước các cột như sau:

```
CREATE TABLE Persons  
    ( LastName varchar(30),  
    FirstName varchar,  
    Address varchar,  
    Age int(3)  
    )
```

Tạo chỉ mục (Index)

Có lẽ chỉ mục là phần bạn chưa học cho đến bây giờ, chỉ mục được tạo ra cho một bảng để giúp việc sắp xếp tìm kiếm nhanh chóng và hiệu quả hơn. Khi một cột được đánh chỉ mục, các thuật toán tìm kiếm sắp xếp sẽ thực hiện trên chỉ mục của cột thay vì trên dữ liệu vật lý lưu trong cột. Bạn có thể tạo chỉ mục cho một hoặc nhiều cột của bảng cùng lúc, và mỗi chỉ mục được đặt một tên riêng.

Người dùng thường không thể nhìn thấy cách lưu trữ những chỉ mục này, chúng được các hệ quản trị phần mềm cơ sở dữ liệu quản lý riêng, mục đích chỉ là tăng tốc độ truy xuất.

Ghi chú: Khi bạn cập nhật một bảng chứa chỉ mục, thời gian sẽ chậm hơn so với cập nhật một bảng không có chỉ mục, lý do là các chỉ mục cần được sắp xếp lại. Tuy nhiên sau đó thì dữ liệu lại được truy tìm nhanh hơn nhiều (có thể gấp 100 lần) so với không đánh chỉ mục. Vì vậy, chỉ nên tạo chỉ mục cho các cột thường sử dụng cho tìm kiếm.

Chỉ mục duy nhất

Chỉ mục duy nhất là chỉ mục trong đó hai hàng không thể có cùng giá trị chỉ mục. Để tạo chỉ mục duy nhất trên một bảng bạn dùng lệnh sau:

```
CREATE UNIQUE INDEX index_name  
ON table_name (column_name)
```

Column_name là cột mà bạn muốn lập chỉ mục.

Chỉ mục đơn giản

Khi không dùng từ khóa UNIQUE, các giá trị chỉ mục được phép trùng, đây chính là dạng chỉ mục thông dụng và đơn giản. Để tạo chỉ mục đơn giản bạn sử dụng lệnh sau:

```
CREATE INDEX index_name  
ON table_name (column_name)
```

Column_name là cột mà bạn muốn lập chỉ mục.

Ví dụ dưới đây sẽ tạo ra một chỉ mục đơn giản, mang tên PersonIndex, chỉ mục được đánh trên cột LastName:


```
CREATE INDEX PersonIndex  
ON Person (LastName)
```

Nếu muốn sắp chỉ mục các giá trị trong một cột theo thứ tự giảm dần, bạn có thể thêm từ khóa DESC sau tên cột:

```
CREATE INDEX PersonIndex  
ON Person (LastName DESC)
```

Nếu muốn sắp chỉ mục trên nhiều cột, bạn có thể liệt kê tên các cột bên trong dấu ngoặc, phân cách bởi dấu phẩy:

```
CREATE INDEX PersonIndex  
ON Person (LastName, FirstName):
```

16. SQL DROP XÓA BẢNG, CHỈ MỤC VÀ CƠ SỞ DỮ LIỆU

DROP INDEX - Xóa chỉ mục

Bạn có thể xóa một chỉ mục hiện hữu trong bảng với lệnh DROP INDEX.

```
DROP INDEX table_name.index_name
```

Xóa bỏ hẳn một bảng hoặc Cơ sở dữ liệu

Để xóa một bảng (tất cả cấu trúc bảng, cột, và chỉ mục của bảng cũng sẽ bị xóa) bạn dùng lệnh DROP TABLE:

```
DROP TABLE table_name
```

Để xóa một cơ sở dữ liệu (tất cả bảng, chỉ mục...) bạn dùng lệnh:

```
DROP DATABASE database_name
```

17. SQL THAY ĐỔI CẤU TRÚC BẢNG

Để thay đổi cấu trúc bảng bạn dùng lệnh ALTER TABLE. Lệnh ALTER TABLE được sử dụng để thêm hoặc xóa các cột trong một bảng dữ liệu.

ALTER TABLE table_name

ADD column_name datatype

ALTER TABLE table_name

DROP COLUMN column_name

Ghi chú: Một số hệ thống cơ sở dữ liệu không cho phép xóa cấu trúc cột trong bảng dữ liệu sau khi dữ liệu đã thêm vào bảng (DROP COLUMN).

Person Table

LastName	FirstName	Address
Pettersen	Kari	Storgt 20

Ví dụ, để thêm một cột tên City vào bảng Person:

ALTER TABLE Person ADD City varchar(30)

Kết quả:

LastName	FirstName	Address	City
Pettersen	Kari	Storgt 20	

Để xóa cột Address trong bảng Person:

ALTER TABLE Person DROP COLUMN Address

Kết quả:

LastName	FirstName	City
Pettersen	Kari	

18. HÀM SQL

SQL cung cấp rất nhiều hàm nội tại cho phép đếm, tính tổng, thống kê.

Cú pháp hàm:

```
SELECT function(column) FROM table
```

Các loại hàm

Các hàm trong SQL có thể phân loại theo nhóm như sau:

- *Hàm tổng (Aggregate Functions):* Đây là các hàm tính tổng trên tập hợp và trả về một giá trị đơn duy nhất. Việc tính tổng có thể là phép đếm (COUNT), phép cộng (SUM), trung bình (AVG)

Ghi chú: Nếu sử dụng giữa nhiều biểu thức khác trong danh sách của lệnh SELECT, lệnh SELECT phải có mệnh đề nhóm GROUP BY kèm theo (chúng ta sẽ học về GROUP BY ngay bên dưới).

- *Hàm vô hướng (Scalar Functions):* Hàm vô hướng thường dùng tính toán trên một giá trị đơn và trả về giá trị đơn. Ví dụ NOW(), LEN().

Mỗi hệ quản trị cơ sở dữ liệu đều có những hàm cài đặt cụ thể và có thể khác nhau, nhìn chung các hàm tổng thì hầu hết là giống nhau.

Danh sách các hàm tổng của SQL

Hàm	Mô tả
AVG(column)	Trả về trị trung bình của các giá trị cột.
COUNT(column)	Đếm số dòng (không tính giá trị NULL) của cột.
COUNT(*)	Trả về số dòng được chọn từ SELECT.
MAX(column)	Trả về giá trị lớn nhất.
MIN(column)	Trả về giá trị nhỏ nhất.
SUM(column)	Trả về tổng các giá trị của cột.

Danh sách các hàm vô hướng trong MS Access và SQL Server

Hàm	Mô tả
UCASE(c)	Chuyển chữ thường thành chữ hoa.
LCASE(c)	Chuyển chuỗi thành chữ thường.
MID(c,start,end))	Copy chuỗi con trong một chuỗi.
LEN(c)	Trả về chiều dài chuỗi.
INSTR(c)	Xác định vị trí ký tự trong chuỗi.
LEFT(c,number_of_char)	Cắt chuỗi từ bên trái.
RIGHT(c,number_of_char)	Cắt chuỗi từ bên phải.
ROUND(c,decimals)	Hàm làm tròn số.
MOD(x,y)	Hàm chia lấy phần dư.
NOW()	Hàm trả về ngày tháng hiện hành.
FORMAT(c,format)	Hàm định dạng.
DATEDIFF(d,date1,date2)	Hàm so sánh ngày tháng và thời gian.

Trước khi xem các ví dụ về những hàm này, bạn cần tìm hiểu thêm mệnh đề nhóm và lọc theo nhóm dùng trong câu lệnh SELECT. Các hàm tổng của SQL thường phải sử dụng chung với những mệnh đề này.

19. SQL GROUP BY VÀ HAVING

Những hàm tổng của SQL như SUM(), COUNT() thường cần mệnh đề GROUP BY để nhóm các thành phần cần tính tổng lại.

GROUP BY:

GROUP BY được thêm vào cấu trúc lệnh SQL do các hàm tính tổng thường trả về tổng của tất cả giá trị trong một cột, nếu không có GROUP BY để nhóm dữ liệu lại, bạn sẽ không biết được và không có cách tính tổng của từng thành phần riêng lẻ.

Cú pháp GROUP BY kết hợp với các hàm tổng như sau:

SELECT column,SUM(column) **FROM** table **GROUP BY** column

Dưới đây là ví dụ sử dụng hàm tính tổng và GROUP BY

Sales Table

Company	Amount
W3Schools	5500
IBM	4500
W3Schools	7100

SELECT Company, SUM(Amount) FROM Sales

Kết quả:

Company	SUM(Amount)
W3Schools	17100
IBM	17100
W3Schools	17100

Kết quả trên có lẽ không phải là điều bạn mong muốn. Lý do là bạn muốn tính tổng cho từng nhóm W3Schools và IBM trong khi lệnh SELECT lại tính tổng tất cả các dòng. Đó là do ta chưa sử dụng GROUP BY. Mệnh đề GROUP BY sẽ giải quyết vấn đề này như sau:

SELECT Company,SUM(Amount) FROM Sales
GROUP BY Company

Kết quả:

Company	SUM(Amount)
W3Schools	12600
IBM	4500

HAVING...

HAVING được thêm vào cấu trúc lệnh SQL do mệnh đề WHERE không thể sử dụng trong các hàm tổng, và nếu không có HAVING chúng ta không thể nào kiểm tra các kết quả trong từng nhóm.

Cú pháp HAVING như sau:

SELECT column,SUM(column) FROM table



GROUP BY column

HAVING SUM(column) condition value

Ví dụ, bạn có bảng Sales như sau:

Company	Amount
W3Schools	5500
IBM	4500
W3Schools	7100

Câu lệnh SQL:

```
SELECT Company,SUM(Amount) FROM Sales
```

```
GROUP BY Company
```

```
HAVING SUM(Amount)>10000
```

Kết quả trả về như sau:

Company	SUM(Amount)
W3Schools	12600

20. SQL VÀ LỆNH SELECT INTO

Lệnh SELECT INTO thường được sử dụng để tạo bảng copy của bảng hoặc trích rút những dòng mẫu tin lưu trữ sang bảng khác.

Cú pháp:

```
SELECT column_name(s) INTO newtable FROM sourcetable
```

Ví dụ sau sẽ chép tất cả các mẫu tin trong bảng Persons sang bảng mới Persons_backup:

```
SELECT * INTO Persons_backup FROM Persons
```

Nếu chỉ muốn sao chép một số cột, bạn có thể liệt kê chúng sau lệnh SELECT, ví dụ:

```
SELECT LastName,FirstName INTO Persons_backup
FROM Persons
```

Bạn cũng có thể thêm mệnh đề WHERE để lọc những mẫu tin nào cần thiết phải sao lưu. Ví dụ sau tạo ra một bảng Persons_backup với hai cột (FirstName và LastName), dữ liệu là các dòng có tên thành phố là Sandnes từ bảng Persons

```
SELECT LastName,Firstname INTO Persons_backup
FROM Persons
WHERE City='Sandnes'
```

Trích lọc dữ liệu từ nhiều bảng cũng hoàn toàn hợp lệ. Ví dụ sau tạo ra một bảng Empl_Ord_backup chứa dữ liệu từ hai bảng Employees và Orders:

```
SELECT Employees.Name,Orders.Product
INTO Empl_Ord_backup
FROM Employees
INNER JOIN Orders
ON Employees.Employee_ID=Orders.Employee_ID
```

21. SQL CREATE VIEW LỆNH TẠO KHUNG NHÌN (VIEW) CỦA DỮ LIỆU

View là gì? Trong SQL, View là một bảng ảo dựa trên tập kết quả trả về từ một hoặc nhiều bảng của lệnh SELECT. View chứa hàng và cột, giống như một bảng thật. Các trường (hay cột) trong một View là các trường từ một hoặc nhiều bảng khác trong cơ sở dữ liệu. Bạn có thể thêm các hàm SQL, mệnh đề lọc WHERE, hay phép kết nối JOIN vào một View và hiển thị dữ liệu như thể là dữ liệu phát xuất từ một bảng đơn duy nhất.

Chú ý: Thiết kế bảng và cấu trúc bảng sẽ **KHÔNG** ảnh hưởng bởi các hàm của View, hoặc mệnh đề WHERE, JOIN mà View sử dụng.

Cú pháp

```
CREATE VIEW view_name AS
```

```
SELECT column_name(s)
FROM table_name
WHERE condition
```

Lưu ý: Cơ sở dữ liệu không cất giữ dữ liệu của View! Mỗi khi bạn muốn xem nội dung của View, chương trình quản trị cơ sở dữ liệu sẽ tái tạo dữ liệu của View bằng cách sử dụng các phát biểu **SELECT** mà View định nghĩa.

Sử dụng View

Một View có thể được sử dụng câu truy vấn **SELECT**, từ các thủ tục Store Procedure hay thậm chí từ một View khác. Cơ sở dữ liệu Northwind mẫu của SQL Server và MS Access có một số View mẫu được thiết đặt theo mặc định. Ví dụ View (mà) “Current Product List” liệt kê tất cả các sản phẩm đang sử dụng trong bảng Products. View này được tạo ra bằng lệnh SQL sau:

```
CREATE VIEW (Current Product List) AS
SELECT ProductID, ProductName
FROM Products
WHERE Discontinued=No
```

Chúng ta có thể truy vấn nội dung View như một bảng dữ liệu thông thường:

```
SELECT * FROM (Current Product List)
```

Một View khác trong cơ sở dữ liệu mẫu Northwind trích lọc tất cả sản phẩm trong bảng Products có đơn giá cao hơn đơn giá trung bình, View này được tạo như sau:

```
CREATE VIEW (Products Above Average Price) AS
SELECT ProductName, UnitPrice
FROM Products
WHERE UnitPrice > (SELECT AVG(UnitPrice) FROM Products)
```

Chúng ta có thể truy vấn nội dung View ở trên như sau:

```
SELECT * FROM (Products Above Average Price)
```


View ví dụ khác từ cơ sở dữ liệu Northwind tính toán toàn bộ doanh số hàng bán theo chủng loại trong năm vào 2005. Chú ý rằng View này chọn và trích rút dữ liệu từ một View khác mang tên “Product Sales for 2005”:

```
CREATE VIEW (Category Sales For 2005) AS
SELECT DISTINCT CategoryName,
                Sum(ProductSales) AS CategorySales
FROM (Product Sales for 2005)
GROUP BY CategoryName
```

Chúng ta có thể truy vấn View ở trên như sau:

```
SELECT * FROM (Category Sales For 2005)
```

Chúng ta cũng có thể thêm điều kiện lọc WHERE vào câu truy vấn View. Ví dụ, để muốn xem toàn bộ hàng bán của chủng loại hàng mang tên “Beverages”:

```
SELECT * FROM (Category Sales For 2005)
WHERE CategoryName='Beverages'
```

22. SQL THAM KHẢO NHANH

Dưới đây là tóm tắt các cú pháp SQL mà bạn có thể nhớ:

Lệnh	Cú pháp
AND / OR	SELECT column_name(s) FROM table_name WHERE condition AND OR condition
ALTER TABLE (add column)	ALTER TABLE table_name ADD column_name datatype
ALTER TABLE (drop column)	ALTER TABLE table_name DROP COLUMN column_name
AS (alias for column)	SELECT column_name AS column_alias FROM table_name
AS (alias for table)	SELECT column_name FROM table_name AS table_alias
BETWEEN	SELECT column_name(s) FROM table_name

	WHERE column_name BETWEEN value1 AND value2
CREATE DATABASE	CREATE DATABASE database_name
CREATE INDEX	CREATE INDEX index_name ON table_name (column_name)
CREATE TABLE	CREATE TABLE table_name (column_name1 data_type, column_name2 data_type,)
CREATE UNIQUE INDEX	CREATE UNIQUE INDEX index_name ON table_name (column_name)
CREATE VIEW	CREATE VIEW view_name AS SELECT column_name(s) FROM table_name WHERE condition
DELETE FROM	DELETE FROM table_name DELETE FROM table_name WHERE condition
DROP DATABASE	DROP DATABASE database_name
DROP INDEX	DROP INDEX table_name.index_name
DROP TABLE	DROP TABLE table_name
GROUP BY	SELECT column_name1,SUM(column_name2) FROM table_name GROUP BY column_name1
HAVING	SELECT column_name1,SUM(column_name2) FROM table_name GROUP BY column_name1 HAVING SUM(column_name2) condition value
IN	SELECT column_name(s) FROM table_name WHERE column_name IN (value1,value2,...)
INSERT INTO	INSERT INTO table_name VALUES (value1, value2, ..) hay

	INSERT INTO table_name (column_name1, column_name2,...) VALUES (value1, value2,...)
LIKE	SELECT column_name(s) FROM table_name WHERE column_name LIKE pattern
ORDER BY	SELECT column_name(s) FROM table_name ORDER BY column_name (ASC DESC)
SELECT	SELECT column_name(s) FROM table_name
SELECT *	SELECT * FROM table_name
SELECT DISTINCT	SELECT DISTINCT column_name(s) FROM table_name
SELECT INTO	SELECT * INTO new_table_name FROM original_table_name hay SELECT column_name(s) INTO new_table_name FROM original_table_name
UPDATE	UPDATE table_name SET column_name=new_value (, column_name=new_value) WHERE column_name=some_value
WHERE	SELECT column_name(s) FROM table_name WHERE condition

23. KIỂM TRA NHANH

Bạn có thể kiểm tra những kỹ năng hiểu biết về SQL của mình bằng 20 câu hỏi đơn giản sau:

1. SQL viết tắt của những từ gì?

Structured Question Language

Structured Query Language

Strong Question Language

2. Câu lệnh SQL nào được sử dụng để trích rút dữ liệu từ một bảng trong cơ sở dữ liệu?

GET

OPEN

EXTRACT

SELECT

3. Câu lệnh SQL nào được sử dụng để cập nhật dữ liệu trong một bảng của cơ sở dữ liệu?

UPDATE

MODIFY

SAVE

SAVE AS

4. Câu lệnh SQL nào được sử dụng để xóa dữ liệu khỏi một bảng của cơ sở dữ liệu?

DELETE

REMOVE

COLLAPSE

5. Câu lệnh SQL nào được sử dụng để chèn dữ liệu mới vào một bảng trong cơ sở dữ liệu?

INSERT INTO

ADD RECORD

ADD NEW

INSERT NEW

6. Câu lệnh SQL nào sau đây bạn có thể lựa chọn được dữ liệu từ một cột có tên **FirstName** từ bảng có tên **Persons**?

```
SELECT FirstName FROM Persons
EXTRACT FirstName FROM Persons
SELECT Persons.FirstName
```

7. Với câu SQL nào sau đây, bạn có thể lựa chọn tất cả các cột từ bảng **Persons**?

```
SELECT Persons
SELECT (all) FROM Persons
SELECT * FROM Persons
```

8. Với lệnh SQL nào sau đây, bạn có thể lựa chọn tất cả các dòng mẫu tin từ một bảng có tên **Persons** với yêu cầu là giá trị cột **FirstName** “Peter”?

```
SELECT (all) FROM Persons WHERE FirstName LIKE 'Peter'
SELECT * FROM Persons WHERE FirstName LIKE 'Peter'
SELECT (all) FROM Persons WHERE FirstName='Peter'
SELECT * FROM Persons WHERE FirstName='Peter'
```

9. Với lệnh SQL nào sau đây, bạn có thể lựa chọn tất cả các dòng mẫu tin từ một bảng có tên **Persons** với yêu cầu giá trị của cột **FirstName** trong dòng mẫu tin bắt đầu bằng ký tự ‘a’?

```
SELECT * FROM Persons WHERE FirstName LIKE '%a'
SELECT * FROM Persons WHERE FirstName='a'
SELECT * FROM Persons WHERE FirstName='%a%'
SELECT * FROM Persons WHERE FirstName LIKE 'a%'
```

10. Toán tử OR hiển thị các dòng mẫu tin nếu có một trong những điều kiện chỉ định thỏa mãn. Toán tử AND hiển thị một dòng mẫu tin nếu tất cả các điều kiện chỉ định đều thỏa mãn.

◊ Đúng

◊ Sai

11. Với lệnh SQL nào sau đây, bạn có thể chọn được tất cả các dòng mẫu tin từ bảng mang tên “Persons” trong đó các dòng mẫu tin phải thỏa mãn điều kiện FirstName tên là ‘Peter’ và LastName tên là ‘Jackson’?

SELECT FirstName='Peter', LastName='Jackson' FROM Persons

SELECT * FROM Persons WHERE FirstName='Peter' AND LastName='Jackson'

SELECT * FROM Persons WHERE FirstName LIKE 'Peter' AND LastName LIKE 'Jackson'

12. Với lệnh SQL nào sau đây, bạn có thể lựa chọn tất cả các dòng mẫu tin từ bảng Persons, trong đó LastName được sắp xếp theo thứ tự abc và tên lấy ra phải nằm giữa hai tên “Hansen” và “Pettersen” ?

SELECT LastName>'Hansen' AND LastName<'Pettersen' FROM Persons

SELECT * FROM Persons WHERE LastName BETWEEN 'Hansen' AND 'Pettersen'

SELECT * FROM Persons WHERE LastName>'Hansen' AND LastName<'Pettersen'

13. Câu lệnh SQL nào sau đây được sử dụng để trả lại chỉ những dòng mang giá trị khác nhau?

SELECT DIFFERENT

SELECT UNIQUE

SELECT DISTINCT

14. Từ khóa SQL nào sau đây được sử dụng để sắp xếp kết quả trả về ?

SORT BY

ORDER BY

SORT

ORDER

15. Với lệnh SQL nào sau đây, bạn có thể trả trích rút ra các dòng mẫu tin từ bảng Persons được sắp xếp thứ tự giảm dần theo giá trị của cột FirstName?

SELECT * FROM Persons SORT 'FirstName' DESC

SELECT * FROM Persons ORDER FirstName DESC

SELECT * FROM Persons SORT BY 'FirstName' DESC

SELECT * FROM Persons ORDER BY FirstName DESC

16. Với lệnh SQL nào sau đây, bạn có thể chèn một dòng mẫu tin mới vào bảng Persons?

INSERT INTO Persons VALUES ('Jimmy', 'Jackson')

INSERT VALUES ('Jimmy', 'Jackson') INTO Persons

INSERT ('Jimmy', 'Jackson') INTO Persons

17. Với lệnh SQL nào sau đây, bạn có thể chèn một dòng mẫu tin mới vào bảng Persons với tên LastName phải là "Olsen" ?\$}

INSERT INTO Persons (LastName) VALUES ('Olsen')

INSERT ('Olsen') INTO Persons (LastName)

INSERT INTO Persons ('Olsen') INTO LastName

18. Lệnh nào sau đây thay đổi tên LastName trong bảng Persons từ "Hansen" thành "Nilsen" ?

MODIFY Persons SET LastName='Nilsen' WHERE LastName='Hansen'

UPDATE Persons SET LastName='Hansen' INTO LastName='Nilsen'

MODIFY Persons SET LastName='Hansen' INTO LastName='Nilsen'

UPDATE Persons SET LastName='Nilsen' WHERE LastName='Hansen'

19. Với lệnh SQL nào sau đây, bạn có thể xóa những dòng mẫu tin với điều kiện FirstName trong bảng Persons bằng "Peter" ?

DELETE ROW FirstName='Peter' FROM Persons

DELETE FROM Persons WHERE FirstName = 'Peter'

DELETE FirstName='Peter' FROM Persons

20. Với lệnh SQL nào sau đây bạn có thể đếm tổng các dòng mẫu tin trong bảng Persons?

SELECT COLUMNS() FROM Persons

SELECT COLUMNS(*) FROM Persons

SELECT COUNT(*) FROM Persons

SELECT COUNT() FROM Persons

cuu duong than cong. com

cuu duong than cong. com

Chương 4:

CƠ SỞ DỮ LIỆU ACCESS

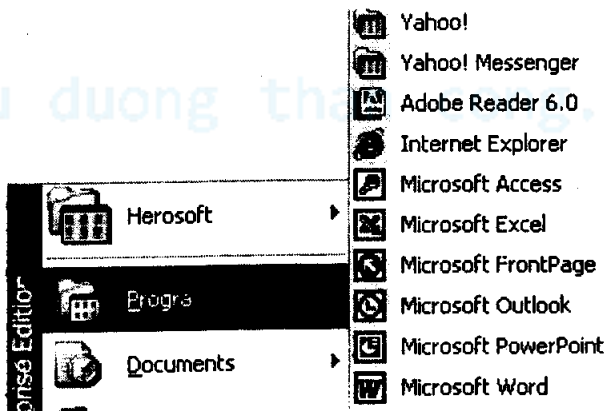
Các vấn đề chính sẽ được đề cập:

- ✓ Giới thiệu MS Access.
- ✓ Tự tạo cơ sở dữ liệu.
- ✓ Tạo cơ sở dữ liệu từ Template.
- ✓ Các thành phần cơ sở dữ liệu.

Microsoft Access là một hệ ứng dụng quản trị cơ sở dữ liệu cho phép bạn tạo và quản lý cơ sở dữ liệu trên nền hệ điều hành Windows. MS Access có thể được sử dụng cho mục đích quản lý thông tin cá nhân, thông tin trong một doanh nghiệp vừa và nhỏ. Nó cho phép tổ chức, quản lý tất cả dữ liệu. Access còn có thể dùng ở quy mô xí nghiệp để giao tiếp với các máy chủ Server.

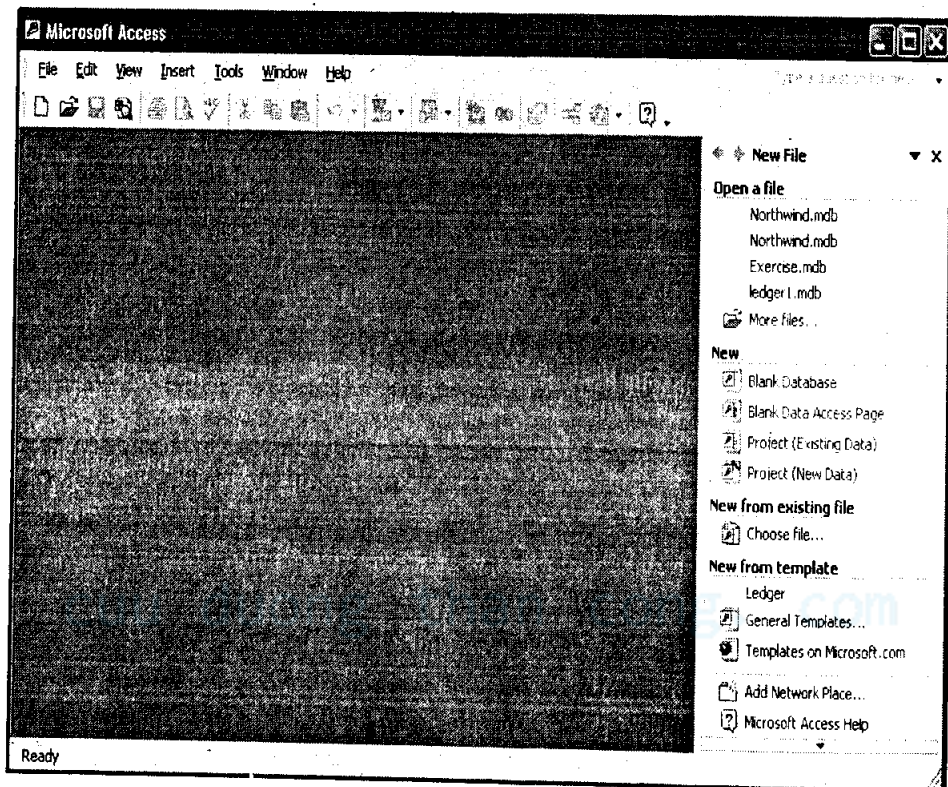
Như mọi ứng dụng máy tính khác, để sử dụng MS Access thì bạn phải cài đặt nó. MS Access nằm trong bộ ứng dụng văn phòng Microsoft Office. Trong các phiên bản Access, Microsoft nâng cấp rất nhiều tính năng về giao diện nhưng trong giáo trình này chúng ta chỉ sử dụng những tính năng cơ bản nhất của một hệ quản trị cơ sở dữ liệu mà Access cung cấp. Những tính năng này là hầu như không đổi và tương thích với hầu hết phiên bản Access.

Để khởi động chương trình này, bạn có thể kích chọn Start -> All Programs -> Microsoft Access



Hình 4-1: Khởi động MS Access

Cửa sổ chương trình Access XP/2002 chính xuất hiện như sau:



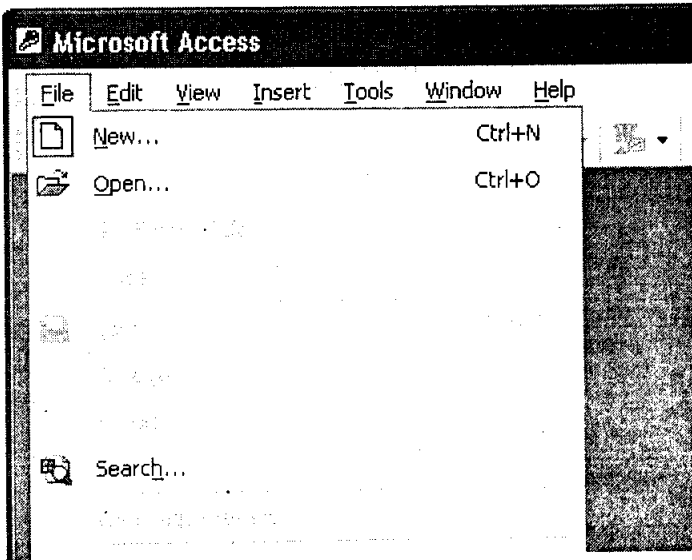
Hình 4-2

Có nhiều cách tạo cơ sở dữ liệu, bạn có thể tự tạo cơ sở dữ liệu của riêng mình hoặc chọn cơ sở dữ liệu mẫu từ thư viện Template của Microsoft. MS Access lưu thông tin các đối tượng của cơ sở dữ liệu như bảng, chỉ mục, form, module ... trong một file duy nhất có tên mở rộng là .mdb

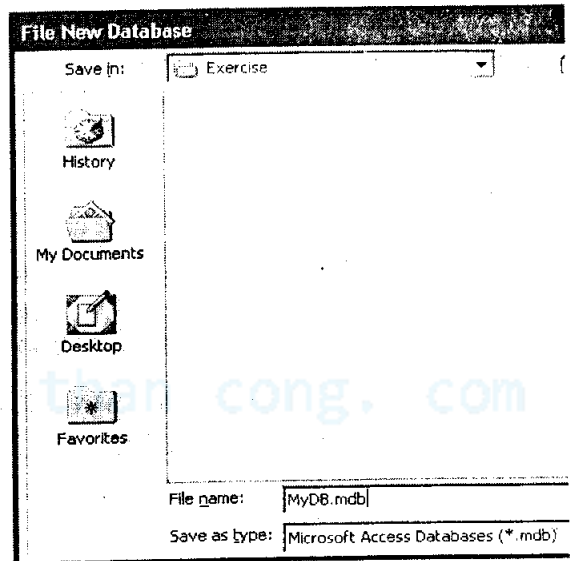
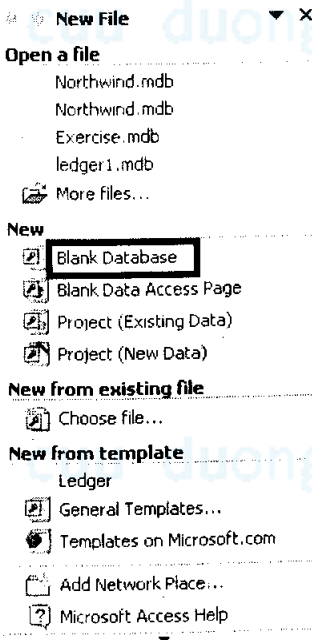
1. TẠO CƠ SỞ DỮ LIỆU (DATABASE)

1.1. Cơ sở dữ liệu tự tạo

1. Bạn chọn File | New từ menu chính của MS Access.
2. Access hiển thị cửa sổ Wizard cho phép bạn chọn tạo mới cơ sở dữ liệu (Hình 4-3). Bạn chọn mục Blank Database ...
3. Hộp thoại tạo file xuất hiện. Bạn gõ vào tên file sẽ đặt cho cơ sở dữ liệu. Ví dụ MyDB.mdb.

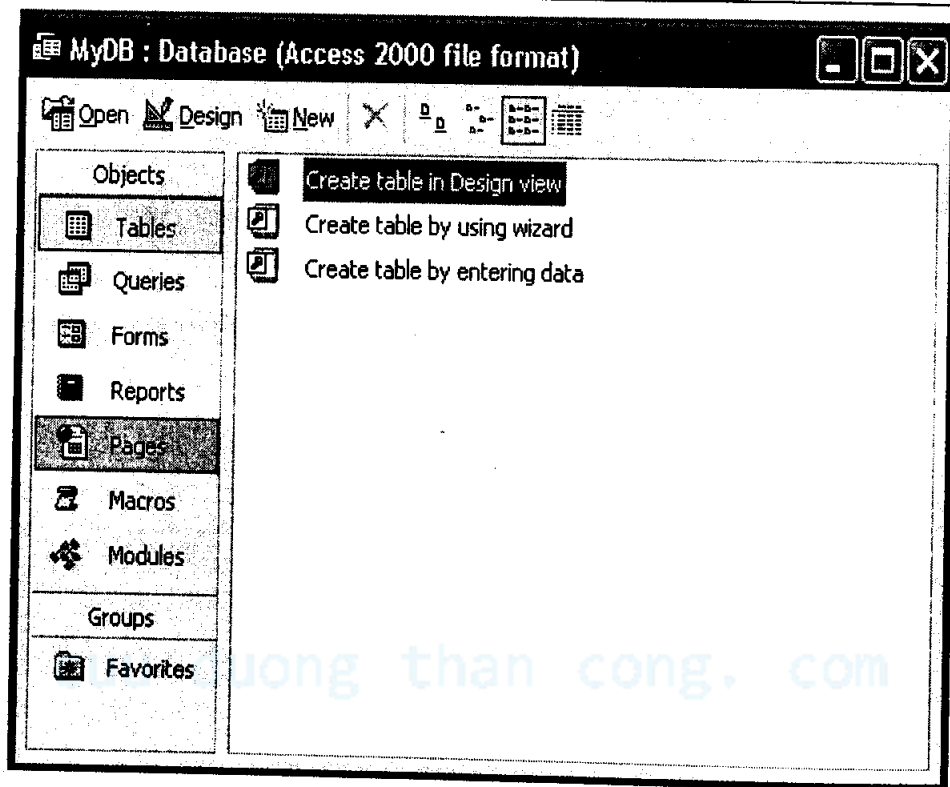


Hình 4-2-1



Hình 4-3

4. Bạn nhấn nút Create, một cơ sở dữ liệu trống được tạo ra để bạn bắt đầu sử dụng.



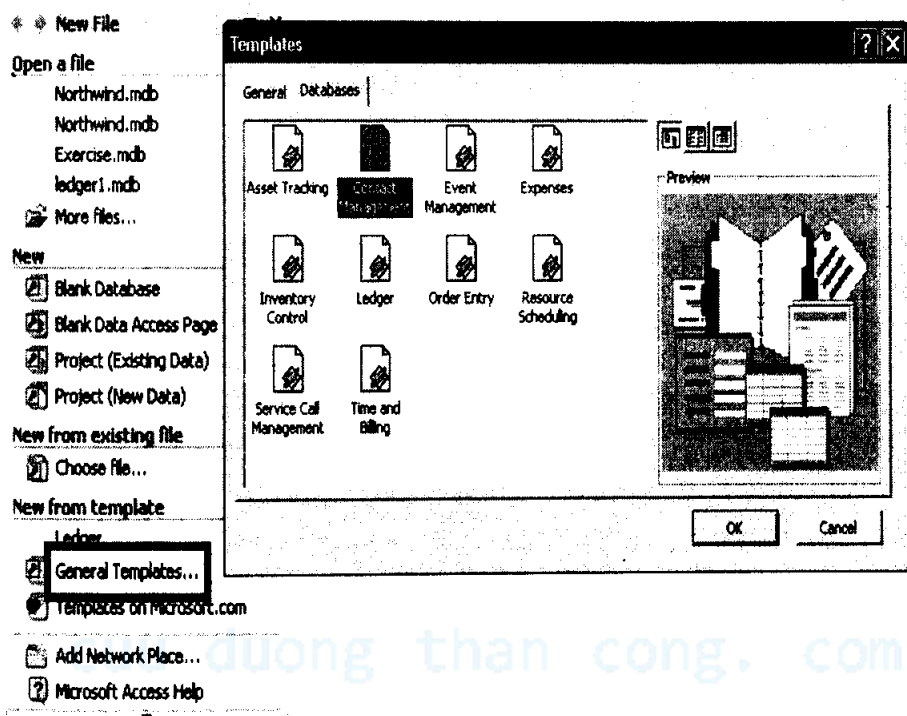
Hình 4-4

Như bạn thấy chúng ta có thể tạo các đối tượng bảng (Tables), câu lệnh SQL (Queries). Các phần mà chúng ta chưa học đó là Forms, Reports, Macros, Modules ... đây là những thành phần dùng để lập trình, tương tác, biến cơ sở dữ liệu thành ứng dụng. Xin chúc mừng, bạn đã tạo được một cơ sở dữ liệu đầu tiên của mình. Tuy nhiên chúng ta chưa có bảng dữ liệu nào cả. Bạn sẽ bắt đầu tạo bảng cho cơ sở dữ liệu sau.

1.2. Tạo cơ sở dữ liệu từ Template

Microsoft cung cấp cho bạn rất nhiều mẫu (Template) cơ sở dữ liệu. Sở dĩ gọi là cơ sở dữ liệu mẫu vì khi bạn chọn tạo cơ sở dữ liệu theo cách này thì Microsoft sẽ tạo sẵn cho bạn các bảng, các quan hệ, forms, reports theo một nghiệp vụ cụ thể nào đó.

Từ cửa sổ New Database File (Hình 4-3) bạn chọn *General Template...* khi cửa sổ tạo Template xuất hiện bạn chọn trang Database. Danh mục các cơ sở dữ liệu mẫu xuất hiện (hình 4-5)

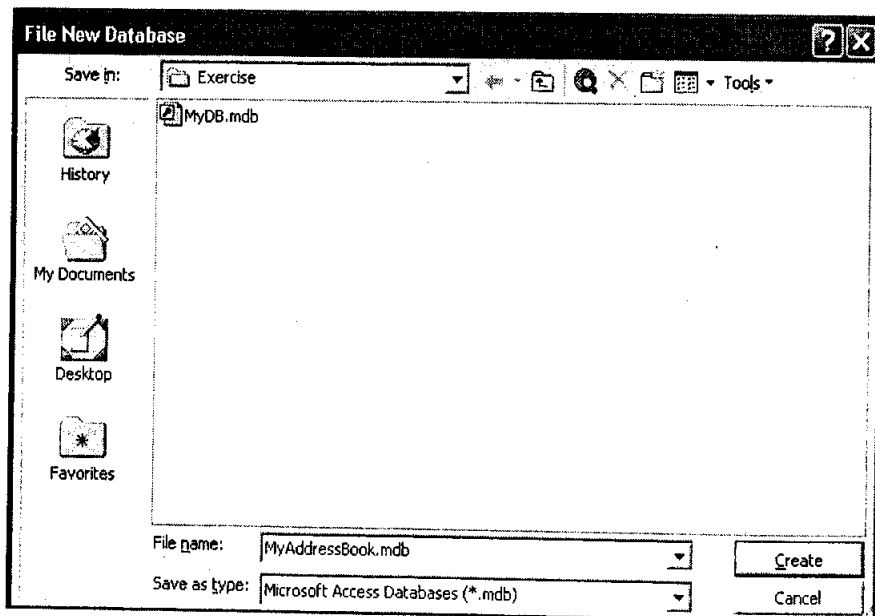


Hình 4-5

Kích chọn Database mẫu mà bạn muốn tạo, sau đó theo các hướng dẫn của trình Wizard, bạn sẽ có một Database mẫu với đầy đủ các bảng cần thiết, form, reports và gần như Microsoft đã xây dựng cho bạn một ứng dụng hoàn chỉnh dựa trên Database mẫu được chọn.

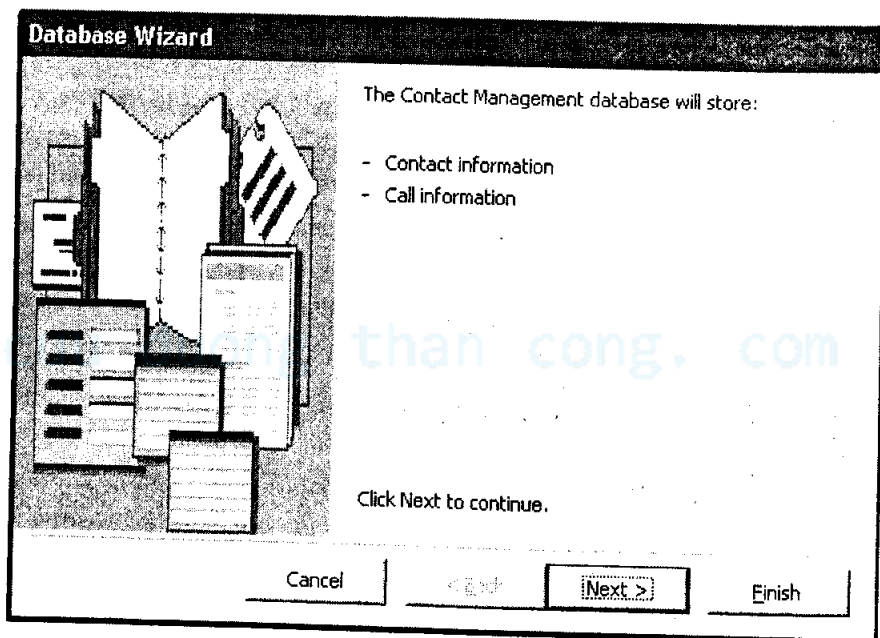
Ví dụ bạn hãy chọn Contact Management và nhấn OK, chúng ta sẽ tạo cơ sở dữ liệu chứa các bảng về quản lý thông tin địa chỉ tương tự quyển sổ địa chỉ Address Book trong điện thoại di động của bạn.

1. Chọn Contact Management từ cửa sổ Template.
2. Hộp thoại File New Database xuất hiện, bạn hãy đặt tên cho cơ sở dữ liệu quản lý thông tin địa chỉ của mình. Ví dụ MyAddressBook.mdb. Nhấn nút Create. Trình Database Wizard của Microsoft Access sẽ lần lượt hướng dẫn bạn hiệu chỉnh các thông số cần thiết trước khi tạo ra cơ sở dữ liệu với đầy đủ các tính năng.



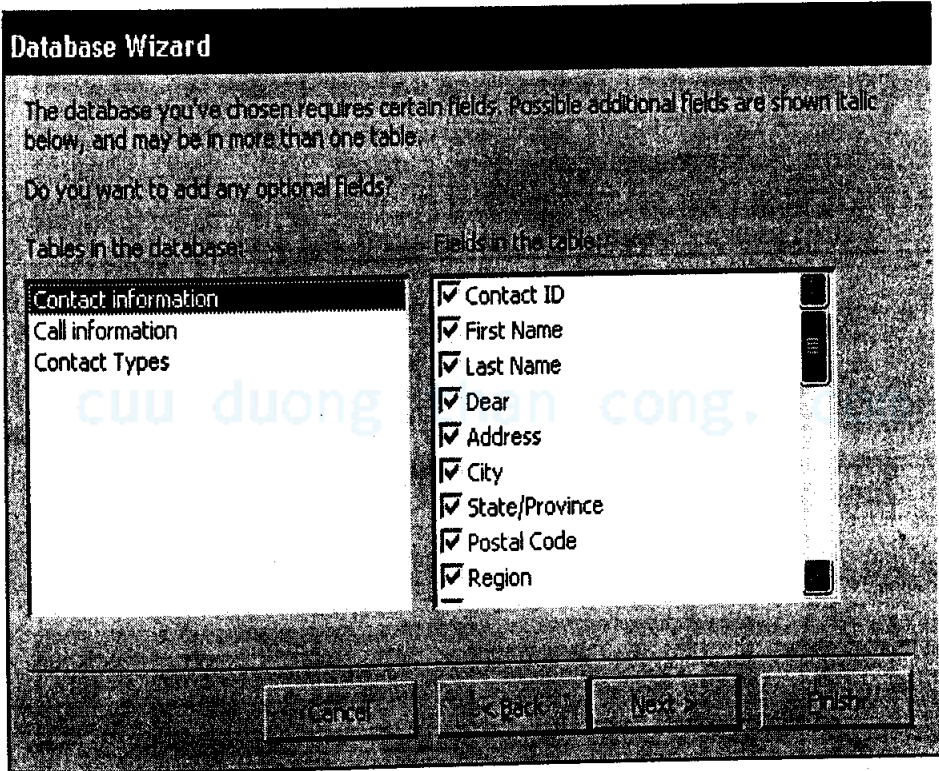
Hình 4-6

3. Cửa sổ Database Wizard xuất hiện, bạn nhấn Next để bắt đầu (hình 4-7).



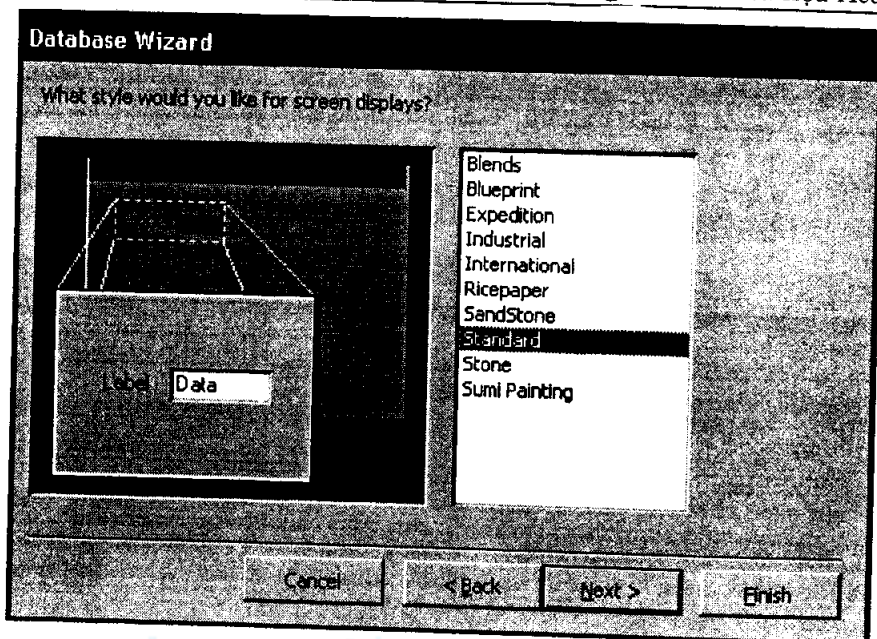
Hình 4-7

4. Trong bước tiếp theo (hình 4-8), với Template mẫu Contact Management, MS Access tạo sẵn cho bạn 3 bảng mang tên Contact information, Call information, Contact Types. Bạn có thể chọn hoặc loại bỏ các cột dữ liệu tương ứng của mỗi bảng trong danh sách bên phải. Muốn bỏ đi cột dữ liệu nào của bảng bạn bỏ dấu chọn đi. Nhấn Next

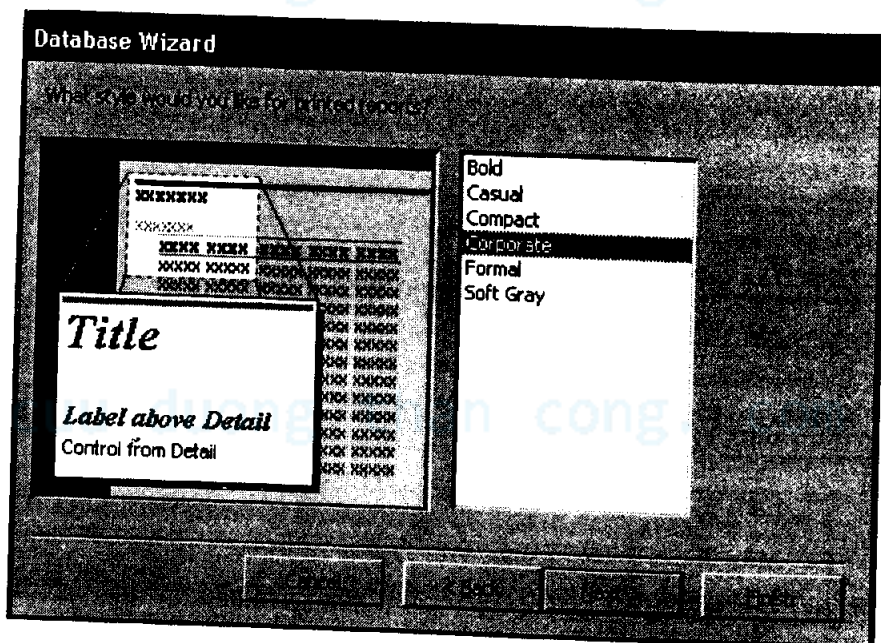


Hình 4-8

5. Bước kế tiếp (hình 4-9), MS Access bắt đầu tạo giao diện cho phép bạn nhập liệu. Cửa sổ xác định những kiểu giao diện mẫu xuất hiện bạn chọn Standard và nhấn Next
6. Màn hình Wizard cho phép bạn tạo Reports (các mẫu báo cáo trình bày kết xuất của dữ liệu) xuất hiện (hình 4-10). Bạn giữ nguyên mặc định mà Access đã chọn và nhấn nút Next



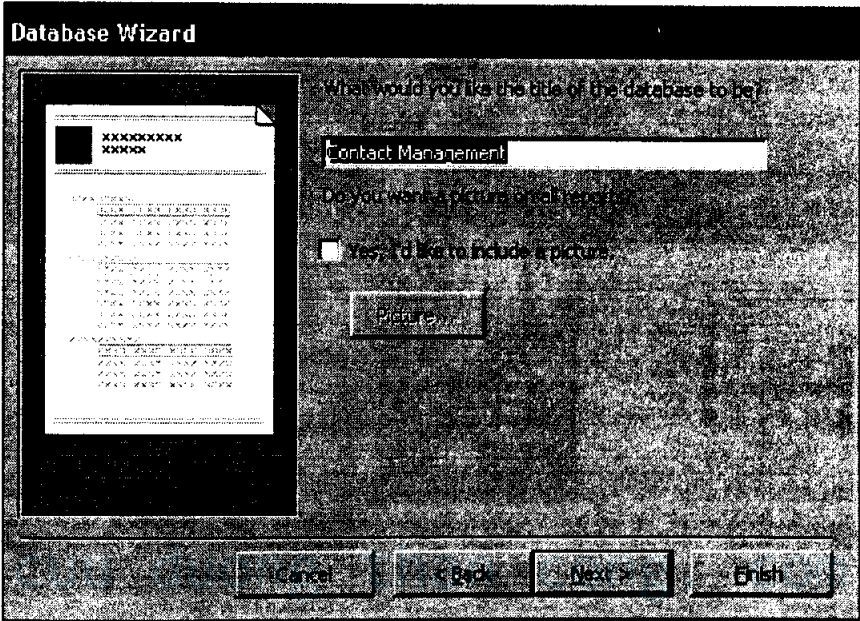
Hình 4-9



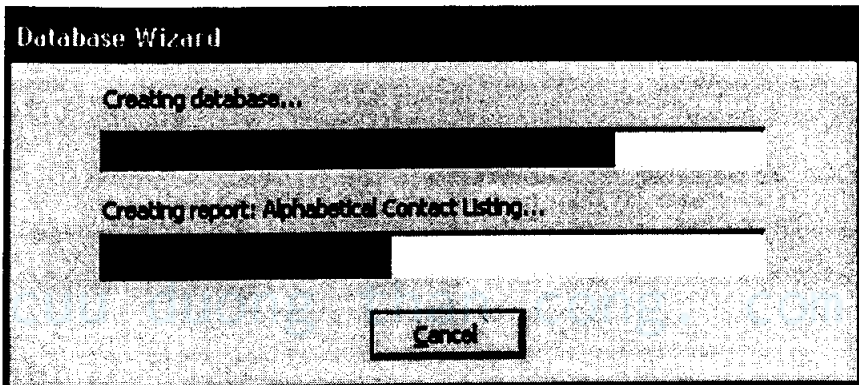
Hình 4-10

7. Bước sau cùng, bạn có thể đặt tên cho cơ sở dữ liệu của mình, tên này sẽ là tiêu đề hiển thị trên các màn hình nhập liệu, báo cáo...

(hình 4-11). Nhấn Finish. MS Access bắt đầu kiến tạo cơ sở dữ liệu đã được bạn tùy biến.



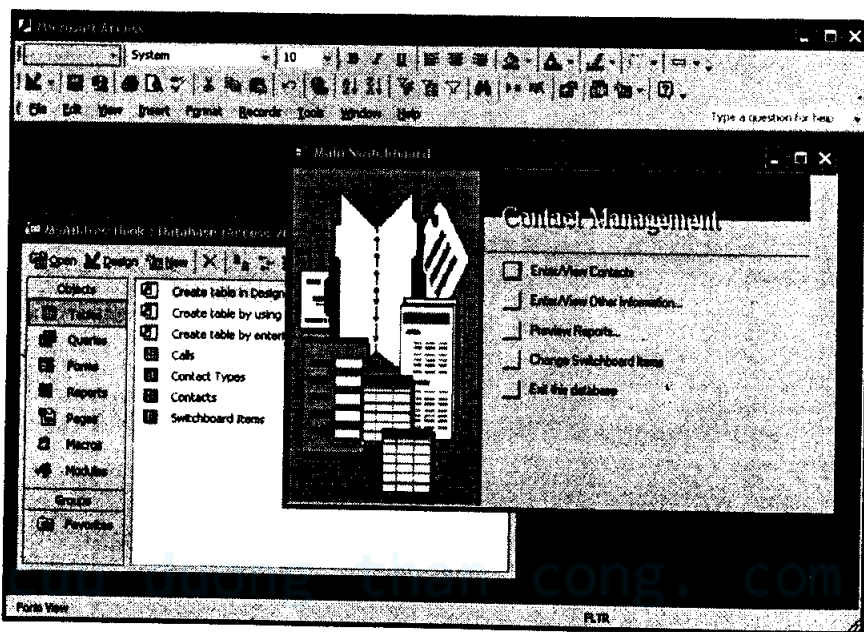
Hình 4-11



Hình 4-12

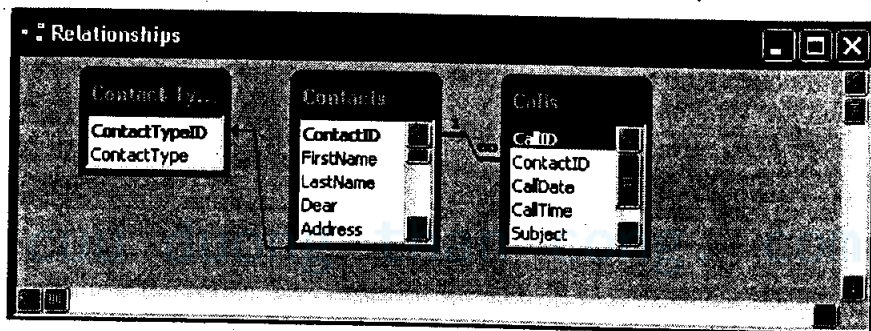
Cuối cùng bạn đã có một cơ sở dữ liệu mẫu để quản lý thông tin địa chỉ và các quan hệ hoàn chỉnh (hình 4-13). Cơ sở dữ liệu này ngoài các bảng dữ liệu Tables ra còn có các thành phần như Queries, Forms, Reports, Macros, Modules. Mặc dù giá trị trình này chúng tôi không chủ định hướng dẫn bạn sử dụng các chức năng của MS Access (bạn có thể tìm hiểu chúng trong các giáo trình chuyên sâu về MS Access mà nhà sách Minh Khai đã xuất bản)

nhưng bạn sẽ hiểu chúng cần thiết như thế nào, và dùng làm gì ngay trong phần sau.



Hình 4-13

Để xem quan hệ giữa các bảng, bạn chọn mục Tools | Relationships.. từ menu. Cửa sổ biểu diễn quan hệ giữa các bảng xuất hiện như hình 4-14.



Hình 4-14

Chương 5:

LẬP TRÌNH KẾT NỐI CƠ SỞ DỮ LIỆU

Các vấn đề chính sẽ được đề cập:

- ✓ Cơ sở dữ liệu trong những ngôn ngữ lập trình.
- ✓ Cursors.
- ✓ Các lời gọi API.
- ✓ Các thành phần liên kết dữ liệu trực quan.
- ✓ Sử dụng bảng tính Spread sheet.
- ✓ Sử dụng PHP và MySQL.
- ✓ Các ưu điểm khi sử dụng chuẩn API.
- ✓ APIs thông dụng.

ODBC - Open Database Connectivity

JDBC

DBI / DBD

ADO

- ✓ Tính hiệu quả.

1. CÁC HỆ NGÔN NGỮ QUẢN TRỊ CƠ SỞ DỮ LIỆU THẾ HỆ 4

Một số hệ quản trị cơ sở dữ liệu quan hệ cung cấp một môi trường lập trình rất đầy đủ (như MS Access, Oracle, Paradox, Foxpro). Những môi trường lập trình như vậy thường được gọi là ngôn ngữ lập trình thế hệ thứ 4 (Fourth Generation Language hay 4GL). Thuật ngữ 4GL nhanh chóng được sử dụng rộng rãi. Một hệ thống hỗ trợ 4GL sẽ bao gồm:

- Một hạt nhân xử lý cơ sở dữ liệu (Database Engine) thường không thấy được bởi người dùng, thậm chí cả người lập trình.
- Một cơ chế cho phép tạo bảng, nhập dữ liệu vào các dòng trong bảng, tạo khóa (Primary Key, Foreign Key), định dạng dữ liệu khi nhập.

- Một bộ công cụ để tạo ra ứng dụng, thường là tạo biểu mẫu Form, báo cáo Report, chúng cho phép nhập liệu và trình bày dữ liệu. Form và Report do lập người dùng khởi dữ liệu thô. Người dùng sẽ xem và chỉ tiếp cận với dữ liệu đã được qua xử lý.
- Bộ trình biên dịch có thể tạo ra một ứng dụng thực thụ kèm theo cơ sở dữ liệu.
- Hỗ trợ cú pháp ngôn ngữ nội tại (như Foxpro sử dụng ngôn ngữ Fox đặc chủng, MS Access sử dụng ngôn ngữ lập trình VBA, Oracle sử dụng PL/SQL...).

Hệ thống 4GL có thể dùng để phát triển những ứng dụng nhanh và mạnh, nó thường được tích hợp với các môi trường phát triển trực quan kiểu kéo và thả (RAD). Tuy nhiên:

- Chính tốc độ phát triển của nó có thể gây ra sự khó khăn về lâu dài - những thứ ban đầu được xem là chuẩn nguyên mẫu (prototype) sau đó lại nhanh chóng trở thành sản phẩm thương mại.
- Việc sở hữu riêng sản phẩm của các công ty phát triển có thể gây ra vấn đề chẳng hạn việc cập nhật thường xuyên các tính năng mới khiến cho sản phẩm càng xa rời chuẩn chung ban đầu.
- Một số môi trường lập trình kiểu này lại ràng buộc sản phẩm vào những hệ điều hành đặc thù nào đó (như chỉ chạy trên môi trường Windows).
- Nhà cung cấp lại luôn thay đổi phiên bản khiến người phát triển khó có thể làm việc được trên phiên bản mới nhất được lâu. Và khi hệ thống lớn lên, thật khó có thể tìm được một chuyên gia có thể nắm bắt hết tất cả tính năng của môi trường phát triển tích hợp mọi thứ, ví dụ như Oracle.

2. CƠ SỞ DỮ LIỆU TRONG NHỮNG NGÔN NGỮ LẬP TRÌNH KHÁC

Thay vì phát triển ứng dụng cơ sở dữ liệu trong một môi trường ngôn ngữ loại 4GL (như MS Access, dBase, Oracle), chúng ta có thể phát triển ứng dụng trong một ngôn ngữ tổng quát như C/C++, Pascal, Visual Basic, Delphi và liên kết môi trường ngôn ngữ lập trình với hạt nhân xử lý cơ sở dữ liệu của các hệ quản trị RDBMS. Có một số cách thực hiện điều này:

- Sử dụng SQL embedding.

- Sử dụng API (giao diện lập trình ứng dụng) như ODBC.
- Cách tiếp cận lập trình trực quan (Visual Basic, Delphi...).

Tất cả cách tiếp cận trên đều liên quan đến một khái niệm chung gọi là cursor.

Cursors

Cursor được xem như một con trỏ đến bảng dữ liệu (hoặc View). Nó thường giúp người lập trình duyệt qua và định vị các mẫu tin trong bảng. Các trường (hay cột dữ liệu) được truy cập từ Cursor thông qua:

- ◇ Điều khiển Textox.
- ◇ Các biến trong chương trình.
- ◇ Các lời gọi hàm API.

3. CÁC HÀM API GIAO TIẾP CƠ SỞ DỮ LIỆU

Hàm API (giao diện lập trình ứng dụng) truy xuất cơ sở dữ liệu là một tập hợp các định nghĩa hàm cho phép chương trình ứng dụng kết nối và giao tiếp với hạt nhân xử lý dữ liệu của một hệ quản trị. Các thao tác tiêu biểu mà API thường thực hiện bao gồm:

- *Connect* (kết nối) – Xác định máy tính cần kết nối tới và tên user cùng với mật khẩu người dùng để xác thực với hạt nhân xử lý dữ liệu Database Engine.
- *Execute* – Gửi một câu lệnh SQL tới Database Engine. Sau khi thực thi trình ứng dụng thường nhận lại một Cursor trả về từ Database Engine chứa tập dữ liệu kết quả.
- *Fetch* (lấy ra) – Đọc một hàng (hay dòng) dữ liệu từ Cursor trả về từ phát biểu SQL sau khi thực thi.
- *Navigate* (di chuyển) – Di chuyển đến các dòng hay vị trí khác trong Cursor.
- *Test* (kiểm tra) – Kiểm tra xem đã vượt qua dòng cuối cùng trong Cursor hay dòng đầu tiên của Cursor chưa.
- *Close* (đóng) – Đóng kết nối trả lại tài nguyên cho hệ thống.

Việc xây dựng một chuẩn API thống nhất có thể giúp người lập trình nhiều lợi thế. Lý tưởng, một người lập trình có thể viết và biên dịch một

chương trình sử dụng sản phẩm cơ sở dữ liệu đặc thù của Microsoft (như SQL Server hay Access) sau đó chuyển sang nhà cung cấp cơ sở dữ liệu khác (như Oracle) mà không cần dùng đến hàm truy xuất cơ sở dữ liệu khác và không phải thay đổi nhiều mã chương trình. Do cả hai nhà sản xuất cung cấp một cài đặt chung của chuẩn API nên mã chương trình làm việc như nhau trong cả hai hệ thống.

- Người lập trình sử dụng SQL không theo tiêu chuẩn sẽ mất đi tính linh hoạt này.
- Những nhà cung cấp cơ sở dữ liệu không tốt sẽ khiến người lập trình sử dụng SQL hoặc API không theo chuẩn SQL dễ dàng khóa mã chương trình của bạn và trói buộc bạn vào một sản phẩm duy nhất của họ.

Ví dụ, các ngôn ngữ lập trình khác nhau đều sử dụng cùng một cách để đọc, duyệt các mẫu tin.

Delphi:

```
Table1.First;
```

```
while not Table1.EOF do begin
```

```
    Memo1.Lines.Add(Table1.FieldName('NAME').AsString);
```

```
    Table1.Next;
```

```
end;
```

```
Table1.Close;
```

Visuale Basic:

```
Table1.MoveFirst
```

```
while not Table1.EOF
```

```
    Memo1.Text= Memo1.Text & Table1('NAME');
```

```
    Table1.NextNext;
```

```
end;
```

```
Table1.Close;
```

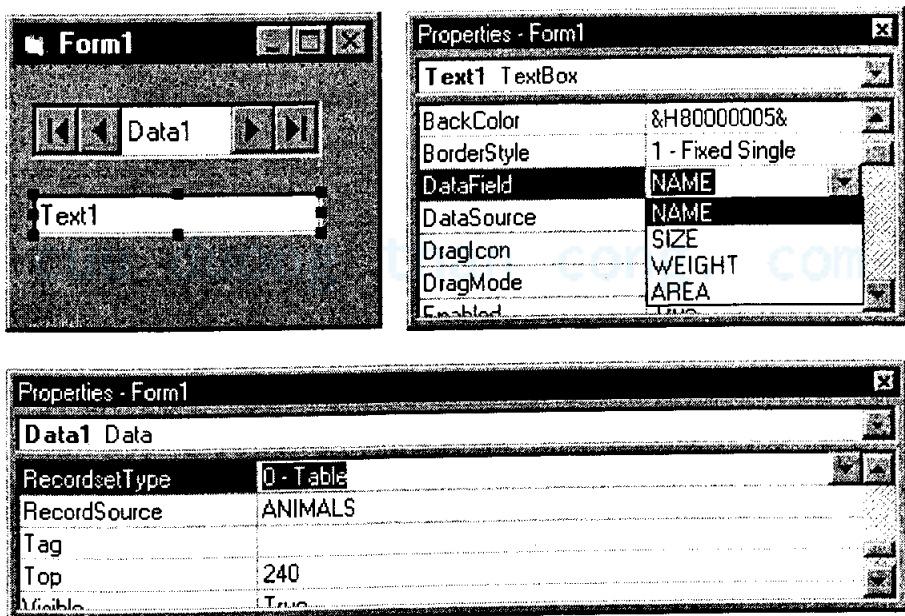
Đoạn mã trên đây gần như là một mẫu cho thao tác duyệt và xử lý các mẫu tin trong Cursor đọc về được từ một bảng.

- Di chuyển về dòng đầu tiên của Cursor.
- Sử dụng vòng lặp đọc dòng hiện hành.

- Di chuyển đến dòng tiếp theo.
- Kiểm tra đã đến hết dòng cuối chưa (EOF hay End Of File). Nếu chưa thì lặp lại thao tác đọc.

4. LIÊN KẾT DỮ LIỆU VỚI THÀNH PHẦN TRỰC QUAN

Ví dụ này cho thấy cách kết nối một ô nhập liệu văn bản liên kết dữ liệu với dữ liệu nguồn chỉ bằng cách kéo thả trong Visual Basic. Các môi trường lập trình trực quan khác cũng có cơ chế tương tự.



Hình 5-1

- Nguồn dữ liệu Data1 cung cấp thuộc tính "Record Source" được đặt bằng tên bảng ANIMALS trong cơ sở dữ liệu.
- Data1 đóng vai trò như một Cursor - mũi tên cho phép người dùng di chuyển tới lui các Record hay mẫu tin trong bảng vào lúc chương trình thực thi.
- Text1 là một liên kết dữ liệu hoặc thành phần nhận thức được dữ liệu (data-aware).

- Thuộc tính `Text1.DataSource` được đặt bằng `Data1`, thuộc tính `Text1.DataField` được đặt bằng cột dữ liệu sẽ hiển thị "Name".
- Khi vị trí hiện hành của các mẫu tin trong `Cursor` di chuyển, dữ liệu trình bày trong `Text1` được cập nhật để phản ánh đúng nội dung của dòng hiện thời.
- `Text1` có thể cũng được thiết lập để tự động cập nhật cơ sở dữ liệu nếu người dùng thực hiện việc thay đổi hay hiệu chỉnh trên đó.

5. SỬ DỤNG BẢNG TÍNH SPREADSHEET

Bảng tính (Spread Sheet) như Excel chẳng hạn cũng được xem là cơ sở dữ liệu cho phép thực hiện các toán tử quan hệ đơn giản (bạn hình dung Workbook là cơ sở dữ liệu còn các Sheet trong đó là các bảng dữ liệu). Ví dụ:

`=VLOOKUP(B1, Sheet2!$A$1:$B$3, 2)`

Chúng ta có thể xem bảng tính như một cơ sở dữ liệu quan hệ khi tuân theo một số quy tắc sau:

- Giữ mỗi dòng một mẫu tin (1NF).
- Sử dụng hàm `VLOOKUP` để truy tìm chỉ số trong bảng tính khác.

6. SỬ DỤNG NGÔN NGỮ LẬP TRÌNH WEB PHP VÀ CƠ SỞ DỮ LIỆU MYSQL

MySQL là một hệ cơ sở dữ liệu có mã nguồn mở được sử dụng khá phổ biến. MySQL thật sự không cung cấp các giao diện Form, Report như MS Access, Oracle nhưng nó là một Database Engine cực kỳ hiệu quả. Bạn có thể tham khảo các giáo trình hướng dẫn sử dụng MySQL và PHP do nhà sách Minh Khai xuất bản. Ở đây, chúng tôi sẽ ví dụ cách kết nối MySQL từ ngôn ngữ lập trình PHP.

Linux/UNIX

```
/usr/local/mysql/bin/mysql -h
```

Windows

```
C:\mysql\bin>mysql
```

```
mysql> use world
```

```
mysql> show tables;
```


Tables_in_worlddb
Nations
Countries

2 rows in set (0.05 sec)

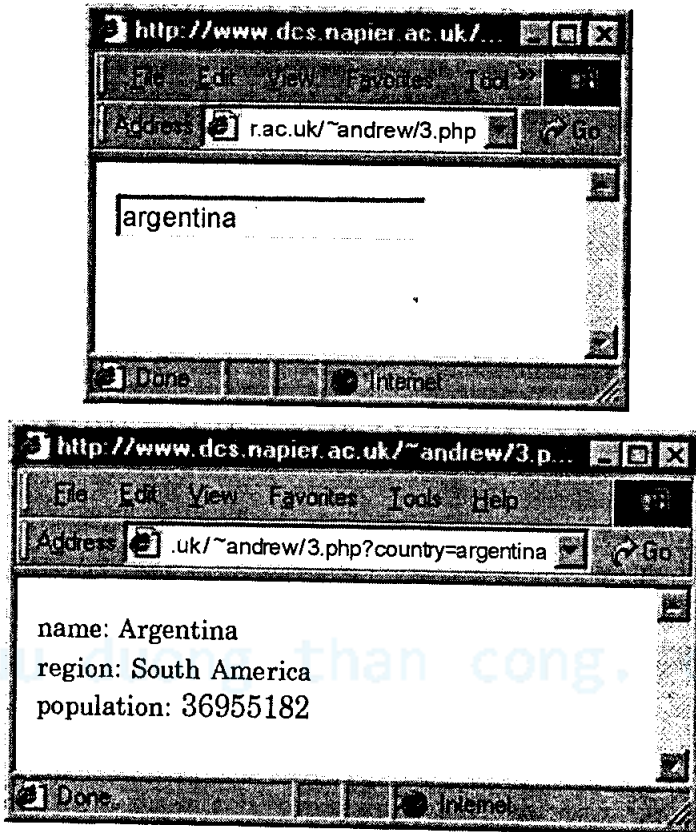
mysql> select * from Countries where
population>200000000;

name	region	area	population	gdp
China	Asia	9596960	1261832482	4800000000000
India	Asia	3287590	1014003817	1805000000000
Indonesia	Southeast Asia	1919440	224784210	610000000000
United States	North America	9629091	275562673	9255000000000

4 rows in set (0.11 sec)

PHP Code

```
<?php
if ($country) {
    $link = mysql_connect("admin","worlddb","*****") or die("Could not connect");
    mysql_select_db("andrew") or die("Could not select database");
    $query = "SELECT name, region, population FROM cia WHERE
name='$country'";
    $result = mysql_query($query) or die("Query failed");
    while ($row = mysql_fetch_array($result)) {
        extract($row);
        print "name: $name<br>\n";
        print "region: $region<br>\n";
        print "population: $population<br>\n";
    }
    print "</table>\n";
    mysql_free_result($result);
    mysql_close($link);
}else{
    print "<form><input name='country'></form>\n";
}
?>
```



Hình 5-2

7. SQL EMBEDDED

Một số ngôn ngữ lập trình như C, sử dụng kỹ thuật nhúng (Embedded) câu lệnh SQL vào mã chương trình, dưới đây là đoạn chương trình C sử dụng SQL nhúng.

Chương trình minh họa này tuy khá đầy đủ nhưng không trình bày hết được các kỹ thuật SQL nhúng. Chương trình yêu cầu người dùng nhập vào số đơn đặt hàng, trích rút ra mã số khách hàng, người bán hàng và tình trạng của đơn hàng, sau đó hiển thị thông tin đọc được ra màn hình.

```
main()
{
    EXEC SQL INCLUDE SQLCA;
    EXEC SQL BEGIN DECLARE SECTION;
```

```
int OrderID; /* Employee ID (from user) */
int CustID; /* Retrieved customer ID */
char SalesPerson[10] /* Retrieved salesperson name */
char Status[6] /* Retrieved order status */
EXEC SQL END DECLARE SECTION;

/* Set up error processing */
EXEC SQL WHENEVER SQLERROR GOTO query_error;
EXEC SQL WHENEVER NOT FOUND GOTO bad_number;

/* Prompt the user for order number */
printf ("Enter order number: ");
scanf ("%d", &OrderID);

/* Execute the SQL query */
EXEC SQL SELECT CustID, SalesPerson, Status
FROM Orders
WHERE OrderID = :OrderID
INTO :CustID, :SalesPerson, :Status;

/* Display the results */
printf ("Customer number: %d\n", CustID);
printf ("Salesperson: %s\n", SalesPerson);
printf ("Status: %s\n", Status);
exit();

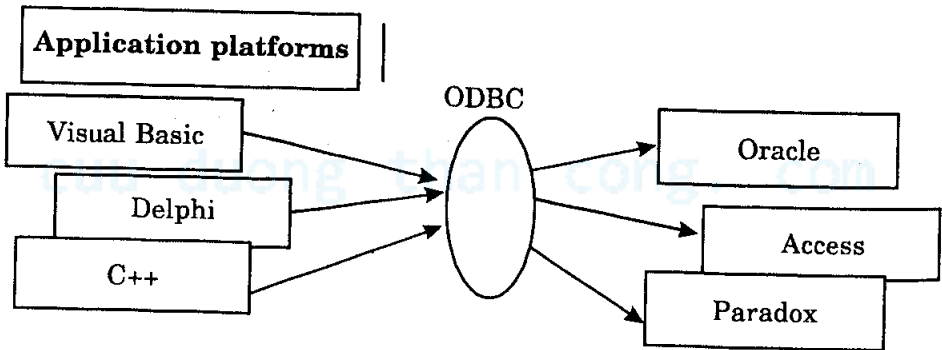
query_error:
printf ("SQL error: %ld\n", SQLCA.SQLCODE);
exit();

bad_number:
```

```
printf("Invalid order number.\n");  
exit();  
}
```

8. ƯU ĐIỂM CỦA CHUẨN API

Một tập API lý tưởng là tập các hàm có thể nối nhiều nền hay môi trường lập trình với nhiều cài đặt cơ sở dữ liệu khác nhau. Nó cho phép những người thiết kế ứng dụng kết nối và truy nhập được đến nhiều cơ sở dữ liệu khác nhau theo cùng cách. Nó cũng cho phép các nhà sản xuất hệ quản trị cơ sở dữ liệu cung cấp một giao diện thống nhất cho tất cả các nền ứng dụng. Microsoft cung cấp cho bạn chuẩn API mang tên ODBC thực hiện công việc này.

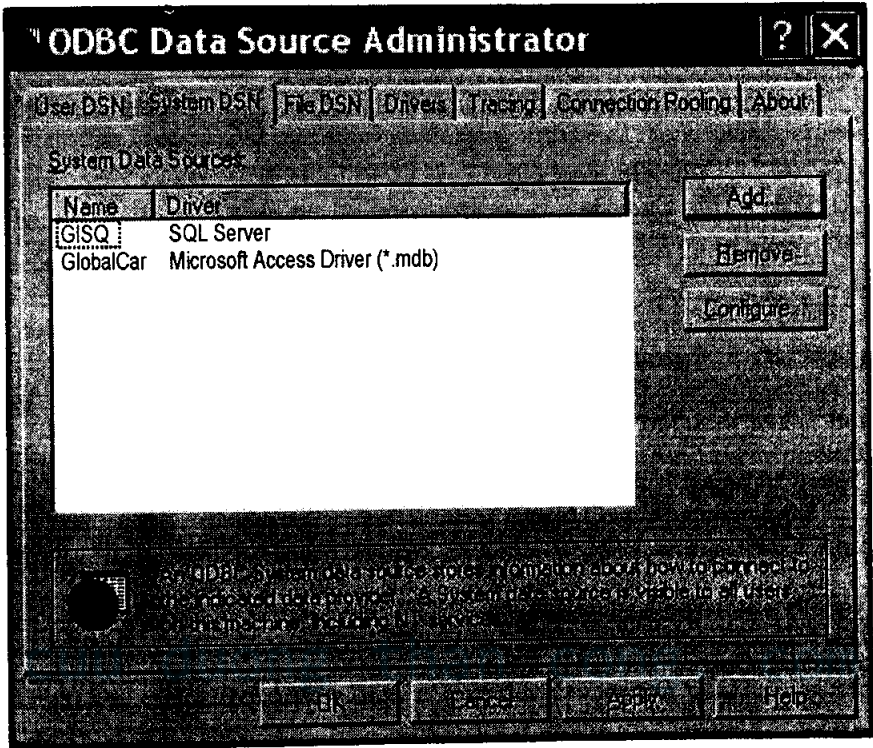


API ODBC

Nếu không có một chuẩn chung mỗi ngôn ngữ phải cung cấp một giao diện truy xuất đến cơ sở dữ liệu đặc thù và điều này là một ác mộng đối với lập trình viên. Chính vì ý nghĩa làm cầu nối, ODBC mang tên là **Open DataBase Connectivity** thường gọi tắt là cầu nối truy xuất cơ sở dữ liệu.

9. ODBC - OPEN DATABASE CONNECTIVITY

Chuẩn này được đặc tả bởi Microsoft và phần lớn liên quan đến hoạt động trên nền Windows. ODBC bao gồm một tập thủ tục để nối kết tới các Database Engine khác nhau của các cơ sở dữ liệu. Thông số kết nối ODBC có thể được thiết lập từ Control Panel của Windows. Đây là một trong những chuẩn API thành công và thông dụng nhất. Hầu hết các ngôn ngữ lập trình đều có thể tạo kết nối ODBC mà không cần biết cơ sở dữ liệu đặc thù là gì. Và đa số các sản phẩm cơ sở dữ liệu thương mại ngày nay đều hỗ trợ đầy đủ API ODBC.



ODBC Data Source Administrator

Để minh họa, dưới đây là cách sử dụng một số hàm API của ODBC.

10. SỬ DỤNG SQLBINDCOL (ODBC)

Ứng dụng truy xuất cột dữ liệu bằng cách gọi hàm `SQLBindCol()`. Hàm này cho phép truy xuất cột dữ liệu vào một thời điểm. Với hàm này, ứng dụng cần chỉ rõ:

- **Số cột.** Cột 0 là cột đánh dấu (bookmark), cột này thường không chứa trong tập kết quả trả về. Tất cả các cột khác đều được đánh số bắt đầu từ 1. Nếu chỉ định số cột lớn hơn tổng số cột hiện có thì ODBC sẽ báo lỗi. Lỗi này không thể phát hiện ra cho đến khi tập kết quả được trả về, vì vậy hàm `SQLFetch` sẽ trả về mã lỗi nếu không đọc được giá trị cột thay cho hàm `SQLBindCol`.
- **Kiểu dữ liệu C, địa chỉ và số byte của biến đọc dữ liệu cột.** Nếu bạn chỉ định kiểu dữ liệu C không đúng với kiểu dữ liệu SQL thì ODBC không thể chuyển đổi. Tương tự, lỗi này không thể phát hiện ra cho đến khi tập kết quả được trả về, vì vậy hàm `SQLFetch` sẽ trả

về mã lỗi nếu không chuyển đổi được giá trị cột thay cho hàm SQLBindCol.

Ví dụ, đoạn mã sau ràng buộc các biến với cột SalesPerson và CustID.

```
SQLCHAR    SalesPerson[11];
SQLINTEGER  CustID;
SQLINTEGER  SalesPersonLenOrInd, CustIDInd;

SQLRETURN  rc;

SQLHSTMT    hstmt;

// Bind SalesPerson to the SalesPerson column and CustID to the CustID
// column.
SQLBindCol(hstmt, 1, SQL_C_CHAR, SalesPerson, sizeof(SalesPerson),
           &SalesPersonLenOrInd);
SQLBindCol(hstmt, 2, SQL_C_ULONG, &CustID, 0, &CustIDInd);

// Execute a statement to get the sales person/customer of all orders.
SQLExecDirect(hstmt, "SELECT SalesPerson, CustID FROM Orders ORDER
BY SalesPerson",
SQL_NTS);

// Fetch and print the data. Print "NULL" if the data is NULL. Code to check if rc
// equals SQL_ERROR or SQL_SUCCESS_WITH_INFO not shown.
while ((rc = SQLFetch(hstmt)) != SQL_NO_DATA) {
    if (SalesPersonLenOrInd == SQL_NULL_DATA)
        printf("NULL");
    else
        printf("%10s ", SalesPerson);

    if (CustIDInd == SQL_NULL_DATA)
        printf("NULL\n");
    else
        printf("%d\n", CustID);
}

// Close the cursor.
SQLCloseCursor(hstmt);
```

Ngày nay, hiếm nhà phát triển ứng dụng nào sử dụng API của ODBC trực tiếp, các ứng dụng tạo một giao tiếp đơn giản hơn ở mức trên như ADO, ADO.NET. Tuy nhiên, nếu bạn dự định phát triển một cơ sở dữ liệu nào đặc thù thì API ODBC vẫn là các hàm cần phải xem xét và cài đặt.

11. JDBC

JDBC cung cấp cách truy xuất cơ sở dữ liệu tương tự ODBC nhưng nó đặc biệt dùng cho ngôn ngữ lập trình Java. Ví dụ sau là một chương trình Java sử dụng JDBC:

```
import java.sql.*;

public class WorldCountry{
    public static void main(String[] args){
        Connection myCon;
        Statement myStmt;
        try{
            Class.forName("com.mysql.jdbc.Driver").newInstance();
            # Connect to an instance of mysql with the follow details:
            # machine address: 127.0.0.1
            # database      : worldbv
            # user name     : admin
            # password      : 123

            myCon = DriverManager.getConnection(
                "jdbc:mysql://localhost/worldbv", "admin", "123");
            myStmt = myCon.createStatement();
            ResultSet result = myStmt.executeQuery(
                "SELECT name FROM cia WHERE population>200000000");
            while (result.next()){
                System.out.println(result.getString("name"));
            }
            myCon.close();
        }
        catch (Exception sqlEx){
            System.err.println(sqlEx);
        }
    }
}
```

```
}  
}  
}
```

Chương trình có thể được biên dịch với lệnh sau:

```
javac WorldCountry.java
```

Và sau đó thực thi bằng lệnh:

```
java WorldCountry
```

Bạn nên tham khảo thêm các giáo trình Java hướng dẫn sử dụng JDBC của nhà sách Minh Khai đã xuất bản.

12. DBI/DBD

Một tập API khác cũng khá phổ biến và thông dụng đó là giao diện DBI/DBD. Đây là một chuẩn với mục đích cung cấp cho người lập trình cách thực thi lệnh SQL với nhiều ngôn ngữ khác nhau. Nó có nhiều nét giống với ODBC, nhưng không phức tạp. Chuẩn này xuất phát từ môi trường UNIX dùng cho ngôn ngữ Perl và sau này được sử dụng bởi rất nhiều ngôn ngữ Scripting khác trên Linux và UNIX. Ví dụ đoạn mã Perl sau sẽ tìm ra họ tên của một nhân viên thuộc mã số phòng ban biết trước.

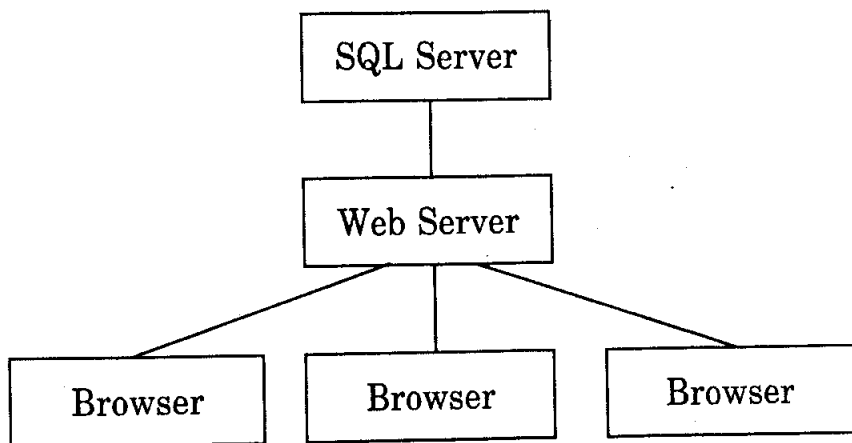
```
my $dbh = DBI->connect("dbname","username","password");  
my $depno = 3;  
my $cmd = $dbh->prepare("SELECT surname FROM employee where  
depno=?");  
my $res = $cmd->execute($depno);
```

```
while (my ($name) = $res->fetchrow_array()) {  
    print "The employee name is $name\n";  
}
```

cuu duong than cong. com

13. ADO VÀ ASP

ADO là một công nghệ truy xuất dữ liệu khác của Microsoft thay thế cho ODBC. ADO cũng cho phép truy xuất đến nhiều cơ sở dữ liệu khác nhau và cải tiến tốc độ cũng như đơn giản các hàm lập trình hơn. ADO sử dụng nhiều cho môi trường lập trình Web với trang Web ASP (Active Server Page). Mã truy xuất cơ sở dữ liệu bằng ADO rất đơn giản.



Ví dụ dưới đây là đoạn mã trang ASP hiển thị danh sách các xe hơi (Car) trong bãi đậu xe.

```

<%
SQL="SELECT carName FROM Cars ORDER BY carName"
set conn = server.createobject("ADODB.Connection")
conn.open "parking"
set cars=conn.execute(SQL)
%>

```

```

<% do while not cars.eof %>
  <%= cars(0) %> <br>
  <%cars.movenext
  loop%>
<% cars.close %>

```

14. NHỮNG VẤN ĐỀ VỀ TÍNH HIỆU QUẢ

Không cần biết kết nối đến cơ sở dữ liệu được thực hiện như thế nào, nhưng bạn chỉ cần quan tâm đến vấn đề kết nối thực hiện hiệu quả ra sao. Ứng dụng kết nối tới một cơ sở dữ liệu thường nằm trên một máy khác với máy thực thi câu lệnh, hay nói khác hơn Database Engine và trình ứng dụng thường nằm trên hai máy khác nhau. Cơ chế client-server này có nhiều lợi thế và quan trọng nếu cơ sở dữ liệu dùng chung bởi nhiều máy và nhiều người dùng. Tuy nhiên, việc trình ứng dụng và máy chủ server chạy cơ sở dữ liệu (database engine) trên hai máy khác nhau sẽ làm phát sinh vấn đề về chi phí phải trả cho tốc độ thực thi. Việc thiết lập một kết nối bao gồm thời gian đăng nhập và xác thực mật khẩu cũng có thể tốn kém về thời gian. Giao tiếp giữa máy chạy ứng dụng và máy chủ server phải

thông qua đường truyền mạng và sẽ chậm hơn đáng kể so với chỉ truy xuất trên đĩa cứng cục bộ. Việc gửi những câu lệnh SQL cho máy chủ server nhanh hơn việc nhận kết quả trả về từ máy chủ. Nhà lập trình cần phải chú ý chỉ lấy dữ liệu cần, những người lập trình “làm biếng” thường yêu cầu server các câu lệnh như “SELECT * FROM Table” nó không hiệu quả bằng việc “SELECT ColumnA FROM Table”, vì nếu Table chứa 100 cột thì tốc độ thực thi là chậm đáng kể khi dữ liệu gửi trả về từ server. Vấn đề kết nối cùng lúc nhiều người dùng cũng có thể làm server bị nghẽn. Các hệ quản trị cơ sở dữ liệu hiện đại xây dựng những Database Engine có khả năng chia sẻ kết nối (Shared Pool) với hàng ngàn truy cập, nhưng ở góc độ nhà phát triển ứng dụng, lập trình viên vẫn phải hạn chế kết nối nhiều lần đến cơ sở dữ liệu. Các ngôn ngữ lập trình Web và mô hình kết nối của ADO.NET sau này cũng chọn giải pháp chỉ kết nối với cơ sở dữ liệu một lần và ngắt ngay kết nối sau khi đã truy vấn xong dữ liệu.

cuu duong than cong. com

cuu duong than cong. com

Chương 6:

METADATA, BẢO MẬT VÀ QUẢN TRỊ (DBA)

Các vấn đề chính sẽ được đề cập:

- ✓ *Metadata (Siêu dữ liệu).*
- ✓ *Bảo mật (Security).*
- ✓ *Các thành phần bảo mật của hệ DBMS.*
- ✓ *Các mức độ bảo vệ DBMS.*
- ✓ *Bảo mật mức người dùng cho SQL.*
- ✓ *Lệnh GRANT.*
- ✓ *Quản trị cơ sở dữ liệu (Database Administrator).*

1. METADATA (SIÊU DỮ LIỆU)

Cho đến lúc này, trong hệ DBMS chúng ta chỉ mới xem đến các đối tượng bảng phục vụ cho mô hình thiết kế cơ sở dữ liệu. Chúng ta cũng đã xem qua và sử dụng đối tượng View, View được xem là một ảnh kết hợp của nhiều bảng.

Tuy nhiên, một lược đồ (Schema) hoàn chỉnh của cơ sở dữ liệu còn rất nhiều đối tượng khác ngoài bảng Table và View như các thủ tục hàm (Store Procedure), bảo mật (Security), thông tin hệ thống (System Object) ... Trong Oracle mọi thứ đều được cài đặt dưới dạng một bảng. Không chỉ những đối tượng chúng ta xem là bảng dữ liệu thuần túy mà còn mọi thứ như thông tin người dùng, thông tin hệ thống, bảo mật đều được tổ chức lưu dưới dạng bảng. Triết lý ở đây là tính đơn giản ... Cài đặt khái niệm bảng dữ liệu và sau đó xây dựng một hệ quản trị DBMS chỉ xoay quanh các bảng. Khái niệm bảo mật cũng không là ngoại lệ. Cơ sở dữ liệu Oracle (và cả những hệ DBMS lớn như SQL Server, DB2) tổ chức lưu các thông tin bảo mật trong những bảng hệ thống. Không gian lưu trữ bảng đặc biệt tablespace của Oracle mang tên SYS được dùng để lưu giữ tất cả thông tin hệ thống. Rất nhiều mức an toàn bảo vệ vùng SYS, do vậy tùy thuộc vào quyền truy nhập mà bạn có thể hoặc không thể nhìn thấy tất cả các bảng chứa trong vùng này. Vùng



SYS nắm giữ hàng trăm bảng. Danh sách dưới đây là một trong các bảng chứa thông tin quan trọng của Oracle.

USER_OBJECTS

TAB

USER_TABLES

USER_VIEWS

ALL_TABLES

USER_TAB_COLUMNS

USER_CONSTRAINTS

USER_TRIGGERS

USER_CATALOG

DBA_USERS

Ví dụ, bảng DBA_USERS nắm giữ thông tin người dùng. Trong Oracle ta có thể xem cấu trúc bảng như sau:

SQL> describe dba_users;

Name	Null?	Type
-----	-----	-----
USERNAME	NOT NULL	VARCHAR2(30)
USER_ID	NOT NULL	NUMBER
PASSWORD		VARCHAR2(30)
ACCOUNT_STATUS	NOT NULL	VARCHAR2(32)
LOCK_DATE		DATE
EXPIRY_DATE		DATE
DEFAULT_TABLESPACE	NOT NULL	VARCHAR2(30)
TEMPORARY_TABLESPACE	NOT NULL	VARCHAR2(30)
CREATED	NOT NULL	DATE
PROFILE	NOT NULL	VARCHAR2(30)
INITIAL_RSRC_CONSUMER_GROUP		VARCHAR2(30)
EXTERNAL_NAME		VARCHAR2(4000)

Bảng DBA_USERS nắm giữ tên username của người dùng. Một số ID định danh cho mỗi người dùng, mật khẩu đăng nhập của họ cùng nhiều chi tiết thông tin cá nhân quan trọng khác. Ví dụ về một bảng khác nắm giữ những thuộc tính quan trọng của hệ DBMS là bảng USER_CONSTRAINTS chứa các ràng buộc toàn vẹn dữ liệu của các bảng khác trong hệ thống.

SQL> describe user_constraints;

Name	Null?	Type
-----	-----	-----
OWNER	NOT NULL	VARCHAR2(30)
CONSTRAINT_NAME	NOT NULL	VARCHAR2(30)
CONSTRAINT_TYPE		VARCHAR2(1)
TABLE_NAME	NOT NULL	VARCHAR2(30)
.....		
DEFERRABLE		VARCHAR2(14)
DEFERRED		VARCHAR2(9)
.....		
LAST_CHANGE		DATE
.....		

ở đây, một hàng liên kết với người dùng chủ sở hữu (owner) các ràng buộc bằng tên của ràng buộc và kiểu ràng buộc. Do thông tin về ràng buộc cũng lưu trên bảng nên hệ DBMS sẽ truy xuất chúng tương tự như bạn truy xuất các bảng dữ liệu thông thường khác của mình.

Qua các ví dụ trên, như bạn thấy, bản thân các hệ DBMS cũng cần lưu lại thông tin hệ thống để sử dụng cho riêng nó. Nếu xét về góc độ người dùng, bạn sử dụng bảng (table) để lưu các thông tin dữ liệu của mình thì xét về góc độ DBMS, hệ thống sử dụng bảng để lưu thông tin mô tả cấu trúc các bảng dữ liệu của người dùng và cách sử dụng chúng. Những thông tin này được gọi là metadata (siêu dữ liệu). Metadata nói ngắn gọn là *dữ liệu dùng để mô tả dữ liệu khác*. Khai thác thông tin metadata bạn sẽ biết được rất nhiều điều bổ ích về thông tin dữ liệu mà nó mô tả.

2. BẢO MẬT (SECURITY)

Bảo mật cơ sở dữ liệu bao gồm những thao tác giúp cơ sở dữ liệu chống lại:

- ◇ Những xâm nhập hệ thống đánh cắp dữ liệu bất hợp pháp.
- ◇ Các thay đổi về nội dung và cấu trúc dữ liệu không hợp lệ.
- ◇ Các hành động phá hủy hệ thống.

Cơ chế bảo vệ an toàn cho cơ sở dữ liệu thường tập trung vào hai lớp người dùng:

- Ngăn người dùng không có quyền truy nhập cơ sở dữ liệu và không thực hiện bất kỳ hình thức tác động nào lên cơ sở dữ liệu.
- Ngăn người dùng có quyền truy nhập cơ sở dữ liệu thực hiện những hoạt động trên cơ sở dữ liệu chưa được phép.

Điều khiển mức vật lý

Bảo mật thường bắt đầu từ những điều khiển mức vật lý. Nếu một người không thể vào tòa nhà nơi cơ sở dữ liệu chạy và có khả năng truy nhập thì người đó không thể truy nhập vào cơ sở dữ liệu được. Dễ thấy nhất là các phòng đặt server chứa cơ sở dữ liệu đang chạy thường khóa trái cửa chỉ những người được phép mới vào tiếp cận.

Điều khiển bằng chính sách

Bảo mật một cơ sở dữ liệu thường là tuân theo các chính sách công ty. Tất cả các công ty cần phải có một quy định về chính sách, liệt kê điều gì được phép làm và điều gì không được phép. Những công ty với các chính sách yếu kém sẽ thường có nhiều lỗ hổng về bảo mật nhất. Ở mức thấp nhất, chính sách có thể quy định nội dung cơ sở dữ liệu và bất kỳ dữ liệu nào chứa trong đó đều không được phép tiết lộ ra bên ngoài công ty. Nếu không có quy định chính sách rõ ràng, thật khó cho rằng một nhân viên đã làm điều gì đó sai trái liên quan đến bảo mật...

Vấn đề thao tác

Nếu chỉ một người truy nhập và sử dụng cơ sở dữ liệu thì việc bảo mật sẽ an toàn và chắc chắn hơn so với hệ thống có nhiều người truy nhập. Bảo mật thường liên quan đến vấn đề thao tác sử dụng và số lượng người tương tác với cơ sở dữ liệu.

Điều khiển phân cứng

Không cần biết cơ sở dữ liệu được bảo mật an toàn như thế nào, nếu một người nào đó có thể đơn giản lấy cắp ổ cứng chứa cơ sở dữ liệu thì vấn đề bảo mật là gần như vô nghĩa vì sớm hay muộn kẻ gian cũng có thể đọc được dữ liệu. Thực hiện bảo mật bạn luôn phải quan tâm đến những vấn đề phân cứng như bảo vệ các bản sao lưu dữ liệu đã chép ra ngoài, bảo vệ các server chứa đĩa cứng.

Bảo mật hệ điều hành

Đa số các hệ DBMS chạy bên trong một hệ điều hành (OS) như Windows 95, Windows NT và Unix. Cơ sở dữ liệu có thể an toàn từ bên trong DBMS, nhưng nếu cơ sở dữ liệu cũng có thể truy nhập từ hệ điều hành thông qua các thao tác sao chép file chẳng hạn thì cơ chế bảo mật cũng không còn an toàn. Các hệ điều hành càng có cơ chế bảo mật mạnh thì cơ sở dữ liệu chạy trên nó càng đảm bảo tính an toàn.

Bảo mật hệ thống cơ sở dữ liệu

Bản thân DBMS sử dụng tài khoản bao gồm tên người dùng và mật khẩu để xác thực quyền truy cập. Quyền hạn truy cập trên từng đối tượng của cơ sở dữ liệu sẽ được phân định dựa trên tài khoản đăng nhập. Hầu như khi bạn học các giáo trình về cơ sở dữ liệu đều tập trung vào mức bảo mật này.

Các thành phần bảo mật DBMS

Những thành phần cần được bảo mật trong cơ sở dữ liệu bao gồm:

- Mức tổng thể là toàn bộ cơ sở dữ liệu.
- Tập hợp các bảng.
- Từng bảng riêng lẻ.
- Một bộ hay tập các dòng dữ liệu trong một bảng.
- Từng dòng dữ liệu lẻ.
- Tập hợp các cột của bảng.
- Một cột riêng lẻ trong bảng.

3. BẢO VỆ MỨC DBMS

Mã hóa dữ liệu

Thường thì khó cản được mọi người sao chép cơ sở dữ liệu và sau đó đem nó sang nơi khác khai phá. Cách đơn giản hơn là tổ chức để dữ liệu trở nên vô nghĩa đối với ai không biết cách sử dụng chúng. Các hệ cơ sở dữ liệu thường dùng cơ chế mã hóa dữ liệu. Mã hóa dữ liệu kết hợp với khóa bí mật chỉ người có trách nhiệm nắm giữ và dùng khóa này để giải mã nội dung dữ liệu.

Kiểm vết (Audit Trails)

Nếu người nào đó đã thâm nhập được vào hệ thống DBMS, điểm chặn cuối cùng là cố gắng theo dõi xem họ đã làm gì và thay đổi dữ liệu ra sao. Các hệ cơ sở dữ liệu DBMS thường sử dụng kỹ thuật kiểm vết (Audit Trails) ghi lại tất cả các hành động tác động lên dữ liệu (bạn xem lại chương **Quản lý giao dịch** để biết kỹ thuật quản lý các thao tác thay đổi dữ liệu). Cơ chế Audit Trails có thể được thiết lập để giảm tối thiểu không gian đĩa, xác định những điểm yếu của hệ thống và tìm ra những người dùng không mời mà đến hay những người dùng tò mò.

4. BẢO VỆ MỨC NGƯỜI DÙNG – HẠN CHẾ SQL

Hầu hết các tương tác với cơ sở dữ liệu ở cấp độ người dùng đều dựa vào ngôn ngữ SQL nên các hệ DBMS có những kỹ thuật hạn chế sử dụng lệnh SQL của người dùng.

- Mỗi người dùng được cấp quyền truy nhập nhất định ngay trên những đối tượng nhất định.
- Các người dùng khác nhau có thể có những quyền truy nhập khác nhau ngay trên cùng đối tượng.
- Các lệnh SQL sẽ không thực thi nếu đối tượng mà nó tác động không có quyền hợp lệ.

Lệnh GRANT

Lệnh GRANT (DBMS Oracle) được sử dụng để cấp quyền cho người dùng trên những đối tượng cơ sở dữ liệu.

Cú pháp:

GRANT privileges ON tablename


```
TO { grantee ... }  
[ WITH GRANT OPTION ]
```

Những đặc quyền có thể cấp:

- * **SELECT**- người dùng có quyền truy vấn dữ liệu.
- * **UPDATE** - người dùng có quyền sửa đổi dữ liệu.
- * **DELETE** - người dùng có thể loại bỏ dữ liệu.
- * **INSERT** - người dùng có thể chèn dữ liệu mới.
- * **REFERENCES** - người dùng có thể làm tham chiếu đến bảng.

Tùy chọn **WITH GRANT OPTION** cho phép người dùng xác định có thể cấp lại quyền của họ trên đối tượng mà họ sở hữu cho người dùng khác.

Ví dụ, để cấp quyền **SELECT** cho tất cả các người dùng, truy vấn bản userlist như sau:

```
GRANT SELECT ON userlist TO PUBLIC;
```

5. NHÀ QUẢN TRỊ CƠ SỞ DỮ LIỆU DBA (DATABASE ADMINISTRATOR)

Bạn thường nghe nói về các DBA (Database Administrator), họ là những người quản trị toàn bộ hệ thống cơ sở dữ liệu. Nhiệm vụ của một người DBA là trông nom cơ sở dữ liệu, cân bằng tất cả các nhu cầu của những người dùng (cho dù người dùng có cần đến hay không).

Ví dụ, không có người dùng nào muốn quan tâm nhiều đến vấn đề bảo mật, nếu một người nào đó xâm nhập (hack) vào cơ sở dữ liệu và xóa tất cả công việc của họ, DBA sẽ là người chịu trách nhiệm. Một thiết kế bảo mật hoàn hảo sẽ hoàn toàn trong suốt với người dùng bình thường, họ không biết được là mình đang được bảo vệ. Đến khi có vấn đề mất mát dữ liệu xảy ra người dùng hầu như mới biết là mình cần phải có người quản trị DBA. Bảo mật không phải là vấn đề duy nhất quan trọng đối với một DBA. Vai trò của người DBA còn liên quan đến:

- Điều chỉnh hiệu suất thực thi của hệ thống.
- Sao lưu và khôi phục dữ liệu định kỳ.
- Nâng cấp sản phẩm cơ sở dữ liệu mới, cài đặt công cụ bảo trì hệ thống.

- Xây dựng tài liệu hệ thống.
- Hỗ trợ người dùng cuối.
- Đào tạo.
- Dự đoán xu hướng phát triển các hệ cơ sở dữ liệu để tư vấn cho các ứng dụng chạy trên đó.

Một DBA hoàn hảo gần như không thể vì có rất nhiều vấn đề vượt ngoài tầm, nhưng DBA có thể làm tốt nhiều việc mà nếu không có anh ta thì công việc quản lý dữ liệu có thể sẽ tồi tệ hơn.

cuu duong than cong. com

cuu duong than cong. com

Chương 7:

PHÂN TÍCH THIẾT KẾ CƠ SỞ DỮ LIỆU

Các vấn đề chính sẽ được đề cập:

- ✓ Chu trình phân tích cơ sở dữ liệu.
- ✓ Mô hình cơ sở dữ liệu ba mức.
- ✓ Các khái niệm cơ sở:
 - Thực thể (Entities).
 - Attribute (Thuộc tính).
 - Keys (Khóa).
- ✓ Quan hệ (Relationships).
- ✓ Các cấp độ quan hệ.
- ✓ Thay thế các mối quan hệ tam phân.
- ✓ Quan hệ chính yếu và tùy chọn.
- ✓ Tập hợp thực thể.
- ✓ Xác nhận tính chính xác.
- ✓ Quan hệ thừa.
- ✓ Chia cắt quan hệ $n.m$.
- ✓ Xây dựng mô hình quan hệ thực thể ER.

Chương này chủ yếu tập trung đến quá trình tiếp nhận đặc tả cơ sở dữ liệu từ khách hàng và thực hiện các cài đặt cần thiết để xây dựng cấu trúc cơ sở dữ liệu tuân theo đặc tả đề ra.

Việc phân tích dữ liệu liên quan đến tính tự nhiên (NATURE) và việc sử dụng (USE) của dữ liệu. Phân tích sẽ nhận ra các phần tử dữ liệu nào cần dùng hỗ trợ cho quá trình xử lý trong hệ thống của công ty hay tổ chức, đặt những phần tử này vào các nhóm logic và định nghĩa mối quan hệ giữa các nhóm. Cách tiếp cận khác dùng cho phân tích dữ liệu là sử dụng mô hình FlowChart biểu diễn đường đi của dòng dữ liệu.

Hệ thống quản lý cơ sở dữ liệu (DBMS) ra đời đã đưa quá trình phân tích dữ liệu lên một mức cao hơn, trong đó các phần tử dữ liệu được định nghĩa bởi một mô hình logic hoặc 'lược đồ' (Schema). Kỹ thuật phân tích dữ liệu có thể được sử dụng như bước đầu tiên tiếp cận những phức tạp của thế

giới thực, đưa các thực thể của thế giới thực vào mô hình trên máy tính được truy nhập bởi nhiều người dùng. Dữ liệu có thể được thu nhập bởi những phương pháp truyền thống, chẳng hạn phỏng vấn những người trong tổ chức và nghiên cứu các tài liệu sẵn có. Các yếu tố dữ liệu sau đó có thể được biểu diễn thành những đối tượng thích hợp. Có rất nhiều công cụ và tài liệu hỗ trợ quá trình phân tích dữ liệu. Các công cụ này có thể ở dạng chương trình hoặc các phương pháp thể hiện lưu đồ quan hệ. Trong phân tích dữ liệu chúng ta phân tích và xây dựng một hệ thống thể hiện các dạng mô hình dữ liệu (mức khái niệm). Mô hình dữ liệu mức khái niệm xác định cấu trúc của dữ liệu và các quá trình sử dụng dữ liệu đó.

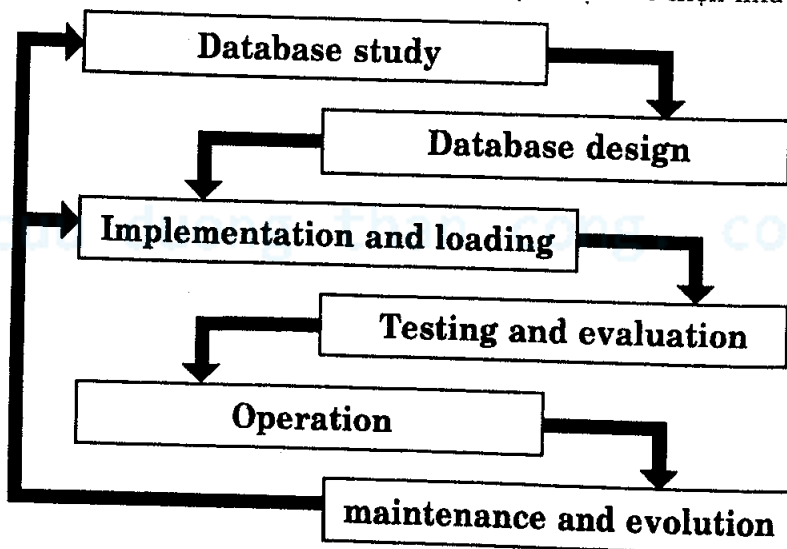
Phân tích dữ liệu = Thiết lập tính tự nhiên của dữ liệu.

Phân tích chức năng = Thiết lập quá trình sử dụng dữ liệu.

Tuy nhiên, do *Phân tích dữ liệu* và *Phân tích chức năng* thường kết hợp với nhau, chúng ta sẽ sử dụng thuật ngữ *Phân tích dữ liệu* để bao trùm cả hai. Việc xây dựng mô hình dữ liệu cho một tổ chức không đơn giản. Toàn bộ tổ chức rất lớn và có rất nhiều thứ cần được mô hình hóa và phải mất thời gian khá lâu.

1. CHU TRÌNH SỐNG CỦA PHÂN TÍCH CƠ SỞ DỮ LIỆU

Chu trình của quá trình phân tích dữ liệu được thể hiện như sau:



Khi một người thiết kế cơ sở dữ liệu tiếp cận vấn đề xây dựng một hệ thống cơ sở dữ liệu, các bước logic cần thực hiện chính là chu trình phân tích cơ sở dữ liệu:

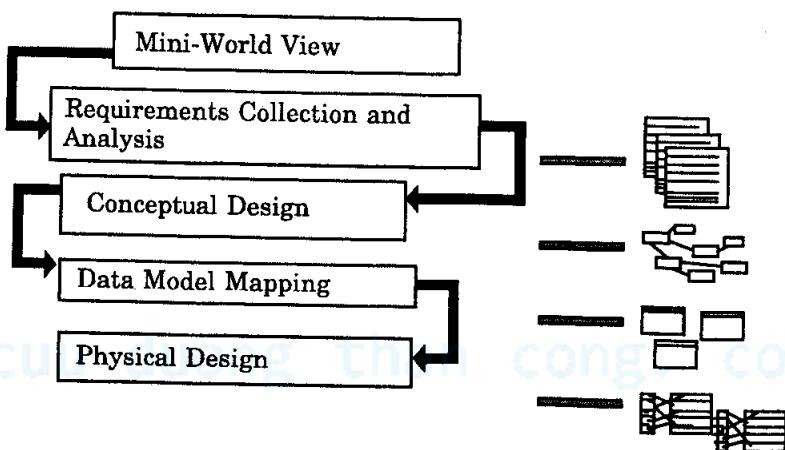
- **Database Study (nghiên cứu cơ sở dữ liệu)** - Ở bước này người thiết kế tạo ra đặc tả mô tả bằng từ ngữ thông thường đối với hệ thống Cơ sở dữ liệu sẽ được xây dựng. Công việc bao gồm:
 - Phân tích tình trạng tổ chức hay công ty – công ty có mở rộng hay không, dữ liệu yêu cầu tĩnh hay động, kiến thức và trình độ nhân viên... Những điều này tác động đến cách đặc tả cơ sở dữ liệu ban đầu.
 - Định nghĩa vấn đề và các ràng buộc như tình trạng hiện thời của công ty là gì? Công ty sẽ giải quyết các công việc hiện tại ra sao khi đưa cơ sở dữ liệu mới vào hoạt động. Giới hạn của hệ thống mới là gì?
 - Định nghĩa mục tiêu - hệ thống cơ sở dữ liệu mới sẽ phải làm gì và làm theo cách nào. Thông tin nào công ty muốn đặc biệt cất giữ và thông tin sẽ được tính toán như thế nào. Dữ liệu phát sinh trong tương lai ra sao?
 - Định nghĩa phạm vi và ranh giới – Thông tin gì được cất giữ trong hệ thống cơ sở dữ liệu mới này và sẽ cất giữ ở đâu. Cần giao tiếp với cơ sở dữ liệu khác hay không?
- **Database Design (Thiết kế cơ sở dữ liệu)** – Đây là bước thể hiện mức khái niệm, mức logic, mức vật lý đưa các đặc tả vào cài đặt cụ thể. Chúng ta sẽ xem bước này đầy đủ hơn trong phần sau.
- **Implementation and loading (Thi hành và chạy)** - Bước này yêu cầu bạn cài đặt hệ quản trị cơ sở dữ liệu cụ thể tạo các bảng dữ liệu mẫu, nhập các thông tin cần thiết (như danh sách nhân viên, danh sách kho, danh sách khách hàng hiện thời, ...).
- **Testing and evaluation (Kiểm tra và đánh giá)** - Sau khi đã cài đặt và thi hành cơ sở dữ liệu cần được kiểm tra xem có đáp ứng được các đặc tả đề ra trước đó hay chưa. Bước này có thể sử dụng các dữ liệu tuân theo nghiệp vụ thực tế để xác định tính đúng đắn của những quan hệ. Xác định độ khác biệt của cài đặt cơ sở dữ liệu hiện hành và đặc tả ban đầu. Đây cũng có thể là bước xem lại quá trình thiết kế để tối ưu hóa hệ thống tốt hơn. Cuối cùng, bước này cũng có thể dùng liên kết cơ sở dữ liệu với tầng ứng dụng để kiểm tra khả năng xử lý dữ liệu của ứng dụng tầng trên.

- **Operate (Hoạt động)** - Bước này thật sự diễn ra khi hệ thống đi vào hoạt động thực tế trong môi trường công ty.
- **Maintenance and evolution (Bảo trì và cải tiến)** - Người thiết kế hiếm khi có mọi thứ hoàn hảo lần đầu, và có thể công ty yêu cầu thêm một số thay đổi để khắc phục lỗi, thiếu sót hay mở rộng thêm tính năng mới. Thông thường, quá trình phát triển diễn ra khi cấu trúc cơ sở dữ liệu không còn thay đổi nhiều nữa. Trong những hệ thống cũ trước đây, một khi cấu trúc cơ sở dữ liệu đã duyệt và chấp nhận thì không cho phép thay đổi nữa.

2. MÔ HÌNH CƠ SỞ DỮ LIỆU BA MỨC

Mô hình ba mức diễn đạt các bước chuyển tải từ khâu viết đặc tả, mô tả các yêu cầu của thế giới thực cho đến bước xây dựng cấu trúc cơ sở dữ liệu hình thức và cài đặt cụ thể về mặt vật lý. Ba mức đó là:

- Thiết kế mức khái niệm (Conceptual Design).
- Ánh xạ mô hình dữ liệu (Data Model Mapping).
- Thiết kế vật lý (Physical Design).



Các đặc tả ban đầu thông thường ở dạng tài liệu viết, chứa những yêu cầu của khách hàng, các báo cáo nháp, những bản vẽ màn hình ... và thường là mô tả bởi khách hàng để chỉ định những yêu cầu hệ thống cuối cùng cần phải có. Thông thường dữ liệu như vậy được tập hợp từ nhiều nguồn bên trong công ty và sau đó được phân tích xem những yêu cầu đó có thật sự cần thiết, đúng đắn và hiệu quả hay không...

Sau khi các yêu cầu của cơ sở dữ liệu đã được đối chiếu, giai đoạn thiết kế ở mức khái niệm sẽ tiếp nhận các yêu cầu và sinh ra một mô hình dữ liệu cấp cao của cấu trúc cơ sở dữ liệu, đó là mô hình thực thể ER (có nhiều mô hình dữ liệu cấp cao nhưng trong giáo trình này chúng ta chỉ sử dụng mô hình ER). Mô hình ER hoàn toàn độc lập với các hệ quản trị DBMS mà bạn sẽ cài đặt cụ thể ở mức vật lý. Tiếp đến, giai đoạn ánh xạ mô hình dữ liệu (Data Model Mapping) sẽ sử dụng mô hình dữ liệu cấp cao chuyển đổi nó thành lược đồ khái niệm (Conceptual Schema), lược đồ này đặc thù với từng hệ DBMS. Cuối cùng, trong giai đoạn thiết kế vật lý, lược đồ khái niệm được chuyển đổi thành các cấu trúc bên trong cơ sở dữ liệu (table, view, index ...). Quá trình thiết kế vật lý đặc thù với từng sản phẩm DBMS khác nhau.

3. MÔ HÌNH HÓA QUAN HỆ THỰC THỂ (ER - ENTITY RELATIONSHIP)

- Là một công cụ thiết kế.
- Là đồ thị biểu diễn hệ thống cơ sở dữ liệu.
- Cung cấp một mô hình dữ liệu cấp cao ở mức khái niệm.
- Diễn đạt nhận thức của người dùng về dữ liệu.
- Độc lập với các hệ DBMS và phần cứng.
- Kết hợp thực thể, thuộc tính và quan hệ giữa các đối tượng.

3.1. Thực thể – (Entities)

- Thực thể là bất kỳ đối tượng nào trong hệ thống mà chúng ta muốn mô hình hóa và cất giữ thông tin.
- Các đối tượng riêng lẻ được gọi là từng thực thể.
- Nhóm các đối tượng cùng kiểu được gọi là kiểu thực thể hoặc tập hợp thực thể.
- Thực thể được biểu diễn bằng những hình chữ nhật (hoặc hình chữ nhật bầu).

Lecturer

Chen's notation

Lecturer

other notations

- Có hai kiểu thực thể: kiểu thực thể mạnh và kiểu thực thể yếu.

3.2. Thuộc tính (Attribute)

- Tất cả dữ liệu liên quan đến một thực thể được cất giữ trong thuộc tính của thực thể.
- Thuộc tính là một đặc tính (property) của thực thể.
- Mỗi thuộc tính có thể nắm giữ bất kỳ giá trị nào trong miền giá trị (domain) của nó.
- Mỗi thực thể nằm bên trong một kiểu thực thể:
 - Có thể có số thuộc tính bất kỳ.
 - Có thể có những giá trị thuộc tính khác nhau so với các thuộc tính trong những thực thể khác.
 - Có cùng số thuộc tính.

Thuộc tính có thể:

- Đơn giản hoặc phức tạp.
- Có một giá trị đơn hoặc nhiều giá trị.
- Biểu diễn được trên mô hình ER. Chúng xuất hiện bên trong những hình oval và gắn với thực thể của nó.

Chú ý rằng kiểu thực thể có thể có nhiều thuộc tính Attributes... Nếu tất cả đều biểu diễn thì sơ đồ sẽ trông rất rối rắm. Chúng ta chỉ biểu diễn các thuộc tính nếu nó thêm thông tin vào sơ đồ ER.



Trong sơ đồ trên, Lecture là thực thể còn Name là thuộc tính.

3.3. Keys (khóa)

- Khóa là một mục dữ liệu cho phép chúng ta xác định duy nhất những cá nhân riêng lẻ hoặc một kiểu thực thể.

- Khóa ứng viên là một thuộc tính hoặc tập hợp những thuộc tính xác định duy nhất những biến cố riêng lẻ hoặc một kiểu thực thể.
- Một kiểu thực thể có thể có một hoặc nhiều khóa ứng viên, khóa ứng viên được chọn sử dụng gọi là khóa chính (Primary key).
- Một khóa phức hợp là khóa ứng viên gồm có hai hoặc nhiều hơn hai thuộc tính.
- Trong mô hình ER tên của mỗi thuộc tính dùng làm khóa chính được gạch chân.

3.4. Mối quan hệ (relationships)

- Một kiểu quan hệ là các liên kết với nhau theo ý nghĩa nào đó giữa các kiểu thực thể.
- Kiểu quan hệ được biểu diễn trên sơ đồ ER bởi một hoặc nhiều đường thẳng nối kết.
- Có nhiều ký pháp để biểu diễn đồ hình ER. Trong ký pháp *Chen* nguyên gốc, quan hệ được đặt bên trong hình bình hành. Ví dụ, quan hệ giám đốc quản lý và nhân viên làm thuê được biểu diễn như sau:



Trong giáo trình, chúng ta sẽ sử dụng ký pháp thay thế khác, trong đó mỗi quan hệ được biểu diễn là một nhãn trên đường thẳng nối kết hai thực thể, về ý nghĩa thì như nhau.

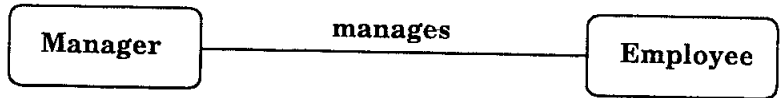


4. CÁC CẤP ĐỘ CỦA QUAN HỆ

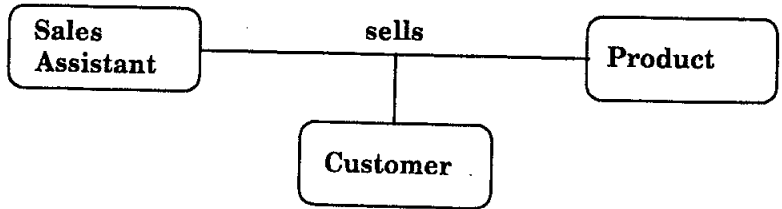
4.1. Biểu diễn cấp độ quan hệ

- Số lượng tham gia của những thực thể trong một mối quan hệ được gọi là cấp độ của quan hệ.

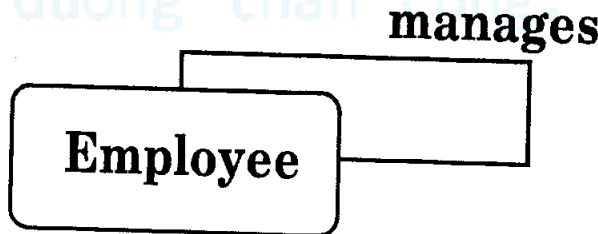
- Nếu chỉ có hai kiểu thực thể quan hệ với nhau ta gọi là kiểu quan hệ nhị phân (Binary Relationship).



- Nếu có ba kiểu thực thể quan hệ với nhau ta gọi là kiểu quan hệ tam phân (Ternary Relationship).

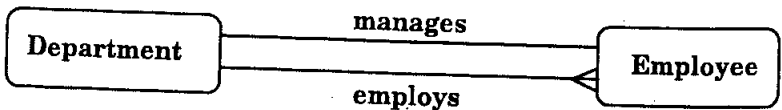


- Có thể có quan hệ n thay vì chỉ 2 hay 3 (ví dụ quan hệ tứ phân hay quan hệ đơn phân).
- Quan hệ đơn phân (unary) còn được gọi là quan hệ đệ quy vì thực thể quan hệ lại với chính nó.



Trong ví dụ ở trên chúng ta đang nói rằng nhân viên làm thuê được quản lý bởi những người nhân viên làm thuê khác. Nếu chúng ta muốn nhiều thông tin về người nào quản lý người nào thì cách biểu diễn trở thành nhị phân là giám đốc quản lý nhân viên.

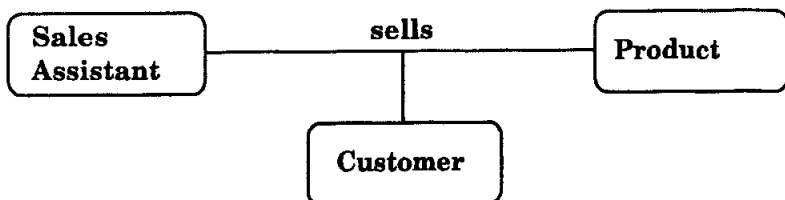
- Có thể có những thực thể liên kết với nhau thông qua hai hoặc nhiều mối quan hệ phân biệt.



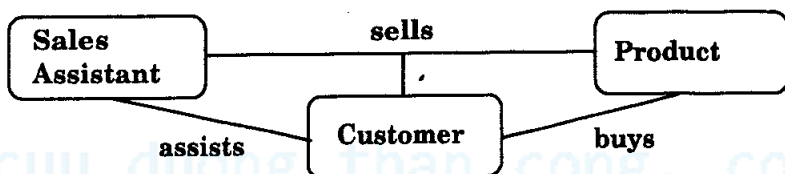
Ý nghĩa trong quan hệ trên là một phòng ban (department) có thể *quản lý* (manages) nhân viên và nhiều nhân viên thì *làm việc* (employs) trong cùng phòng ban.

4.2. Thay thế những mối quan hệ tam phân

Khi những mối quan hệ tam phân xuất hiện trong mô hình ER chúng cần phải được loại bỏ trước khi hoàn chỉnh mô hình. Đôi khi những mối quan hệ tam phân có thể được thay thế bởi nhiều quan hệ nhị phân liên kết những cặp của mối quan hệ tam phân với nhau. Ví dụ:



Có thể thay thế bằng:



Tuy nhiên, điều này có thể dẫn đến sự mất mát thông tin, như sẽ không còn biết nhân viên hỗ trợ bán hàng (Sales Assistant) nào bán cho khách hàng một sản phẩm đặc biệt nào đó.

- Mối quan hệ thường là những động từ, còn tên thực thể thường là các danh từ.
- Có thể biến mối quan hệ bán hàng (Sell) thành kiểu thực thể thông tin bán hàng như sau (trong mô hình một nhân viên hỗ trợ bán hàng có thể liên kết tới một khách hàng đặc biệt và cả hai cùng có thể bán một sản phẩm):



4.3. Quan hệ chính yếu

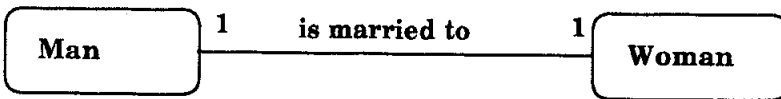
Mối quan hệ thường ít khi là 1:1. Ví dụ, một giám đốc thường quản lý nhiều nhân viên thay vì chỉ có một. Có bốn loại quan hệ chính yếu:

- Một-Một (1:1).

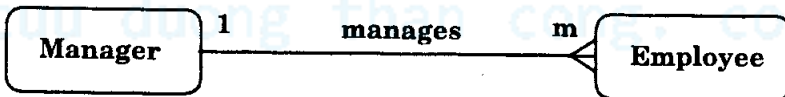
- Một-Nhiều (1:m).
- Nhiều-Một (m:1).
- Nhiều-Nhiều (m:n).

Trên một sơ đồ ER, nếu kết thúc của một mối quan hệ là đường thẳng tròn thì nó đại diện cho quan hệ 1, nếu đầu kết thúc dạng “chân chim” thì nó đại diện cho quan hệ nhiều (m hay n).

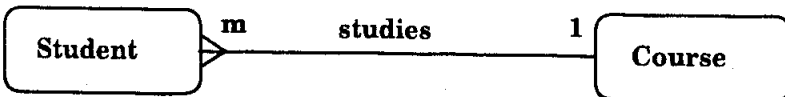
- Quan hệ Một-Một: Một người đàn ông chỉ có thể kết hôn (Is married to) với một phụ nữ và một phụ nữ chỉ có thể kết hôn với một người đàn ông, đó là mối quan hệ (1:1).



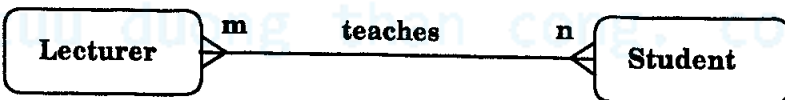
- Quan hệ Một-Nhiều: Một giám đốc quản lý nhiều nhân viên, nhưng mỗi nhân viên chỉ có một giám đốc, như vậy đây là quan hệ Một-Nhiều (1:m).



- Quan hệ Nhiều-Một: Nhiều sinh viên học cùng một môn học, như vậy đây là quan hệ Nhiều-Một (m:1).



- Quan hệ Nhiều-Nhiều. Một giảng viên dạy nhiều sinh viên và một sinh viên được dạy bởi nhiều giảng viên, như vậy đây là quan hệ Nhiều-Nhiều (m:n).

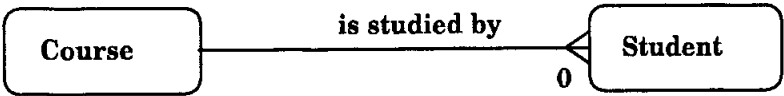


4.4. Quan hệ tùy chọn

Một mối quan hệ có thể bắt buộc hoặc tùy chọn. Nếu mối quan hệ là bắt buộc, thực thể của đầu quan hệ này phải quan hệ với thực thể đầu quan hệ còn lại. Với mối quan tùy chọn thì thực thể ở đầu quan hệ còn lại có thể thay đổi và khác nhau.

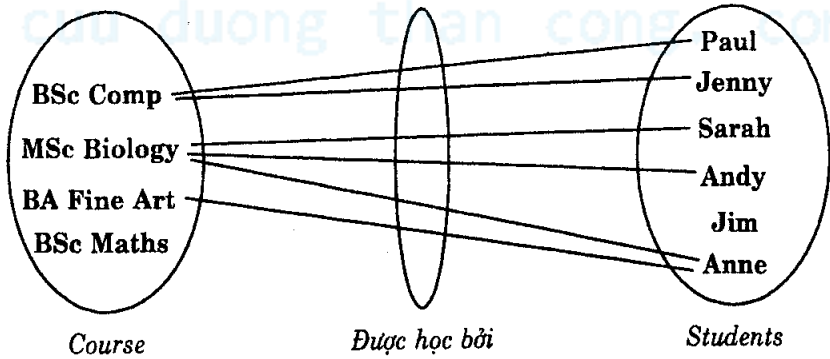
Ví dụ, một sinh viên phải tham gia một khóa học, đây là quan hệ bắt buộc. Tuy nhiên, khóa học có thể tồn tại trước khi có các sinh viên ghi danh. Do đó mỗi quan hệ “khóa học được học bởi sinh viên” là tùy chọn.

- Để biểu diễn khả năng tùy chọn của quan hệ, bạn đặt dấu tròn ở cuối quan hệ. Mỗi quan hệ tùy chọn “khóa học được học bởi sinh viên” là một tùy chọn đối với sinh viên nên dấu O được đặt ở cuối kết nối quan hệ sinh viên.

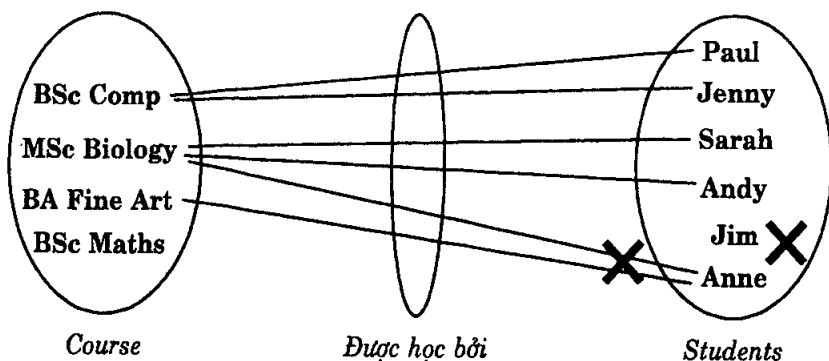


5. TẬP HỢP THỰC THỂ

Đôi khi cần phải thử nhiều ví dụ của các thực thể từ mô hình ER. Nó tương tự như việc bạn nhập thử dữ liệu để xem xét. Khi đó có thể dùng đồ thị tập hợp thực thể để biểu diễn.



- Sử dụng sơ đồ trên để cho thấy tất cả mối quan hệ có thể có của những tình huống.
- Quay về với các yêu cầu đặc tả và kiểm tra xem chúng được cho phép hay không.
- Nếu không, bạn đặt dấu chéo để ngăn quan hệ. Nó cho phép bạn tìm ra các quan hệ tùy chọn và quan hệ chính yếu.



Hình trên dẫn xuất ra những tham số có mối quan hệ.

Để kiểm tra xem chúng ta có được những tham số đúng hay không bạn hãy đặt hai câu hỏi:

1. Một khóa học được học bởi bao nhiêu sinh viên? Trả lời = “0 hay nhiều”. Câu trả lời này cho ta biết cấp độ quan hệ phía đầu sinh viên là 1 hay m. Câu trả lời “0 hay nhiều” có thể chia ra nếu là nhiều thì đầu quan hệ là chính yếu, nếu là 0 đầu quan hệ có thể là tùy chọn. Nếu câu trả lời là “một hoặc nhiều” thì mối quan hệ là bắt buộc.
2. Một sinh viên học bao nhiêu khóa học? Trả lời = “Một”. Nó cho phép chúng ta biết cấp độ quan hệ phía đầu khóa học là 1 hay m. Câu trả lời là “Một” có nghĩa rằng quan hệ là chính yếu và đây là quan hệ 1 mang tính bắt buộc. Nếu câu trả lời là “0 hoặc một” thì quan hệ là 1 nhưng tùy chọn.

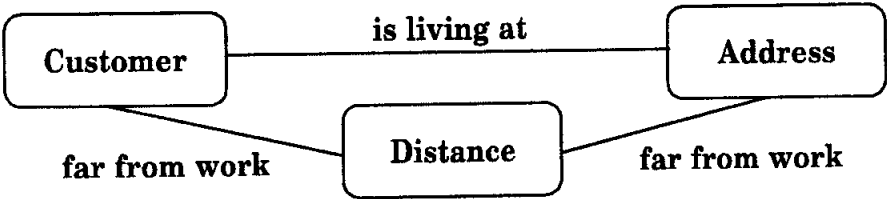
6. DƯ THỪA QUAN HỆ

Một số sơ đồ ER tạo nên quan hệ vòng (lặp lại) khiến nó trở nên dư thừa. Bạn có thể kiểm tra xem mô hình có rơi vào thiết kế này không để bề gây lặp hoặc loại bỏ quan hệ mà không làm mất thông tin.

Ví dụ, cho ba thực thể A, B, C, trong đó có ba quan hệ A-B, B-C và C-A, hãy kiểm tra xem có thể xác định A và C thông qua B hay không. Nếu có thể thì A-C là mối quan hệ thừa.

- Luôn luôn kiểm tra đường đi của các quan hệ để đơn giản hóa sơ đồ ER của bạn.
- Khiến cho sơ đồ dễ đọc với những thông tin còn lại.

Dưới đây là một ví dụ về quan hệ dư thừa trong mô hình ER: Các thực thể Customer (khách hàng), Address (địa chỉ khách hàng) và Distance (khoảng cách từ công ty đến địa chỉ khách hàng). *Is living at* là quan hệ sống tại địa chỉ, *far from work* là quan hệ cách xa.



7. CHIA CẮT QUAN HỆ N:M

Tuy quan hệ Nhiều-Nhiều (hay n:m) trong mô hình ER không phải là sai nhưng chúng có thể được thay thế bằng những thực thể trung gian để quy về quan hệ đơn giản hơn là 1:m. Điều này nên làm khi:

- Mỗi quan hệ n:m che giấu một thực thể.
- Sơ đồ ER mong muốn kết quả dễ hiểu hơn.

Dưới đây là ví dụ về cách chia cắt mối quan hệ n:m. Chúng ta xét trường hợp của một công ty cho thuê ô tô. Một khách hàng có thể thuê nhiều ô tô và một ô tô thì có thể được thuê bởi nhiều khách hàng. Và như vậy quan hệ khách hàng thuê ô tô là quan hệ n:m.



Mối quan hệ n:m có thể bị bẻ gãy thành hai quan hệ 1:m và 1:n để làm lộ ra thực thể trung gian bị ẩn (Hire) chứa thuộc tính “Ngày cho thuê”.



8. XÂY DỰNG MÔ HÌNH ER

Trước khi bắt đầu vẽ mô hình ER, bạn hãy đọc các yêu cầu đặc tả cẩn thận. Viết ra giấy tất cả những gì mình cần phải làm:

1. **Xác định các thực thể** – Liệt kê tất cả các kiểu thực thể có thể có. Đây là những đối tượng của thế giới thực mà bạn muốn mô hình hóa

vào hệ thống. Tốt nhất là ghi ra càng nhiều thực thể càng tốt trong giai đoạn này và từ từ lược bỏ chúng về sau nếu thấy không cần thiết.

2. Loại bỏ các thực thể trùng lặp - Bảo đảm rằng các thực thể khác biệt nhau về kiểu và không trùng tên. Đừng xem toàn bộ hệ thống là một kiểu thực thể. Ví dụ, nếu mô hình hóa một thư viện, các kiểu thực thể có thể là quyển sách, người mượn sách... Còn bản thân thư viện là hệ thống, và như vậy không thể xem thư viện là một kiểu thực thể được.

3. Liệt kê những thuộc tính của mỗi thực thể (tất cả các đặc tính mô tả thực thể liên quan đến ứng dụng).

- Bảo đảm rằng kiểu thực thể là thật sự cần thiết.
- Xem thử chúng có là thuộc tính của kiểu thực thể khác không? Nếu có đánh dấu chéo thì loại bỏ ra khỏi thực thể. Không được có những thuộc tính của một thực thể này đi đóng vai trò thuộc tính của thực thể khác!

4. Tạo khóa chính Primary Key.

- Tìm những thuộc tính nào dùng xác định tính duy nhất của kiểu thực thể.
- Một số thực thể yếu bạn có thể không tìm thấy, khi đó có thể xác định các số thứ tự tăng dần như Auto Increment được hỗ trợ bởi hầu hết các hệ DBMS làm khóa chính).

5. Định nghĩa quan hệ: Khảo sát mỗi kiểu thực thể để xem mối quan hệ của nó với những thực thể khác.

6. Mô tả quan hệ chính yếu và quan hệ tùy chọn bằng cách khảo sát ràng buộc giữa các thực thể tham gia trong quan hệ.

7. Loại bỏ những mối quan hệ dư thừa: Khảo sát lại toàn bộ mô hình ER và loại bỏ những mối quan hệ thừa nhằm giúp mô hình trở nên đơn giản và dễ hiểu hơn.

Mô hình hóa ER là một quá trình lặp đi lặp lại, vì vậy việc phải vẽ đi vẽ lại là thường xuyên cho đến khi bạn hài lòng với nó. Chú ý rằng, không có một câu trả lời đúng hoàn toàn cho một vấn đề cụ thể của bạn, kết hợp nhiều giải pháp với nhau vẫn là cách tốt nhất!

Chương 8:

XÂY DỰNG MÔ HÌNH QUAN HỆ THỰC THỂ

Các vấn đề chính sẽ được đề cập:

- ✓ *Mô hình quản lý công ty vận tải xe Bus.*
- ✓ *Các thực thể.*
- ✓ *Xây dựng quan hệ.*
- ✓ *Vẽ sơ đồ ER.*
- ✓ *Xác định thuộc tính.*
- ✓ *Các vấn đề với mô hình ER.*
- ✓ *Các bẫy (trap) kết nối thực thể.*
- ✓ *Mở rộng hình ER (EER).*
 - *Đặc thù hóa.*
 - *Tổng quát hóa.*

Trong chương này chúng ta sẽ ứng dụng xây dựng một mô hình ER cụ thể, đồng thời tìm hiểu các vấn đề phát sinh liên quan đến mô hình ER, những mở rộng của mô hình ER.

1. MÔ HÌNH QUẢN LÝ CÔNG TY VẬN TẢI XE BUS

Công ty vận tải xe Bus sở hữu nhiều xe buýt. Mỗi chiếc xe buýt được phân bổ đi theo một tuyến đường đặc biệt, mặc dù vậy một số tuyến đường có thể cùng lúc có nhiều xe buýt cùng vận hành.

Mỗi con đường (route) thường đi xuyên qua một số thành phố. Các tài xế được phân bổ lái xe trên các tuyến (stage) đường đi qua một hoặc nhiều (thậm chí tất cả) thành phố. Một số thành phố có gara đỗ xe, nơi xe buýt có thể đỗ lại nghỉ, đồng thời mỗi xe buýt được cấp cho một mã số đăng ký và có thể chở số lượng hành khách khác nhau do xe có nhiều kích cỡ. Mỗi con

đường được xác định bởi một mã số đường và tài xế được cấp mã số nhân viên theo tên, họ, địa chỉ và cả số điện thoại.

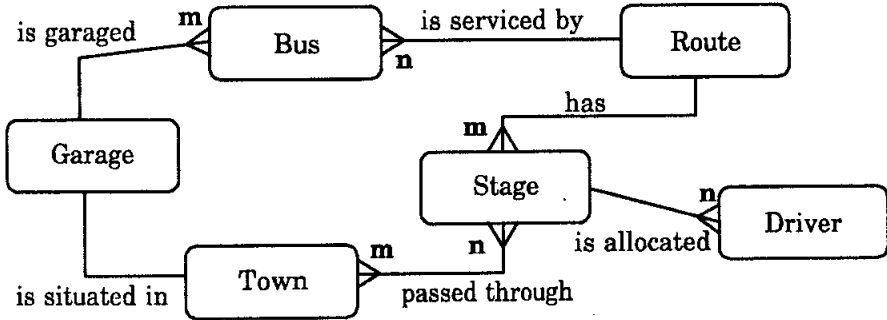
1.1. Xác định thực thể

- Bus (xe) – Công ty sở hữu các xe buýt và cần giữ thông tin về chúng.
- Route (con đường) – Các đối tượng xe Bus đi trên các con đường và cần phải mô tả thông tin về chúng.
- Town (thành phố) – Xe Bus di chuyển qua các thành phố và do đó cần biết thông tin về khoảng cách giữa các thành phố.
- Driver (tài xế) – Công ty thuê tài xế thông tin cá nhân của tài xế cần được lưu giữ lại.
- Stage (tuyến đường) – Mỗi tuyến đường có thể bao gồm nhiều con đường.
- Garage (gara) – Nhà đỗ xe, chúng ta cần biết vị trí chúng nằm ở đâu.

1.2. Xác định các mối quan hệ

- Một xe buýt được phân bổ chạy trên một con đường và một con đường do đó có thể có nhiều xe buýt cùng chạy. Ta có Bus-route (m:1) và tên quan hệ là *is serviced by*.
- Một con đường có thể nằm trên một hoặc nhiều tuyến đường khác nhau, rõ ràng quan hệ route-stage là (1:m) và tên quan hệ là *has*.
- Một hoặc nhiều tài xế khác nhau được yêu cầu lái trên cùng một tuyến đường nên quan hệ driver-stage (m:1) và tên quan hệ là *is allocated*.
- Các tuyến đường có thể đi xuyên qua một hoặc tất cả các thành phố nên quan hệ stage-town là (m:n) và tên quan hệ là *passed through*.
- Các con đường có thể đi xuyên qua một hoặc tất cả các thành phố nên quan hệ route-town là (m:n).
- Một số thành phố có gara nên garage-town là (1:1), tên quan hệ là *is situated in*.
- Một xe bus có thể đậu trong một hoặc nhiều gara khác nhau nên garage-bus là (m:1), tên quan hệ là *is garaged*.

1.3. Vẽ lược đồ ER



1.4. Xác định thuộc tính

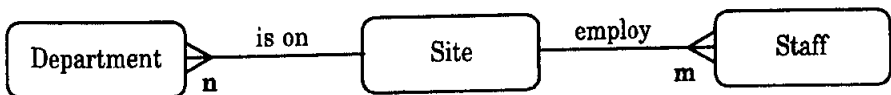
- Bus (reg-no, make, size, deck, no-pass)
- Route (route-no, avg-pass)
- Driver (emp-no, name, address, tel-no)
- Town (name)
- Stage (stage-no)
- Garage (name, address)

2. CÁC VẤN ĐỀ PHÁT SINH VỚI MÔ HÌNH ER

Có rất nhiều vấn đề có thể phát sinh khi thiết kế mô hình dữ liệu ở mức khái niệm. Chúng được gọi là các bẫy (trap) kết nối. Có hai kiểu bẫy kết nối chính:

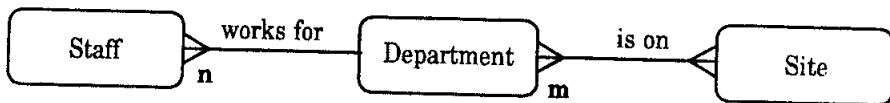
- Bẫy hình cung (Fan traps).
- Bẫy mất lối (Chasm traps).

Bẫy hình cung xuất hiện khi một mô hình biểu diễn mối quan hệ giữa những kiểu thực thể, nhưng đường đi giữa những thực thể nào đó không rõ ràng. Nó xuất hiện khi mối quan hệ 1:m xòe ra từ một thực thể đơn.

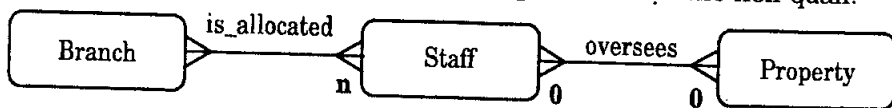


Một nơi làm việc (Site) có thể có nhiều phòng ban (Department) và thuê nhiều nhân viên (Staff). Tuy nhiên, nhân viên nào làm việc trong phòng

ban nào làm sao biết được? Bấy hình cung này được giải quyết bằng cách tổ chức lại mô hình ER cho đúng hơn.

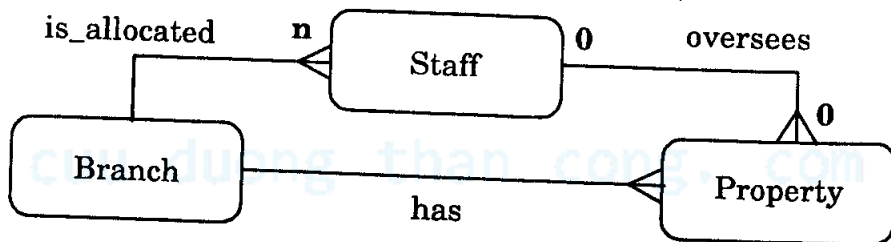


Bấy mất lối xuất hiện khi một mô hình cho thấy sự tồn tại của một mối quan hệ giữa các kiểu thực thể, nhưng đường nối chúng không tồn tại giữa những thực thể. Nó xuất hiện khi có một mối quan hệ với sự tham gia chỉ một phần nhưng lại hình thành nên đường nối giữa các thực thể liên quan.



Trong sơ đồ trên, một chi nhánh văn phòng (Branch) có nhiều nhân viên (Staff) hoạt động và mỗi nhân viên giám sát các tài sản (Property) cho thuê của văn phòng. Không phải tất cả nhân viên đều giám sát tài sản và không phải tất cả tài sản được quản lý hay giám sát bởi một nhân viên. Vậy thì câu hỏi đặt ra là không tìm được những tài sản nào hiện có ở một văn phòng chi nhánh? Sự tham gia chỉ một phần của quan hệ giám sát (oversee) giữa Staff và Property chỉ ra rằng có một số tài sản không thể thuộc về văn phòng chi nhánh (thực thể Branch) thông qua một thành viên nào đó của Staff.

- Chúng ta cần thêm mối quan hệ bị mất "has" (có) giữa thực thể Branch và Property.
- Bạn cần cẩn thận khi loại bỏ những mối quan hệ dư thừa.

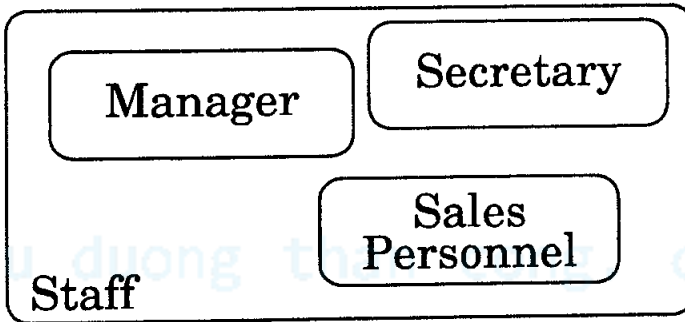


3. MỞ RỘNG MÔ HÌNH ER (EER HAY ENHANCED ER)

Những khái niệm cơ bản của mô hình ER đôi khi không đủ mạnh cho các ứng dụng phức tạp. Chúng yêu cầu cần thêm một số bổ sung các khái niệm về mô hình ngữ nghĩa.

Trước hết chúng ta cần xây dựng thêm một số thành phần thực thể ngữ nghĩa mới.

- **Superclass (lớp cha)** - Một kiểu thực thể bao gồm các thực thể lớp con subclass phân biệt biểu diễn trong mô hình dữ liệu.
- **Subclass (lớp con)** - Một kiểu thực thể có các vai trò (role) phân biệt và cũng là một thành viên của thực thể Superclass. Các lớp con không cần phải loại trừ lẫn nhau một thành viên của Staff có thể là thư ký (Secretary) hoặc cũng có thể là một giám đốc quản lý (Manager) hay nhân viên bán hàng (Sales Personnel).



Mục đích của việc giới thiệu lớp cha Superclass và lớp con Subclass là tránh mô tả những kiểu Staff có thể có những thuộc tính khác nhau bên trong cùng một thực thể. Điều này có thể lãng phí không gian và bạn có thể muốn một số thuộc tính là bắt buộc đối với một số kiểu nhân viên nhưng những nhân viên khác không cần phải có thuộc tính này.

3.1. Chuyên biệt hóa

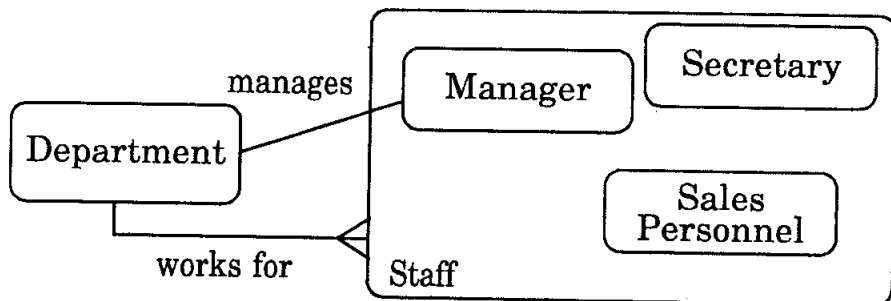
Đây là quá trình xác định tối đa các khác biệt giữa những thành viên của một thực thể thông qua những đặc trưng của chúng.

Staff(staff_no, name, address, dob)

Manager(bonus)

Secretary(wp_skills)

Sales_personnel(sales_area, car_allowance)



- Ở đây chúng ta biểu diễn mối quan hệ quản lý (**manages**) chỉ liên quan đến lớp con Subclass **Manager**, trong khi mối quan hệ **works_for** (làm việc cho) áp dụng với tất cả nhân viên.
- Bạn cũng có thể tạo những lớp con bên trong các lớp con khác.

3.2. Tổng quát hóa

Tổng quát hóa là quá trình tối giản sự khác nhau giữa các thực thể thông qua xác định những đặc tính chung nhất của chúng. Đây cũng là quá trình xác định những thuộc tính và những mối quan hệ chung. Ví dụ:

```

car(regno,colour,make,model,numSeats)
motorbike(regno,colour,make,model,hasWindshield)
  
```

Ở đây **car** và **motorbike** đều có điều chung là xe cộ. Chúng có thể tổng quát hóa thành:

```

vehicle(regno,colour,make,model,numSeats,hasWindshielf)
  
```

Những thuộc tính nào có trong **vehicle** nhưng không có trong **car** hay **motorbike** có thể đặt giá trị **NULL**.

cuu duong than cong. com

Chương 9:

ÁNH XẠ MÔ HÌNH THỰC THỂ ER

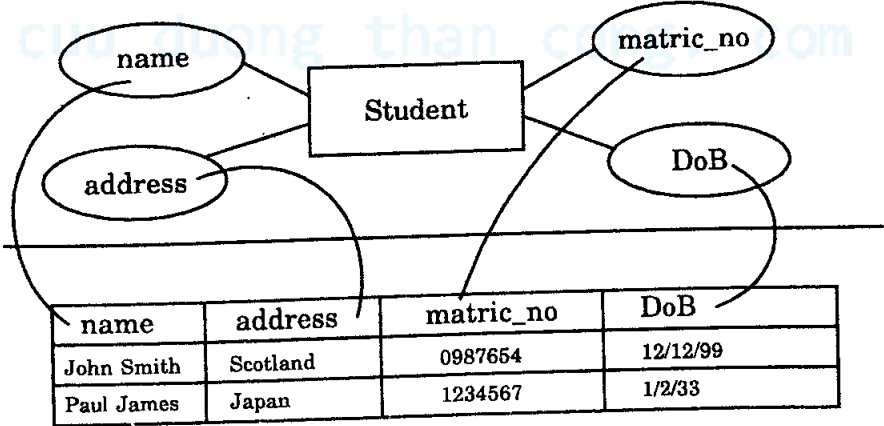
Các vấn đề chính sẽ được đề cập:

- ✓ Mô hình ER xem quan hệ relation là gì.
- ✓ Khóa ngoại.
- ✓ Chuẩn bị ánh xạ mô hình ER.
- ✓ Ánh xạ quan hệ liên kết 1:1.
- ✓ Quan hệ bắt buộc hai phía.
- ✓ Khi nào nên và không nên kết hợp.
- ✓ Quan hệ bắt buộc và tùy chọn.

1. QUAN HỆ LÀ GÌ?

Một quan hệ là một bảng nắm giữ dữ liệu mà chúng ta quan tâm đến. Bảng tương đương mảng hay ma trận hai chiều gồm các dòng và cột. Mỗi kiểu thực thể trong mô hình ER được ánh xạ vào trong một quan hệ (một bảng).

- Quan hệ trở thành bảng.
- Thuộc tính trở thành cột.
- Từng thực thể riêng lẻ trở thành dòng.



Quan hệ thay vì biểu diễn bằng hình vẽ có thể biểu diễn bằng mô tả như sau:

tablename(primary key, attribute 1, attribute 2, ... *foreign key*)

Trong ví dụ trên, nếu *matric_no* là khóa chính (primary key) và không có khóa ngoại thì bảng ở trên có thể được biểu diễn như sau:

student(matric no, name, address, date_of_birth)

2. KHÓA NGOẠI (FOREIGN KEY)

Khóa ngoại là một thuộc tính (hoặc nhóm các thuộc tính) đóng vai trò là khóa chính của một quan hệ khác.

- Nói dễ hiểu, mỗi khóa ngoại biểu diễn cho một mối quan hệ liên kết giữa hai kiểu thực thể.
- Chúng được thêm vào các quan hệ trong quá trình ánh xạ chuyển đổi mô hình ER sang cấu trúc cơ sở dữ liệu.
- Chúng cho phép các quan hệ liên kết với nhau.
- Một quan hệ có thể có nhiều khóa ngoại.
- Trong biểu diễn bằng mô tả, khóa ngoại thường được in nghiêng còn khóa chính biểu diễn với một đường gạch chân.

3. CHUẨN BỊ ÁNH XẠ MÔ HÌNH ER

Trước khi bắt đầu quá trình ánh xạ bạn cần chắc chắn rằng chúng ta đã đơn giản hóa mô hình ER ở mức đơn giản nhất. Đây là lúc kiểm tra lại mô hình lần cuối, vì thật sự cũng là cơ hội sau cùng để bạn chỉnh sửa thay đổi lên mô hình ER lần chót mà không gây những vấn đề phiền phức sau này, khi mô hình đã thực sự ánh xạ xong lên cấu trúc cơ sở dữ liệu.

3.1. Ánh xạ mối quan hệ liên kết 1:1

Trước khi giải quyết mối quan hệ liên kết 1:1, chúng ta cần biết quan hệ này là tùy chọn hay chính yếu. Có ba khả năng mà mỗi quan hệ liên kết có thể có:

1. Bắt buộc ở cả hai phía.
2. Bắt buộc ở một phía và tùy chọn phía còn lại khác.
3. Tùy chọn ở cả hai phía.

Trường hợp bắt buộc cả hai phía

- Nếu mỗi quan hệ liên kết là bắt buộc cả hai phía thực thể thì thường bạn có thể gộp hai kiểu thực thể lại làm một.
- Việc lựa chọn kiểu thực thể nào để gộp vào kiểu thực thể còn lại phụ thuộc vào kiểu thực thể nào được xem là quan trọng nhất (nhiều thuộc tính hơn, khóa tốt hơn, ngữ nghĩa dễ hiểu hơn).
- Kết quả của sự pha trộn này là tất cả thuộc tính của thực thể yếu sẽ trở thành những thuộc tính của thực thể mạnh, quan trọng hơn.
- Khóa của kiểu thực thể bị gộp vào trở thành một thuộc tính bình thường.
- Nếu có bất kỳ thuộc tính nào chung thì phần trùng lặp được loại bỏ.
- Khóa chính của thực thể mới kết hợp thường là khóa chính của thực thể quan trọng ban đầu.

Khi nào thì không nên kết hợp

Có một số lý do bạn không nên kết hợp mỗi quan hệ liên kết bắt buộc 1:1:

- Hai kiểu thực thể biểu diễn những thực thể khác nhau trong thế giới thực.
- Các thực thể tham gia trong những mối quan hệ liên kết rất khác nhau với những thực thể khác.

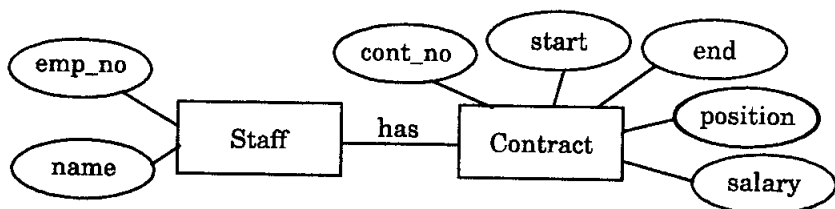
Nếu không kết hợp...

Nếu hai kiểu thực thể được giữ riêng biệt thì việc kết hợp chúng nên được biểu diễn qua một khóa ngoại.

- Khóa chính của một kiểu thực thể thành khóa ngoại bên trong kiểu thực thể khác.
- Việc chọn đặt khóa ngoại trong thực thể nào là không quan trọng nhưng đừng đặt khóa ngoại trong cả hai thực thể.

Ví dụ, ta có hai kiểu Staff (nhân viên) và Contract (hợp đồng):

- Mỗi thành viên của Staff phải có một hợp đồng Contract và mỗi hợp đồng phải có một thành viên Staff kết hợp với nó.
- Do vậy, đây là quan hệ liên kết bắt buộc ở cả hai phía kết nối thực thể Staff và Contract.



- Các kiểu thực thể này có thể trộn lẫn vào một.

Staff(emp_no, name, cont_no, start, end, position, salary)

- Hoặc giữ riêng và liên kết nhau bằng khóa ngoại.

Staff(emp_no, name, contract_no)

Contract(cont_no, start, end, position, salary)

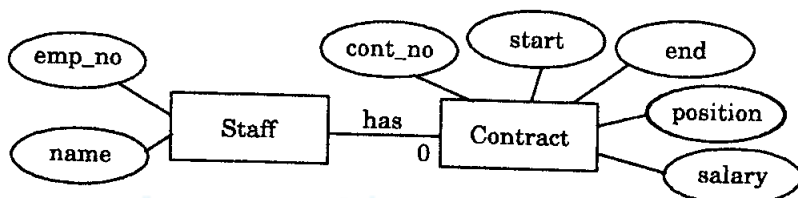
Hoặc:

Staff(emp_no, name)

Contract(cont_no, start, end, position, salary, emp_no)

Bắt buộc <-> Tùy chọn (dấu <-> viết tắt của quan hệ liên kết)

Kiểu liên kết thực thể có phía quan hệ liên kết tùy chọn có thể được gộp vào chung với thực thể phía liên kết bắt buộc như chúng ta đã xem qua trong ví dụ trước đây. Tuy nhiên, không nên gộp phía liên kết bắt buộc vào phía liên kết tùy chọn vì nó có thể tạo ra các mục nhập rỗng (NULL). Hãy xét mô hình ER sau:



Nếu chúng ta thêm vào đặc tả rằng mỗi thành viên Staff có thể có một hợp đồng như vậy sẽ khiến cho quan hệ là tùy chọn phía liên kết Contract.

- Ánh xạ khóa ngoại vào trong bảng Staff – khóa sẽ mang trị NULL đối với những thành viên không có hợp đồng.

Staff(emp_no, name, contract_no)

Contract(cont_no, start, end, position, salary)

- Ánh xạ khóa ngoại vào bảng Contract - emp_no là bắt buộc và luôn có giá trị, không bao giờ bị NULL.

Staff(emp_no, name)

Contract(cont_no, start, end, position, salary, emp_no)

Hãy xét ví dụ sau:

Staff "Gordon", emp_no 10, contract_no 11.

Staff "Andrew", empno 11, no contract.

Contract 11, from 1st Jan 2001 to 10th Jan 2001, lecturer, salary 2.00

Khóa ngoại đối với bảng Staff:

Bảng Contract:

Cont_no	Start	End	Position	Salary
11	1 st Jan 2001	10 th Jan 2001	Lecturer	£2.00

Staff Table:

Empno	Name	Contract No.
10	Gordon	11
11	Andrew	NULL

Tuy nhiên, khóa ngoại trong bảng Contract:

Contract Table:

Cont_no	Start	End	Position	Salary	Empno
11	1 st Jan 2001	10 th Jan 2001	Lecturer	£2.00	10

Staff Table:

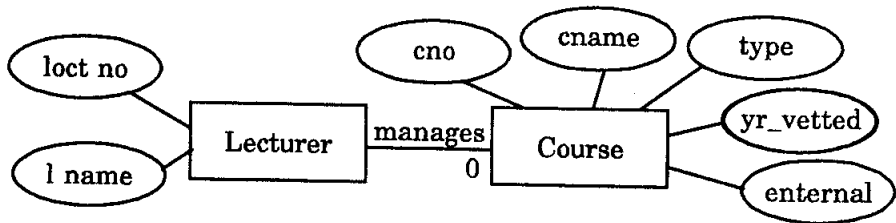
Empno	Name
10	Gordon
11	Andrew

Như bạn có thể thấy, cả hai cách đều lưu giữ cùng thông tin, nhưng cách thứ hai không có giá trị NULL.

Bắt buộc <-> Tùy chọn - Gộp vào hay không?

Lý do để không gộp hai thực thể dạng này vào cũng tương tự như trước đây nhưng bổ sung thêm:

- Rất ít những thực thể có phía quan hệ liên kết bắt buộc liên quan đến các quan hệ liên kết khác. Điều này có thể gây ra lãng phí không gian lưu trữ khi lưu giữ các giá trị NULL.



Trong sơ đồ trên, nếu chỉ có một số ít thực thể giảng viên Lecture quản lý những khóa học và thực thể khóa học Lecture được gộp vào thực thể Lecture thì sẽ làm xuất hiện nhiều mục mang giá trị NULL trong bảng.

Lecturer(lect_no, l_name, cno, c_name, type, yr_vetted, external)

- Tốt hơn là giữ cho chúng riêng biệt như sau:

Lecturer(lect_no, l_name)

Course(cno, c_name, type, yr_vetted, external, lect_no)

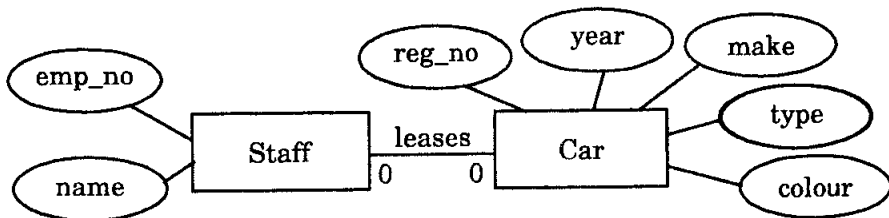
Tóm lại, với mối quan hệ liên kết 1:1 tùy chọn, hãy lấy khóa chính từ phía thực thể bắt buộc và thêm nó vào phía tùy chọn với vai trò khóa ngoại. Như vậy, nếu cho hai thực thể A và B, A <-> B là một quan hệ trong đó A là phía liên kết tùy chọn, kết quả thường là:

A (primary key, attribute,..., khóa ngoại (khóa chính B))

B (primary key, attribute,...)

Liên kết là tùy chọn ở cả hai phía...

Những ví dụ như vậy không thể trộn lẫn do bạn không thể lựa chọn khóa chính. Thay vào đó, khóa ngoại được sử dụng như ví dụ trước đây. Trong sơ đồ ER sau:



- Mỗi thành viên Staff có thể thuê một ô tô.
- Mỗi ô tô có thể được thuê bởi một số thành viên Staff.
- Nếu chúng kết hợp với nhau:

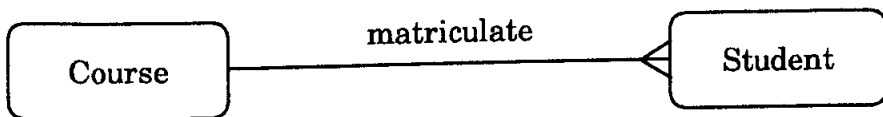
Staff_car(emp_no, name, reg_no, year, make, type, colour)

Vậy khóa chính sẽ là gì?

- Nếu *emp_no* được sử dụng thì tất cả các ô tô không được thuê sẽ không có khóa.
- Tương tự, nếu *reg_no* được sử dụng thì tất cả nhân viên không thuê ô tô sẽ không có khóa.
- Một khóa hỗn hợp cũng sẽ không làm việc.

3.2. Ánh xạ mối quan hệ liên kết 1:m

Để ánh xạ mối quan hệ liên kết 1:m, khóa chính của thực thể phía quan hệ liên kết 1 được dùng làm khóa ngoại cho thực thể phía m. Ví dụ, xét mối quan hệ trúng tuyển (matriculate) 1:m của hai thực thể *khóa học-sinh viên* “Course – Student” sau:



- Giả sử các thực thể có những thuộc tính sau:

Course(course_no, c_name)

Student(matric_no, st_name, dob)

- Sau khi ánh xạ, những quan hệ sau được sinh ra:

Course(course_no, c_name)

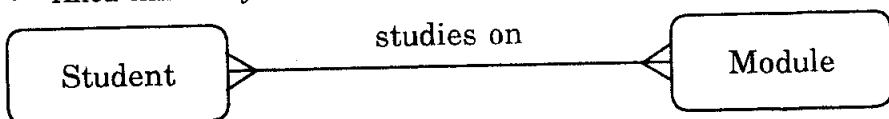
Student(matric_no, st_name, dob, course_no)

- Nếu một kiểu thực thể tham gia nhiều mối quan hệ liên kết 1:m, bạn áp dụng quy tắc này cho từng quan hệ liên kết 1:m và thêm khóa ngoại thích hợp cho từng thực thể.

3.3. Ánh xạ quan hệ n:m

Nếu bạn có mối quan hệ n:m trong mô hình ER thì những mối quan hệ này được ánh xạ theo cách sau:

- Một quan hệ mới được sinh ra chứa các khóa chính từ cả hai phía của mối quan hệ liên kết.
- Khóa chính này hình thành một khóa chính phức hợp.



- Như vậy, với mô hình ER trên:

Student(matric_no, st_name, dob)

Module(module_no, m_name, level, credits)

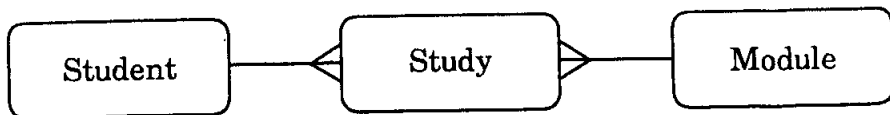
sẽ trở thành:

Student(matric_no, st_name, dob)

Module(module_no, m_name, level, credits)

Studies(matric_no, module_no)

Nó tương đương với sơ đồ ER mới như sau:



4. TÓM LƯỢC

- **Với mối quan hệ liên kết 1-1:** Tùy thuộc vào phía tùy chọn của mối quan hệ liên kết, các thực thể có thể kết hợp hoặc dùng khóa chính của một kiểu thực thể đặt làm khóa ngoại trong quan hệ khác.
- **Với mối quan hệ liên kết 1-m:** Khóa chính từ phía liên kết 1 được đặt làm khóa ngoại cho phía liên kết m.
- **Với mối quan hệ liên kết n-m:** Một quan hệ mới được tạo ra với các khóa chính lấy từ mỗi thực thể hình thành một khóa chính phức hợp khác.

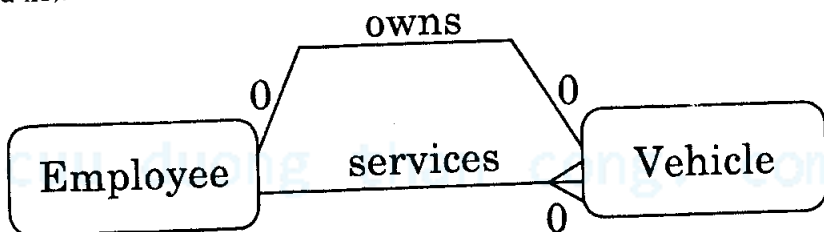
5. ÁNH XẠ ER MỞ RỘNG

Trong phần này chúng ta sẽ xem tiếp cách ánh xạ theo mô hình ER mở rộng, cùng những mối quan hệ liên kết đặc biệt khác như:

- Ánh xạ những mối quan hệ song song.
- Ánh xạ 1:m trong những mối quan hệ vòng.
- Ánh xạ lớp cha (superclass) và lớp con (subclass).

5.1. Ánh xạ những mối quan hệ song song

Các mối quan hệ song song xuất hiện khi có hai hoặc nhiều mối quan hệ liên kết giữa hai kiểu thực thể (ví dụ nhân viên có thể sở hữu và sử dụng nhiều xe).



- Để phân biệt giữa hai vai trò chúng ta có thể tạo các khóa ngoại mang tên khác nhau.
- Mỗi mối quan hệ liên kết được ánh xạ theo những quy tắc của chúng và chúng ta kết thúc với hai khóa ngoại thêm vào bảng Vehicle. Như vậy chúng ta thêm employee_no với tên owner_no để biểu diễn mối quan hệ liên kết sở hữu owns. Sau đó, thêm employee_no với tên serviced_by để biểu diễn mối quan hệ liên kết sử dụng serviced_by.

Trước khi ánh xạ:

Employee(employee_no,...)

Vehicle(registration_no,...)

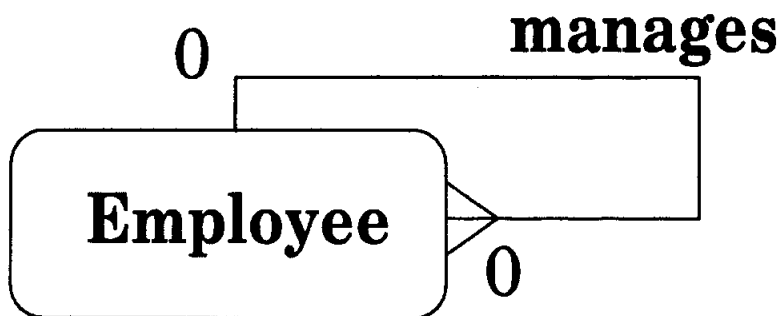
Sau khi ánh xạ:

Employee(employee_no,...)

Vehicle(registration_no,owner_no,serviced_by,...)

5.2. Ánh xạ 1:m trong mối quan hệ liên kết vòng

Xét sơ đồ ER sau:



Đây là sơ đồ thể hiện liên kết vòng:

- Thực thể Employee liên kết với chính nó thông qua quan hệ liên kết quản lý (manages). Về ngữ nghĩa bạn có thể là Nhân viên-quản lý-Nhân viên.
- Mỗi nhân viên có một khóa `employee_no` là khóa chính. Chúng ta biểu diễn mối quan hệ liên kết quản lý manages bằng cách thêm vào một thuộc tính `manager_no` đóng vai trò khóa ngoại. Đây thật ra là mã số `employee_no` của chính nhân viên làm giám đốc thực hiện công tác quản lý. Chúng ta đặt một tên khác để làm rõ ràng hơn nội dung mà nó thể hiện và hơn nữa để tuân theo quy tắc tên thuộc tính phải là duy nhất.

Sau ánh xạ:

Employee(employee_no, manager_no, name,...)

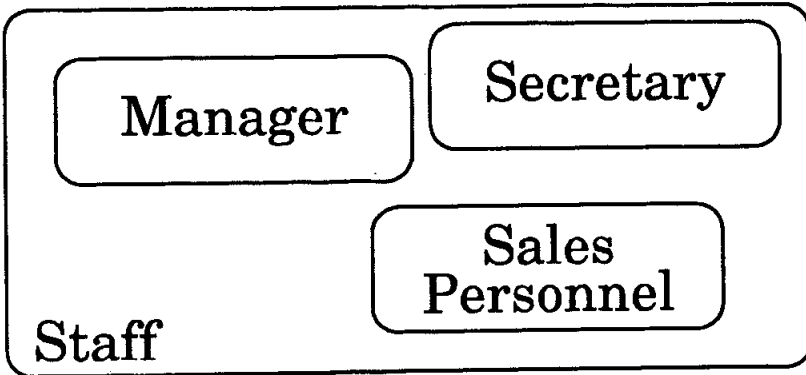
Nói chung, với những mối quan hệ liên kết vòng 1:m, khóa ngoại thường là khóa chính của cùng bảng đó nhưng được đặt cho một tên khác. Lưu ý, mỗi quan hệ liên kết là tùy chọn trong cả hai hướng vì không phải tất cả nhân viên đều có thể là giám đốc và giám đốc có chức vụ cao nhất thì không bị quản lý bởi ai cả.

5.3. Ánh xạ lớp cha Superclass và lớp con Subclass

Trong chương trước, ở mô hình EER mở rộng bạn đã học về hai kiểu thực thể khác là Superclass và Subclass. Có ba cách cài đặt ánh xạ cho Superclass và Subclass, tùy vào ứng dụng mà chọn ra cách thích hợp nhất.

1. Một quan hệ cho Superclass và một quan hệ cho mỗi lớp con Subclass.
2. Một quan hệ cho mỗi lớp con Subclass.
3. Một quan hệ cho lớp cha Superclass.

Một quan hệ cho Superclass và một quan hệ cho mỗi lớp con Subclass



Staff(staff_no,name,address,dob)

Manager(bonus)

Secretary(wp_skills)

Sales_personnel(sales_area, car_allowance)

Một quan hệ cho Superclass và một quan hệ cho mỗi lớp Subclass:

Staff(staff_no,name,address,dob)

Manager(staff_no,bonus)

Secretary(staff_no,wp_skills)

Sales_personnel(staff_no,sales_area, car_allowance)

Khóa chính của thực thể Superclass được ánh xạ vào trong mỗi lớp con và trở thành khóa chính cho lớp con.

Một quan hệ cho mỗi lớp con

Manager(staff_no,name,address,dob,bonus)

Secretary(staff_no,name,address,dob,wp_skills)

Sales_personnel(staff_no,name,address,dob,sales_area, car_allowance)

Tất cả các thuộc tính đều được ánh xạ vào trong mỗi lớp con. Nó tương đương với ba kiểu thực thể và không có lớp cha Superclass nào. Ánh xạ này hữu ích nếu không có thực thể nào gối lên nhau và không có các mối quan hệ liên kết giữa lớp cha Superclass và những kiểu thực thể khác.

Một quan hệ cho lớp cha Superclass

Staff(staff_no, name, address, dob, bonus, wp_skills, sales_area, car_allowance)

Ánh xạ này tương đương với một kiểu thực thể đơn và không có lớp con Subclass nào. Nhưng nó sẽ là không tốt nếu có những mối quan hệ liên kết giữa những lớp con và các thực thể khác. Ngoài ra, sẽ có nhiều mục giá trị NULL nếu các lớp con không chồng lên nhau nhiều.

cuu duong than cong. com

cuu duong than cong. com

Chương 10:

ĐẠI SỐ QUAN HỆ

Các vấn đề chính sẽ được đề cập:

- ✓ Thuật ngữ đại số quan hệ.
- ✓ Toán tử – Ghi.
- ✓ Toán tử – Trích rút dữ liệu.
- ✓ Quan hệ SELECT.
- ✓ Quan hệ PROJECT.
- ✓ SELECT và PROJECT.
- ✓ Phép toán tập hợp – ngữ nghĩa học.
- ✓ Phép toán tập hợp – các yêu cầu.
- ✓ UNION.
- ✓ INTERSECTION.
- ✓ DIFFERENCE.
- ✓ Phép tích - CARTESIAN PRODUCT.
- ✓ JOIN.
- ✓ OUTER JOIN.
- ✓ Biểu diễn toán học của Đại số quan hệ.

1. ĐẠI SỐ QUAN HỆ LÀ GÌ?

Đối với các nhà sản xuất phần mềm cơ sở dữ liệu, để cài đặt một hệ quản trị cơ sở dữ liệu DBMS phải tồn tại một tập hợp các quy tắc quy định những yêu cầu mà một hệ thống cơ sở dữ liệu hành xử. Ví dụ, đâu đó bên trong hệ DBMS phải có một tập hợp các phát biểu cho phép cập nhật hay chèn các dòng dữ liệu mới mà người sử dụng mong muốn thực hiện theo ý của họ. Một trong các cách là mô tả bằng “từ ngữ” cách thức mà một hệ RDBMS hoạt động, hay nói cách khác đó là mô tả các cú pháp lệnh và hướng dẫn ý nghĩa của nó để người dùng hiểu được. Tuy nhiên, diễn đạt bằng từ ngữ thông thường thường là không chính xác. Thay vào đó, các cơ sở dữ liệu quan hệ thường được diễn đạt và định nghĩa thông qua các phép toán của Đại số quan hệ.

Đại số quan hệ:

- Là mô tả hình thức các hoạt động của một cơ sở dữ liệu quan hệ.
- Là cơ sở toán học để cài đặt cú pháp SQL cũng như diễn đạt các thao tác xử lý quan hệ.

Tuy vậy, các toán tử trong đại số quan hệ không tất yếu giống như những toán tử của cú pháp SQL, thậm chí ngay cả khi chúng có cùng tên. Ví dụ, phát biểu SELECT tồn tại trong cú pháp SQL lẫn đại số quan hệ, nhưng cách sử dụng lại khác nhau. Các hệ DBMS phải tiếp nhận câu lệnh SQL mà người dùng nhập vào và dịch chúng thành những toán tử hay cú pháp của đại số quan hệ trước khi áp dụng chúng vào các hàm xử lý cơ sở dữ liệu.

2. THUẬT NGỮ

- Quan hệ (Relation) - Tập hợp của những bộ dữ liệu (tuples).
- Bộ dữ liệu (Tupes) - Tập hợp của những thuộc tính dùng mô tả một thực thể nào đó của thế giới thực.
- Thuộc tính (Attribute) - Trong thế giới thực nó đóng vai trò như một miền giá trị (domain) có tên xác định.
- Miền (Domain) - Một tập hợp của các giá trị.
- Tập hợp (Set) - Một định nghĩa toán học chứa tập các đối tượng không trùng nhau.

Bạn lưu ý một số khái niệm khác nhau giữa Đại số quan hệ và SQL mà ta đã học qua các chương trước. Nếu SQL xem tập hợp các dòng hình thành nên một bảng (table) thì Đại số quan hệ gọi Table là Relation, các dòng bên trong bảng (table) được Đại số quan hệ gọi là bộ dữ liệu (tuples). Cột của bảng tương đương với thuộc tính (Attribute) của Đại số quan hệ. Miền (Domain) tương đương kiểu dữ liệu.

3. TOÁN TỬ GHI (WRITE)

- INSERT (phép chèn) - Cung cấp một danh sách các thuộc tính cho một bộ dữ liệu mới trong một quan hệ. Toán tử này tương đương với lệnh INSERT của SQL.
- DELETE (phép xóa) - Cung cấp điều kiện trên các thuộc tính của một quan hệ để xác định bộ dữ liệu nào sẽ bị loại bỏ khỏi quan hệ. Toán tử này tương đương với lệnh SQL cùng tên.
- MODIFY (phép thay đổi) - Thay đổi các giá trị của một hoặc nhiều thuộc tính trong một hoặc nhiều bộ dữ liệu của một quan hệ, được xác định bởi một điều kiện thao tác trên thuộc tính của quan hệ. Toán tử này tương đương lệnh SQL UPDATE.

4. CÁC TOÁN TỬ TRÍCH RÚT DỮ LIỆU

Có hai nhóm toán tử:

- Nhóm lý thuyết toán tập hợp trên cơ sở quan hệ gồm: UNION (hợp), INTERSECTION (giao), DIFFERENCE (khác) và CARTESIAN PRODUCT (tích).
- Nhóm toán tử xử lý trên cơ sở dữ liệu: SELECT (không như lệnh SELECT của SQL), PROJECT (phép chiếu) và JOIN (phép kết).

4.1. Quan hệ lựa chọn SELECT

SELECT được sử dụng để thu về một tập con các bộ dữ liệu của một quan hệ thỏa mãn một điều kiện lựa chọn nào đó. Ví dụ, hãy tìm tất cả các nhân viên (employee) sinh sau ngày 1/1/1950:

SELECT dob '01/01/1950'(employee)

Dob (date of birth) là thuộc tính ngày sinh trong quan hệ employee.

4.2. Quan hệ chiếu PROJECT

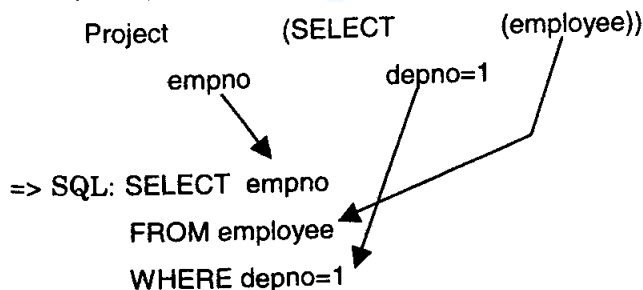
Toán tử PROJECT được sử dụng để lựa chọn một tập con những thuộc tính của một quan hệ bằng cách chỉ rõ tên của các thuộc tính cần lấy. Ví dụ, để lấy ra một danh sách tất cả các tên (username) và mã số nhân viên (empno) trong quan hệ employee:

PROJECT username, empno (employee)

4.3. SELECT và PROJECT

SELECT và PROJECT có thể kết hợp chung với nhau. Ví dụ, để lấy danh sách mã số các nhân viên thuộc về phòng ban 1 bạn sử dụng phép chọn SELECT và phép chiếu PROJECT như sau:

=> Đại số quan hệ



5. TOÁN TỬ TẬP HỢP - NGŨ NGHĨA

Xét hai quan hệ R và S.

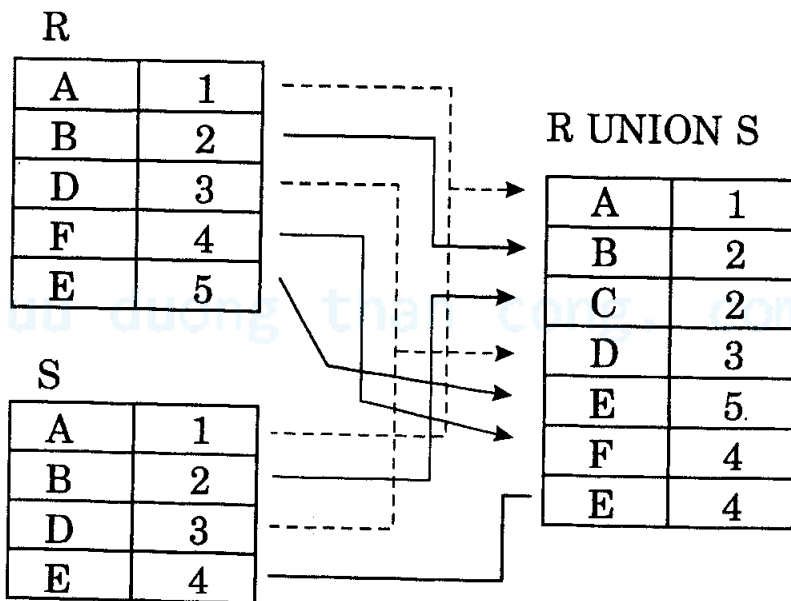
- Hợp (UNION) R và S là một quan hệ bao gồm tất cả các bộ dữ liệu tồn tại trong R hoặc trong S hoặc trong cả R lẫn S. Các bộ dữ liệu trùng nhau được loại bỏ.
- Giao (INTERSECTION) của R và S là một quan hệ bao gồm tất cả các bộ dữ liệu có cả trong R và S.
- Phân khác nhau (DIFFERENCE) của R và S là quan hệ chứa tất cả các bộ dữ liệu có trong R nhưng không có trong S.

6. CÁC TOÁN TỬ TẬP HỢP - YÊU CẦU

Để các toán tử tập hợp làm việc chính xác trên hai quan hệ R và S thì R và S phải hợp với nhau.

- Chúng phải có cùng số thuộc tính.
- Miền của mỗi thuộc tính theo thứ tự cột phải giống nhau trong cả R lẫn S.

Ví dụ về phép hợp UNION:



Ví dụ về phép giao INTERSECTION:

R

A	1
B	2
D	3
F	4
E	5

R INTERSECTION S

A	1
D	3

S

A	1
C	2
D	3
E	4

Ví dụ về phép khác DIFFERENCE:

R

A	1
B	2
D	3
F	4
E	5

R DIFFERENCE S

B	2
F	4
E	5

S

A	1
C	2
D	3
E	4

S DIFFERENCE R

C	2
E	4

7. PHÉP TÍCH DECAC

Phép tích Decac cũng là một toán tử làm việc trên hai tập hợp. Phép tích này đôi khi còn được gọi là CROSS PRODUCT hay CROSS JOIN. Nó kết hợp các bộ dữ liệu của một quan hệ với tất cả các bộ dữ liệu của quan hệ khác. Dưới đây là một ví dụ về phép tích Decac:

R	
A	1
B	2
D	3
F	4
E	5

S	
A	1
C	2
D	3
E	4

R CROSS			
A	1	A	1
A	1	C	2
A	1	D	3
A	1	E	4
B	2	A	1
B	2	C	2
B	2	D	3
B	2	E	4
D	3	A	1
D	3	C	2
D	3	D	3
D	3	E	4

F	4	A	1
F	4	C	2
F	4	D	3
F	4	E	4
E	5	A	1
E	5	C	2
E	5	D	3
E	5	E	4

8. TOÁN TỬ KẾT NỐI - JOIN

JOIN được sử dụng để kết hợp các bộ dữ liệu liên quan với nhau (từ hai quan hệ trở lên):

- Dạng đơn giản nhất của toán tử JOIN chính là tích Decac của hai quan hệ.
- Khi kết nối JOIN phức tạp hơn, các bộ dữ liệu được loại bỏ khỏi tích Decac để kết quả có ý nghĩa hơn.
- Phép nối JOIN cho phép bạn ước lượng một điều kiện kết nối giữa các thuộc tính của những quan hệ mà phép nối thực hiện.

Ký pháp sử dụng:

$R \text{ JOIN}_{\text{join condition}} S$

Ví dụ phép nối:

R	ColA	ColB
	A	1
	B	2
	D	3
	F	4
	E	5

S	SColA	SColB
	A	1
	C	2
	D	3
	E	4

$R \text{ JOIN}_{R \text{ ColA} = S.SColA} S$			
A	1	A	1
D	3	D	3
E	5	E	4

$R \text{ JOIN}_{R \text{ ColB} = S.SColB} S$			
A	1	A	1
B	2	D	2
D	3	D	3
F	4	E	4

9. OUTER JOIN

Chú ý rằng khá nhiều bộ dữ liệu bị mất đi khi áp dụng phép nối JOIN hai quan hệ. Trong một số trường hợp các bộ dữ liệu bị mất này có thể giữ thông tin quan trọng. Phép nối OUTER JOIN giữ lại thông tin mà phép JOIN đánh mất, đồng thời thay thế các vùng dữ liệu không tìm thấy bằng giá trị NULL (rỗng). Có ba dạng kết nối OUTER JOIN, tùy thuộc vào dữ liệu nào cần được giữ lại.

- LEFT OUTER JOIN - Giữ lại dữ liệu trong quan hệ bên trái.
- RIGHT OUTER JOIN - Giữ lại dữ liệu trong quan hệ bên phải.
- FULL OUTER JOIN - Giữ lại dữ liệu trong cả hai quan hệ bên trái và phải.

Ví dụ OUTER JOIN 1:

R

ColA	ColB
A	1
B	2
D	3
F	4
E	5

R LEFT OUTER JOIN R.ColA = S.SColA S

A	1	A	1
D	3	D	3
E	5	E	4
B	2	-	-
F	4	-	-

S

SColA	SColB
A	1
C	2
D	3
E	4

R RIGHT OUTER JOIN R.ColA = S.SColA S

A	1	A	1
D	3	D	3
E	5	E	4
-	-	C	2

Ví dụ OUTER JOIN 2:

R

ColA	ColB
A	1
B	2
D	3
F	4
E	5

R FULL OUTER JOIN R.ColA = S.SColA S

A	1	A	1
D	3	D	3
E	5	E	4
B	2	-	-
F	4	-	-
-	-	C	2

S

SColA	SColB
A	1
C	2
D	3
E	4

10. CÁC VÍ DỤ VỀ ĐẠI SỐ QUAN HỆ

Trong phần này chúng ta sẽ xem qua một số ví dụ về Đại số quan hệ và cú pháp SQL quen thuộc. Chúng ta sẽ xem qua:

- Các ký hiệu Đại số quan hệ.
- Cách dùng.
- Toán tử đổi tên.
- Toán tử dẫn xuất.
- Phép tương đương.
- So sánh Đại số quan hệ và SQL.

Hãy xem câu lệnh SQL sau dùng để tìm ra các nhân viên của phòng ban nào đã tham gia khóa học “Accounting”.

```
SELECT DISTINCT dname
```

```
FROM department, course, empcourse, employee
```

```
WHERE cname = 'Accounting'
```

```
AND course.courseno = empcourse.courseno
```

```
AND empcourse.empno = employee.empno
```

```
AND employee.deptno = department.deptno;
```

Tương đương cú pháp trong Đại số quan hệ biểu diễn như sau:

```
PROJECTdname (department JOINdeptno = deptno (
```

```
PROJECTdeptno (employee JOINempno = empno (
```

```
PROJECTempno (empcourse JOINcourseno = courseno (
```

```
PROJECTcourseno (SELECTcname = 'Accounting' course)
```

```
))
```

```
))
```

```
))
```

Các ký hiệu Đại số quan hệ

Từ ví dụ trên, nếu một câu truy vấn phức tạp bạn phải viết một chuỗi các tên toán tử khá nhiều như JOIN, PROJECT, SELECT. Các tên này có thể được thay thế bằng những ký hiệu toán học:



- SELECT $\Rightarrow \sigma$ (sigma)
- PROJECT $\Rightarrow \pi$ (pi)
- PRODUCT $\Rightarrow \times$ (times)
- JOIN $\Rightarrow \bowtie$ (bow-tie)
- UNION $\Rightarrow \cup$ (cup)
- INTERSECTION $\Rightarrow \cap$ (cap)
- DIFFERENCE $\Rightarrow -$ (minus)
- RENAME $\Rightarrow \rho$ (rho)

Cách dùng

Các ký hiệu toán học trên sử dụng hoàn toàn tương đương với cách diễn đạt của động từ Đại số quan hệ. Ví dụ, để tìm tất cả các nhân viên trong phòng ban 1:

Phát biểu thông thường:

$\text{SELECT}_{\text{depno} = 1}(\text{employee})$

Phát biểu theo ký hiệu:

$\sigma_{\text{depno} = 1}(\text{employee})$

Các điều kiện có thể được kết hợp cùng nhau bằng ký hiệu $\wedge$ (và) và $\vee$ (hoặc). Ví dụ, tìm tất cả các nhân viên trong phòng ban có mã số 1 và tên gọi "Smith":

Phát biểu thông thường:

$\text{SELECT}_{\text{depno} = 1 \wedge \text{surname} = \text{'Smith'}}(\text{employee})$

Phát biểu theo ký hiệu:

$\sigma_{\text{depno} = 1 \wedge \text{surname} = \text{'Smith'}}(\text{employee})$

Như bạn thấy, sử dụng ký pháp toán học sẽ đơn giản và gọn hơn. Ví dụ trước đây về câu lệnh SQL dùng để tìm ra các nhân viên của phòng ban nào đã tham gia khóa học "Accounting":

Phát biểu thông thường:

```
PROJECTdname (department JOINdepno = depno (
PROJECTdepno (employee JOINempno = empno (
PROJECTempno (empcourse JOINcourseno = courseno (
PROJECTcourseno (SELECTcname = 'Further Accounting' course)
))
))
))
```

Phát biểu theo ký hiệu:

```
 $\pi_{dname}$  (department  $\bowtie$  (
 $\pi_{depno}$  (employee  $\bowtie$  (
 $\pi_{empno}$  (empcourse  $\bowtie$  (
 $\pi_{courseno}$  ( $\sigma_{cname = 'Accounting'}$  course) )))))
```

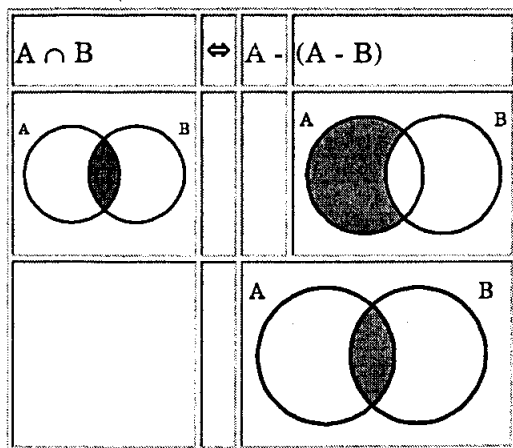
10.1. Toán tử đổi tên

Toán tử đổi tên trả lại một quan hệ hiện hữu bằng một tên mới. $\rho_A(B)$ là quan hệ B với tên của nó thay đổi thành A. Ví dụ, hãy tìm các nhân viên trong cùng phòng ban như nhân viên có mã số 3. Phát biểu theo ký hiệu của Đại số quan hệ như sau:

```
 $\rho_{emp2.surname, emp2.forenames}$  (
 $\rho_{employee.empno = 3 \wedge employee.depno = emp2.depno}$  (
employee  $\times$  ( $\rho_{emp2.employee}$ )
)
)
```

10.2. Toán tử dẫn xuất

- Các toán tử cơ sở: σ , π , $\times$, $\cup$, $-$, ρ
- Các toán tử dẫn xuất: $\bowtie$, $\cap$



10.3. Tương đương

$$A \bowtie_c B \Leftrightarrow \pi_{a_1, a_2, \dots, a_N}(\sigma_c(A \times B))$$

- Trong đó c là điều kiện kết nối JOIN (ví dụ $A.a_1 = B.a_1$).
- $a_1, a_2, \dots, a_N$ là tất cả các thuộc tính của A và B không trùng nhau.

c được gọi là điều kiện kết nối và thường là sự so sánh giữa khóa chính và khóa ngoại. Nếu có N bảng, thường ta sẽ có $N-1$ phép nối JOIN. Dưới đây là một số phép tương đương khác:

- $A \times B \Leftrightarrow B \times A$
- $A \cap B \Leftrightarrow B \cap A$
- $A \cup B \Leftrightarrow B \cup A$
- $(A - B) \neq (B - A)$
- $\sigma_{c_1}(\sigma_{c_2}(A)) \Leftrightarrow \sigma_{c_2}(\sigma_{c_1}(A)) \Leftrightarrow \sigma_{c_1 \wedge c_2}(A)$
- $\pi_{a_1}(A) \Leftrightarrow \pi_{a_1}(\pi_{a_1, \text{etc}}(A))$

Trong đó etc là các thuộc tính khác của A .

Các biểu thức Đại số quan hệ có thể được viết theo nhiều cách. Thứ tự bộ dữ liệu xuất hiện trong quan hệ không quan trọng.

11. SO SÁNH ĐẠI SỐ QUAN HỆ VÀ SQL

Đại số quan hệ:

- Kết quả của mỗi biểu thức là một quan hệ.
- Có một cơ sở toán chặt chẽ.
- Ngữ nghĩa đơn giản.
- Được sử dụng cho suy luận, tối ưu hóa biểu thức.

SQL:

- Có những tiện lợi về diễn đạt theo ngôn ngữ thường.
- Cung cấp các hàm tổng (SUM, COUNT...).
- Phức tạp về ngữ nghĩa.
- Là một ngôn ngữ lập trình thân thiện.

cuu duong than cong. com

cuu duong than cong. com

Chương 11:

TRANH CHẤP VÀ QUẢN LÝ GIAO DỊCH

Các vấn đề chính sẽ được đề cập:

- ✓ *Giao dịch (Transaction).*
- ✓ *Lịch biểu cho các giao dịch (Transaction Schedules).*
- ✓ *Kịch bản cập nhật bị mất mát.*
- ✓ *Phụ thuộc dữ liệu không được xác nhận.*
- ✓ *Không tương thích dữ liệu.*
- ✓ *Khả năng tuần tự.*

Mục đích trong môi trường quản trị DBMS có khả năng xử lý “đồng thời” (concurrent) là cho phép nhiều người dùng truy nhập cơ sở dữ liệu cùng thời điểm mà không gây ảnh hưởng đến nhau. Vấn đề thường xảy ra với môi trường đa người dùng của các hệ DBMS là khả năng xảy ra trường hợp hai người dùng cùng muốn thay đổi dữ liệu ngay tại một thời điểm. Nếu kiểu tác động này không được kiểm soát cẩn thận thì sự mâu thuẫn về mặt dữ liệu sẽ dễ dàng xảy ra. Để điều khiển quá trình truy nhập dữ liệu, trước hết chúng ta cần nắm vững khái niệm cho phép đóng gói các thao tác truy nhập cơ sở dữ liệu. Một gói nhiều thao tác truy nhập dữ liệu tuần tự tại một thời điểm được gom lại thành một hành động gọi là “Giao dịch” hay Transaction.

1. GIAO DỊCH (TRANSACTION)

Một giao dịch Transaction thường là một đơn vị công việc logic có tác động thay đổi hoặc khôi phục dữ liệu, nó cần thỏa mãn bốn điều kiện sau

- A - atomicity (đơn nhất bảo đảm cho yếu tố toàn vẹn).
- C - consistency (tương thích).
- I - isolation (cô lập).
- D - durability (ổn định).

Transaction được hỗ trợ sẵn trong ngôn ngữ SQL. Một số ứng dụng có khả năng yêu cầu thực hiện các giao dịch lồng nhau và kéo dài. Sau khi công việc được thực hiện trong một giao dịch, hai kết quả có thể xảy ra như sau:

- ◊ Commit (Xác nhận giao dịch) - Tất cả các thay đổi thực hiện trong thời gian giao dịch diễn ra được xác nhận là đúng đắn và ảnh hưởng ngay đến cơ sở dữ liệu.
- ◊ Abort (Hủy bỏ) - Tất cả các thay đổi trong thời gian giao dịch diễn ra yêu cầu hủy bỏ, không được tác động lên cơ sở dữ liệu. Kết quả của hành động Abort sẽ khiến cho cơ sở dữ liệu hoàn toàn không bị ảnh hưởng gì, nó gần như là chẳng có giao dịch gì đã xảy ra trước đó.

2. LỊCH BIỂU CHO CÁC GIAO DỊCH

Một lịch biểu giao dịch là một bảng cho biết thời gian một tập lệnh trong giao dịch thực thi như thế nào về thứ tự thời gian. Lịch biểu này hữu ích trong việc theo dõi những trường hợp các lệnh trong giao dịch gây ra vấn đề. Bên trong lược đồ lịch biểu rất nhiều thao tác được sử dụng:

- READ (a) – Đây là hành động đọc một thuộc tính hoặc mục dữ liệu tên “a”.
- WRITE (x,a) – Đây là hành động ghi hay cập nhật giá trị trên một thuộc tính hoặc mục dữ liệu “a”, trong đó “x” là giá trị thay thế “a”.
- tn (ví dụ t1,t2,t10) – Là giá trị thời gian mà các hành động xuất hiện. Đơn vị không quan trọng, nhưng tn luôn luôn xuất hiện trước tn +1.

Ví dụ xét giao dịch A, giao dịch này nạp một tài khoản X (ban đầu giá trị là 20) và cộng thêm vào tài khoản 10 đồng. Một lịch biểu cho giao dịch này có thể biểu diễn như sau:

Time	Transaction A
t1	TOTAL := READ (X)
t2	TOTAL := TOTAL + 10
t3	WRITE (TOTAL, X)

Bây giờ, giả sử vào cùng thời gian khi giao dịch A đang chạy, giao dịch B xuất hiện và chạy đồng thời. Giao dịch B tăng giá trị tất cả các tài khoản lên 10%. Vậy X lúc này là 32 hay 33?

T	Transaction A	Value TOTAL	Transaction B	Value BALANCE
T1			BALANCE := READ (X)	20
T2	TOTAL := READ (X)	20		
T3	TOTAL := TOTAL + 10	30		
T4	WRITE (TOTAL, X)	30		
T5			BALANCE := BALANCE * 110%	22
T6			WRITE (BALANCE, X)	22

Câu trả lời là ... X bằng 22! Tùy thuộc vào sự đan xen của hai giao dịch mà X có thể mang các giá trị 32, 33 hoặc là 30. Chúng ta sẽ phân tích những trường hợp cập nhật sai ngay sau đây.

3. CẬP NHẬT MẤT MẮT DỮ LIỆU

Time	Transaction A	Transaction B
t1	X=READ(R)	
t2		Y=READ(R)
t3	WRITE(X,R)	
t4		WRITE(Y,R)

Thao tác cập nhật của giao dịch A bị mất vào thời điểm t4, vì giao dịch B ghi đè lên nó. B mất đi giá trị cập nhật của A tại thời điểm t3 vì nó đã cập nhật giá trị R ở thời điểm t2.

4. PHỤ THUỘC KHÔNG XÁC NHẬN

Time	Transaction A	Transaction B
t1		WRITE(X,R)
t2	Y=READ(R)	
t3		ABORT

Giao dịch A được cho phép đọc (hoặc ghi) giá trị R đã được cập nhật bởi giao dịch khác nhưng chưa được xác nhận (commit) (trong trường hợp này bị sai và đã hủy bỏ).

5. SỰ MÂU THUÃN KHÔNG TƯƠNG THÍCH (INCONSISTENCY)

Hãy xem các giá trị đan xen phát sinh giữa hai giao dịch trong bảng lịch biểu dưới đây để thấy sự không tương thích về mặt dữ liệu.

Time	X	Y	Z	Transaction A		Transaction B
				Hành động	SUM	
t1	40	50	30	SUM:=READ(X)	40	
t2	40	50	30	SUM+=READ(Y)	90	
t3	40	50	30			ACC1=READ(Z)
t4	40	50	20			WRITE(ACC1-10,Z)
t5	40	50	20			ACC2=READ(X)
t6	50	50	20			WRITE(ACC2+10,X)
t7	50	50	20			COMMIT

t7	50	50	20	SUM+=READ(Z)	110	
				SUM lẽ ra là 120...		

6. TÍNH TUẦN TỰ (SERIALISABILITY)

- Một “lịch biểu” thật sự là quá trình thực thi tuần tự của hai hoặc nhiều giao dịch đồng thời.
- Một lịch biểu của hai giao dịch T1 và T2 được xem là tuần tự hóa nếu và chỉ nếu khi thực thi có cùng hiệu ứng, nghĩa là thực thi T1;T2 có kết quả cũng như thực thi T2;T1.

7. ĐỒ THỊ MỨC ƯU TIÊN

Để biết một lịch biểu của giao dịch nào đó có tuần tự hóa hay không, chúng ta có thể vẽ một đồ thị mức ưu tiên. Đây là một đồ thị gồm các nút và đỉnh, trong đó nút là tên giao dịch và đỉnh là những thuộc tính bị xung đột. Lịch biểu được xem là tuần tự nếu và chỉ nếu không có chu trình vòng trong sơ đồ kết quả. Các phương pháp thực hiện vẽ đồ thị mức ưu tiên như sau:

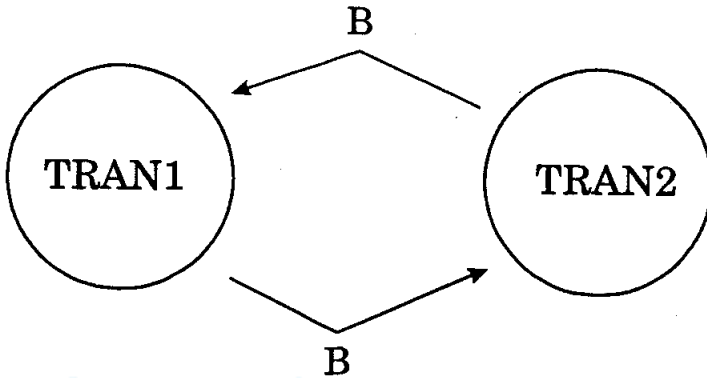
- ◇ Vẽ một nút cho mỗi giao dịch trong lịch biểu.
- ◇ Khi giao dịch A ghi một giá trị vào thuộc tính mà giao dịch B đang đọc, vẽ một đường thẳng từ B đến A.
- ◇ Khi giao dịch A ghi một giá trị vào thuộc tính mà giao dịch B cũng đang ghi vào thuộc tính đó, bạn vẽ một đường thẳng nối từ B tới A.
- ◇ Khi giao dịch A đọc từ một thuộc tính mà giao dịch B đang ghi, bạn vẽ một đường từ B tới A.

Ví dụ 1, hãy xem xét lịch biểu sau:

Time	TRAN1	TRAN2
t1	READ(A)	
t2	READ(B)	
t3		READ(A)

t4		READ(B)
t5	WRITE(x,B)	
t6		WRITE(y,B)

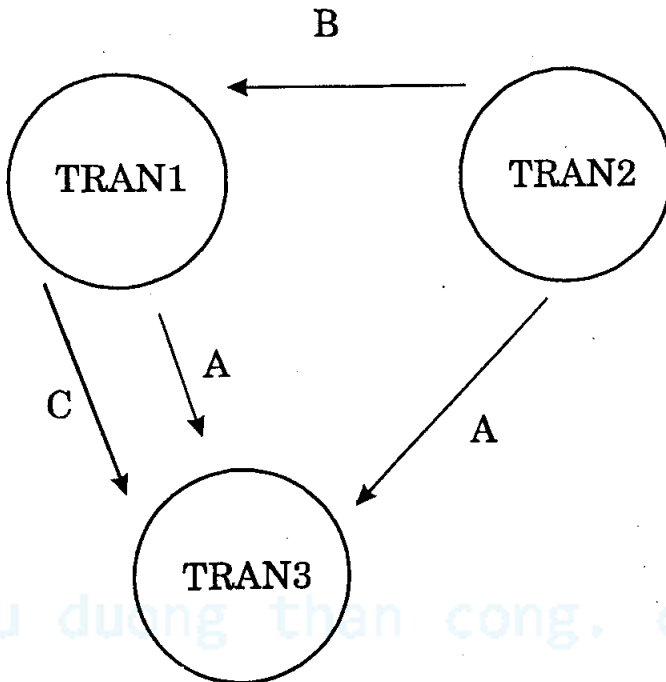
Đồ thị mức ưu tiên được vẽ như sau:



Ví dụ 2, xem xét lịch biểu sau:

Time	TRAN1	TRAN2	TRAN3
t1	READ(A)		
t2	READ(B)		
t3		READ(A)	
t4		READ(B)	
t5			WRITE(x,A)
t6	WRITE(v,C)		
t7	WRITE(w,B)		
t8			WRITE(z,C)

Đồ thị mức ưu tiên được vẽ như sau:



8. ĐỒNG BỘ

- ◊ Khóa.
- ◊ Khóa chết (deadlock).
- ◊ Giải quyết Deadlock.
- ◊ Các phương pháp tương thích dữ liệu.
- ◊ Quy tắc Timestamping.

8.1. Khóa (Lock)

Giải pháp nào để khiến các giao dịch là tuần tự? Rất nhiều hệ thống sử dụng cơ chế khóa (lock) để điều khiển việc xử lý đồng thời (bạn lưu ý khóa ở đây có nghĩa là “ngăn” lại). Khi một giao dịch cần sự bảo đảm đối tượng mà nó đang tác động sẽ không thay đổi theo cách bất ngờ nào đó trong một thời gian nhất định, nó yêu cầu khóa đối tượng đó lại.

- Một giao dịch giữ một khóa chỉ đọc được trên đối tượng đó nhưng không được thay đổi nó.

- Nhiều giao dịch có thể giữ một khóa cho phép đọc trên cùng đối tượng.
- Thông thường, chỉ có một giao dịch có thể giữ khóa cho phép ghi trên đối tượng tại một thời điểm.
- Trên lịch biểu giao dịch, chúng ta sử dụng “S” để chỉ báo một khóa dùng chung và “X” để chỉ báo một khóa nắm quyền ghi độc quyền.

8.2. Khóa - phụ thuộc dữ liệu không xác nhận

Khóa sẽ giải quyết vấn đề phụ thuộc dữ liệu không được xác nhận đã nêu trước đây. Hãy xem lịch biểu được sắp xếp lại như sau:

Time	Transaction A	Transaction B	Lock R
t1		WRITE(p,R)	- = X
t2	READ R (Chờ)		X
t3	...chờ...	ABORT	X = -
t4	READ R (Tiếp tục)		- = S

8.3. Khóa chết (Deadlock)

Tình trạng Deadlock xuất hiện khi các khóa đều đang được sử dụng, và gây ra các giao dịch chờ lẫn nhau. Tình trạng chờ diễn ra mãi mãi vì giao dịch này chờ giao dịch kia kết thúc mới tiếp tục.

time	Transaction A	Transaction B	Lock State	
			X	Y
t1	WRITE(p,X)		- = X	-
t2		WRITE(q,Y)	X	- = X

t3	READ(Y) (WAIT)		X	X
t3	...Chờ...	READ(X) (Chờ)	X	X
t3	...Chờ...	...Chờ...	X	X

8.4. Giải quyết Deadlock

Nếu một tập hợp các giao dịch được xem rơi vào tình trạng Deadlock bạn nên:

- Chọn ra giao dịch “nạn nhân” (ví dụ giao dịch vừa xảy ra gần nhất).
- Hủy bỏ giao dịch (Rollback) nạn nhân này và khởi động lại. Hành động Rollback tương đương Abort nó kết thúc giao dịch, hủy bỏ tất cả thay đổi và tháo khóa trên các giao dịch đang chờ.
- Một thông báo được chuyển cho giao dịch nạn nhân và tùy vào hệ thống, giao dịch có thể được khởi động lần nữa một cách tự động hoặc không.

8.5. Các phương pháp tương thích cơ sở dữ liệu khác

Một số hệ RDBMS khác sử dụng cơ chế đánh dấu thời gian Timestamping. Cơ chế Timestamping không sử dụng các khóa để ngăn ngừa những giao dịch thay đổi dữ liệu, tất cả các thao tác cập nhật vật lý đều được xác định theo thời gian. Dưới đây là những quy tắc Timestamping.

Quy tắc sau sẽ được kiểm tra khi giao dịch T thử thay đổi một mục dữ liệu:

- ◊ Nếu quy tắc yêu cầu ABORT thì giao dịch T được quay ngược lại (rollback) và hủy bỏ (có thể khởi động lại).
- ◊ Nếu T thử đọc một mục dữ liệu đã được cập nhật bởi một giao dịch trước đó thì hủy T.
- ◊ Nếu T thử ghi một mục dữ liệu đã được đọc hoặc ghi bởi một giao dịch trước nó thì hủy T.

- ◊ Nếu giao dịch T hủy bỏ, tất cả các giao dịch khác đã đọc mục dữ liệu ghi bởi T cũng phải hủy bỏ. Ngoài ra, việc hủy bỏ giao dịch này có thể gây ra hủy bỏ những giao dịch liên quan khác. Cơ chế này gọi là hủy bỏ dây chuyền ("cascading rollback").

9. KHÔI PHỤC (RECOVERY)

- ◊ Khôi phục từ file dump.
- ◊ Khôi phục từ thông tin Log của giao dịch.
- ◊ Trì hoãn cập nhật.
- ◊ Cập nhật tức thời.
- ◊ Rollback.

Một cơ sở dữ liệu có thể rơi vào trạng thái dữ liệu không đúng đắn khi:

- Tình trạng khóa deadlock diễn ra.
- Một giao dịch hủy bỏ sau khi cập nhật cơ sở dữ liệu.
- Lỗi phần mềm hoặc phần cứng.
- Cập nhật dữ liệu sai được thực hiện trên cơ sở dữ liệu.

Nếu cơ sở dữ liệu rơi vào trạng thái dữ liệu không đúng đắn, bạn phục hồi hay khôi phục nó về trạng thái đúng đắn sau cùng. Việc khôi phục cơ sở dữ liệu cơ bản là cần đến một bản sao lưu (backup) trước đó.

9.1. Khôi phục từ các file thô (dump)

Kỹ thuật sao lưu đơn giản nhất đó là xuất tất cả dữ liệu ra lưu dạng thô (dump)

- Toàn bộ nội dung của cơ sở dữ liệu được sao lưu vào nơi cất giữ an toàn khác.
- Việc sao lưu chỉ thực hiện khi trạng thái của cơ sở dữ liệu đang đúng đắn - không có giao dịch sửa đổi cơ sở dữ liệu nào đang chạy.
- Việc xuất hay dump dữ liệu thô có thể mất một khoảng thời gian dài thực hiện.

- Bạn cần cất giữ dữ liệu của mình đến hai lần (một bản thật và một bản sao lưu).

Do chi phí lưu trữ và thời gian khi thực hiện kỹ thuật dump cao, nó không được chọn sử dụng thường xuyên. Một kỹ thuật cải tiến khác có thể sử dụng đó là Snapshot hay lưu nhanh lại những vùng hay thay đổi nhất của các giao dịch trước đó.

9.2. Khôi phục dựa vào các file log của giao dịch

Một kỹ thuật thường được sử dụng để thực hiện khôi phục lại dữ liệu là dựa vào quá khứ giao dịch đã diễn ra trước đó.

- Ghi lại thông tin về quá trình xảy ra giao dịch trong một file log tính từ trạng thái đúng đắn của dữ liệu cuối cùng.
- Cơ sở dữ liệu do vậy nhận biết trạng thái đúng đắn của nó trước và sau mỗi giao dịch. Do vậy, mỗi khi cơ sở dữ liệu được trả về trạng thái đúng đắn sau khi giao dịch chấm dứt, file log sẽ loại bỏ những thông tin giao dịch trước đó. Khi cơ sở dữ liệu được trả lại trạng thái đúng đắn, quá trình này thường được gọi là điểm dừng “checkpoint”.

9.3. Trì hoãn cập nhật

Trì hoãn cập nhật là một giải thuật để hỗ trợ ABORT hủy giao dịch trong những tình huống máy hỏng.

- Trong khi một giao dịch đang chạy, không có sự thay đổi nào thực hiện bởi giao dịch đó được ghi hay cập nhật vào cơ sở dữ liệu.
- Khi thao tác xác nhận (Commit) được gọi, dữ liệu mới ghi trong file log hoặc trong bộ đệm được đẩy xuống đĩa ghi vào cơ sở dữ liệu.
- Nếu lệnh ABORT (hay Rollback) được gọi (cơ sở dữ liệu không thay đổi), toàn bộ dữ liệu trên bộ đệm hay file log bị xóa đi, không có tác động thay đổi nào xảy ra trên cơ sở dữ liệu.
- Trường hợp phần cứng hay máy tính bị hỏng, toàn bộ dữ liệu vẫn được bảo đảm toàn vẹn.

Ví dụ, hãy xem giao dịch TRAN1 sau:

Transaction TRAN1
read(A)
write(10,B)
write(20,C)

Sử dụng cơ chế cập nhật trì hoãn, quá trình diễn ra như sau:

Time	Action	Log																		
t1	START	-																		
t2	read(A)	-																		
t3	write(10,B)	B = 10																		
t4	write(20,C)	C = 20																		
t5	COMMIT	COMMIT																		
DISK	<table> <tr> <th colspan="3">Trước</th></tr> <tr> <td></td><td></td><td>B = 6</td></tr> <tr> <td>A = 5</td><td>C = 2</td><td></td></tr> </table>	Trước					B = 6	A = 5	C = 2		<table> <tr> <th colspan="3">Sau</th></tr> <tr> <td></td><td></td><td>B = 10</td></tr> <tr> <td>A = 5</td><td>C = 20</td><td></td></tr> </table>	Sau					B = 10	A = 5	C = 20	
Trước																				
		B = 6																		
A = 5	C = 2																			
Sau																				
		B = 10																		
A = 5	C = 20																			

Nếu hệ thống DBMS gặp lỗi và được khởi động lại:

- Nếu đĩa bị hỏng về mặt vật lý hoặc logic, khôi phục dữ liệu từ file log của giao dịch là không thể và giải pháp khôi phục từ file dump là bắt buộc.
- Nếu đĩa vật lý vẫn tốt thì việc khôi phục cơ sở dữ liệu về trạng thái đúng đắn là cần phải quan tâm.

- Phân tích nội dung file log của một giao dịch đã được kết thúc với lệnh COMMIT sau đó áp dụng toàn bộ nội dung giao dịch trước lệnh COMMIT vào cơ sở dữ liệu.
- Các mục giao dịch sau COMMIT thì hủy bỏ.
- Đẩy dữ liệu xuống đĩa và loại bỏ giao dịch để giảm kích thước file log.
- Quá trình xử lý lại giao dịch từ file log đôi khi còn gọi là Rollforward vì nó ngược lại với quá trình Rollback.

9.4. Cập nhật tức thời

Cập nhật tức thời hoặc UNDO/REDO là giải thuật khác để hỗ trợ tình huống xử lý thất bại ABORT hay máy hỏng.

- Khi một giao dịch đang chạy, các thay đổi thực hiện bởi giao dịch đó có thể được ghi ngay xuống cơ sở dữ liệu vào bất kỳ lúc nào. Tuy nhiên, bản chính và dữ liệu mới ghi xuống phải được lưu giữ lại trong file log trước khi chuyển thật sự sang cơ sở dữ liệu.
- Khi xác nhận giao dịch bằng lệnh COMMIT:
 - Tất cả các cập nhật chưa được ghi trên đĩa trước hết được lưu vào file log và chuyển xuống đĩa.
 - Dữ liệu mới sau đó được ghi vào cơ sở dữ liệu.
- Khi giao dịch bị hủy bỏ bằng lệnh ABORT hoặc khi máy bị hỏng và được khởi động lại, thực hiện thao tác đọc lại dữ liệu cũ từ file log.

Ví dụ, sử dụng cơ chế cập nhật tức thời, giao dịch TRAN1 có quá trình xử lý như sau:

Time	Action	LOG
t1	START	-
t2	read(A)	-
t3	write(10,B)	B = 6, giờ là 10
t4	write(20,C)	C = 2, giờ là 20
t5	COMMIT	COMMIT

D I S K	Trước			Diễn ra		
			B= 6			B= 10
	A= 5	C= 2		A = 5	C= 2	

Sau		
		B= 10
A = 5	C= 20	

Nếu hệ thống RDBMS thất bại và được khởi động lại:

- Nếu đĩa bị hỏng về mặt vật lý hoặc logic, khôi phục dữ liệu từ file log của giao dịch là không thể và giải pháp khôi phục từ file dump là bắt buộc.
- Nếu đĩa vật lý vẫn tốt thì việc khôi phục cơ sở dữ liệu về trạng thái đúng đắn là cần phải quan tâm.
 - Phân tích file log tìm vị trí trước lệnh COMMIT và lấy tất cả dữ liệu mới trong file log chuyển vào cơ sở dữ liệu.
 - Với các mục của file log sau COMMIT, chép dữ liệu cũ từ file log vào lại cơ sở dữ liệu.

10. ROLLBACK (QUAY NGƯỢC)

Quá trình hủy bỏ những thay đổi trong cơ chế cập nhật tức thời được gọi là thao tác Rollback (quay ngược). Do hệ quản trị DBMS không ngăn một giao dịch đọc những dữ liệu chưa được xác nhận (còn gọi là “dữ liệu bẩn” hay dirty data) của giao dịch khác, nên việc hủy bỏ giao dịch đầu tiên cũng kéo theo hủy bỏ tất cả giao dịch đã thực hiện việc đọc dữ liệu không đúng đắn này. Thao tác Rollback ảnh hưởng dây chuyền còn gọi là ‘Cascade Rollback’.

Chương 12:

LÝ THUYẾT CÀI ĐẶT VÀ XÂY DỰNG HỆ DBMS

Các vấn đề chính sẽ được đề cập:

- ✓ Các thành phần lõi.
- ✓ Đĩa và bộ nhớ.
- ✓ Tổ chức chỉ mục lưu trữ.
- ✓ Bảng băm – Hash Table.
- ✓ Cây nhị phân cân bằng – B+ Tree.
- ✓ Chỉ mục và truy cập.
- ✓ Chi phí đánh chỉ mục và truy cập chỉ mục.

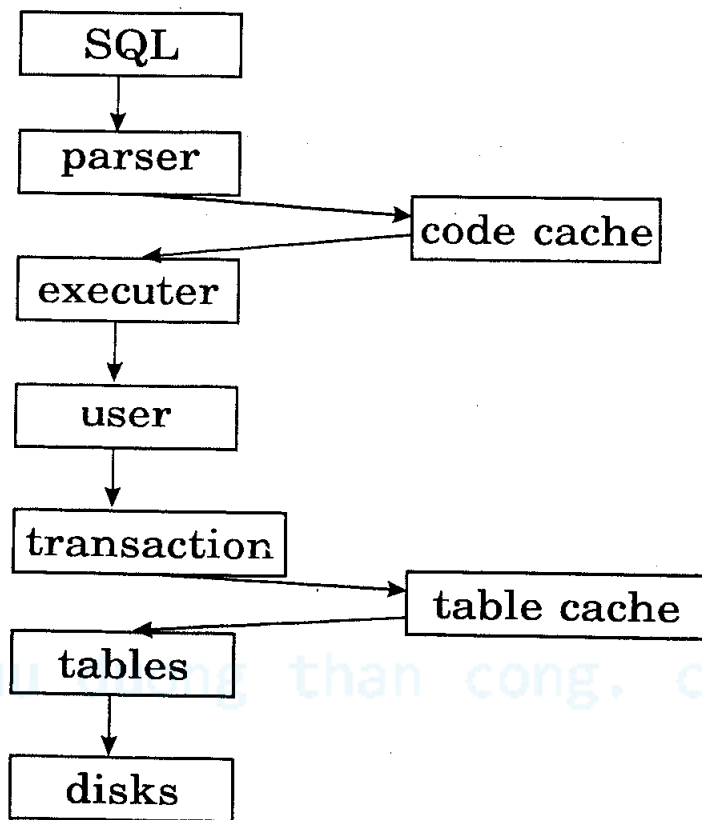
1. CÁC THÀNH PHẦN LỖI

Một hệ thống quản lý cơ sở dữ liệu điều khiển những yêu cầu phát sinh từ giao diện SQL của người dùng, sinh ra hoặc thay đổi dữ liệu theo các yêu cầu này.

Cơ chế này đòi hỏi rất nhiều mức xử lý hệ thống. Hình 12-1 là quá trình một hệ DBMS thực hiện và phân tích thực thi câu lệnh SQL gửi đến bởi người dùng.

- **Parser (Phân tích cú pháp):** Phân tích cú pháp để nhận dạng ra các lệnh SQL cần xử lý. Thông báo lỗi cú pháp cho người dùng nếu họ nhập sai về câu lệnh hoặc tham số. Việc phân tích có thể mất thời gian vì vậy các hệ DBMS thường sử dụng bộ đệm lưu lại các câu lệnh đã phân tích để nếu câu lệnh có sử dụng lại thì sẽ không mất thời gian phân tích nữa. Để làm điều này đa số hệ thống sử dụng tham số thay thế ví dụ:
`SELECT empno FROM employee where surname = ?`

Ký tự '?' đại diện cho tất cả thông tin có thể có và được người dùng cung cấp khi câu lệnh SQL thực thi.



Hình 12-1

- **Executer (Bộ thực thi):** Đây là lõi của hệ thống SQL, nó tiếp nhận cú pháp SQL đã qua phân tích và chuyển thành cú pháp của Đại số quan hệ. Mỗi đoạn mã trong Đại số quan hệ được tối ưu hóa và chuyển xuống mức dưới hành động cho ra kết quả.
- **User (người dùng):** Khái niệm người dùng trong vấn đề bảo mật và sử dụng hệ thống được cài đặt từ đây. Mỗi hệ thống có thể tổ chức khái niệm người dùng theo cách riêng của mình.
- **Transaction (giao dịch):** Các câu lệnh truy vấn được thực thi bên trong một đơn vị giao dịch. Cùng câu truy vấn từ một người dùng có thể thực thi nhiều lần khác nhau trong nhiều giao dịch khác nhau. Mỗi giao dịch là hoàn toàn tách biệt.

- **Table (Bảng):** Cấu trúc bảng được kiểm soát ở mức thấp. Hầu hết toàn bộ thông tin hệ thống cũng như dữ liệu người dùng được các hệ DBMS lưu trong bảng dưới dạng siêu dữ liệu metadata
- **Table Cache (Bộ đệm dữ liệu bảng):** Truy xuất trực tiếp đĩa cứng thì khá chậm mặc dù đĩa là nơi lưu giữ dữ liệu nhiều và dài hạn khá hiệu quả. Ký ức bộ nhớ nhanh hơn nhiều, bộ đệm bảng được dùng để lưu các thông tin mô tả bảng cùng những dữ liệu yêu cầu truy xuất thường xuyên trong các giao dịch. Đĩa vẫn được đồng bộ hóa với ký ức bộ nhớ như là một phần của giao dịch.
- **Disk (Đĩa lưu trữ):** Tất cả các hệ thống cơ sở dữ liệu đều dùng đĩa cứng làm hệ thống lưu trữ. Đĩa là nơi lưu tất cả các bảng hệ thống DBMS, thông tin người dùng, những định nghĩa mô hình và dữ liệu người dùng. Nó cũng cung cấp những công cụ lưu trữ thông tin lịch sử (log) của các giao dịch đã diễn ra.

Ngữ cảnh “người dùng” được cài đặt khác nhau tùy vào hệ thống cơ sở dữ liệu. Sơ đồ sau sẽ cho bạn một ý tưởng về cách tiếp cận ngữ cảnh User của hai hệ thống khác nhau là Oracle và MySQL.

MySQL

Oracle

users

databases

tables

columns

databases

users

tables

columns

Hình 12-2

Tất cả người dùng trong hệ thống đều có một tên (Username) và mật khẩu (Password) để đăng nhập. Trong Oracle, quá trình kết nối cần cung cấp tên cơ sở dữ liệu. Điều này cho phép hệ Oracle DBMS chạy nhiều cơ sở dữ liệu hiệu quả và các hệ cơ sở dữ liệu được cô lập lẫn nhau. Trong khi đó, với MySQL người dùng kết nối chỉ cần username và mật khẩu, MySQL sẽ đặt người dùng vào một vùng tablespace đặc biệt chứa cơ sở dữ liệu khi người dùng chỉ định. Trong Oracle, không gian cơ sở dữ liệu và username là tương đương nhau. Một khi đã ở bên trong không gian của cơ sở dữ liệu thì bạn có thể trông thấy các bảng cũng như có thể truy xuất được các cột dữ liệu của bảng.

2. ĐĨA VÀ KÝ ỨC BỘ NHỚ

Sự đánh đổi giữa sử dụng không gian đĩa hoặc sử dụng ký ức nhớ là hoàn toàn dễ hiểu ...

Vấn đề	Ký ức nhớ so với đĩa
Tốc độ	Ký ức nhanh hơn 1000 lần so với đĩa cứng.
Không gian lưu trữ	Đĩa có thể lưu trữ thông tin hơn bộ nhớ cả ngàn thậm chí cả triệu lần với cùng chi phí.
Liên tục	Khi tắt nguồn, đĩa giữ dữ liệu vĩnh viễn còn ký ức thì xóa mất mọi thứ.
Thời gian truy nhập	Ký ức truyền gửi dữ liệu tính theo nanô-giây, trong khi đĩa tính bằng mili-giây.
Kích thước khối xử lý	Ký ức truy nhập 16 đến 32-bits (2-4 bytes), trong khi đĩa có thể đọc 1 lần một khối nhiều bytes.

Cân đối được thời gian truy xuất đĩa và bộ nhớ là một nghệ thuật mà các hệ DBMS tạo nên thế mạnh cho riêng mình.

3. TỔ CHỨC CHỈ MỤC

Một hệ DBMS tốt phải xây dựng được thuật giải lưu trữ và truy xuất hiệu quả dữ liệu trên đĩa. Các hệ thống DBMS hiện đại đều hỗ trợ cơ chế đánh chỉ mục (Index), đây là cách dò tìm và xử lý dữ liệu nhanh nhất, đáp ứng các yêu cầu về tốc độ.

Với cơ chế đánh chỉ mục, DBMS cho phép truy tìm dữ liệu nhanh và hiệu quả, không phải đọc nhiều thông tin thừa khác lưu trên đĩa. Có nhiều cách tiếp cận với cơ chế đánh chỉ mục, nhưng hai thuật giải quan trọng thường sử dụng là lưu theo bảng băm (Hash table) và lưu theo cấu trúc cây nhị phân (Binary tree). Ví dụ, khi điều khiển xử lý file Log của giao dịch, (chúng ta đã nghiên cứu trong chương trước) kỹ thuật này thường được sử dụng để cài đặt.

3.1. Hash Table

Hash Table là một trong những cấu trúc chỉ mục đơn giản nhất mà một hệ cơ sở dữ liệu có thể thực hiện. Thành phần chính của chỉ mục Hash Table là hàm băm (hash function), hàm này sẽ tạo khóa (hay chỉ số truy tìm) cho từng mục dữ liệu mà hệ cơ sở dữ liệu sẽ truy tìm. Để hiệu quả, các hệ DBMS thường xây dựng sẵn chỉ mục cho mỗi bảng mà bạn tạo ra có chỉ định thuộc tính khóa chính, ví dụ:

```
CREATE TABLE test (  
  id INTEGER PRIMARY KEY  
  ,name varchar(100)  
);
```

Trong bảng ví dụ trên, KEY được chỉ định là khóa chính và được xây dựng chỉ mục, giả sử chúng ta tạo mới 4 dòng:

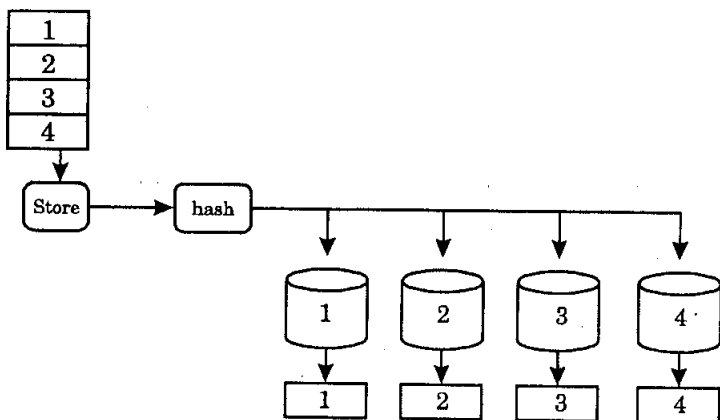
```
insert into test values (1,'Gordon');
```

```
insert into test values (2,'Jim');
```

```
insert into test values (4,'Andrew');
```

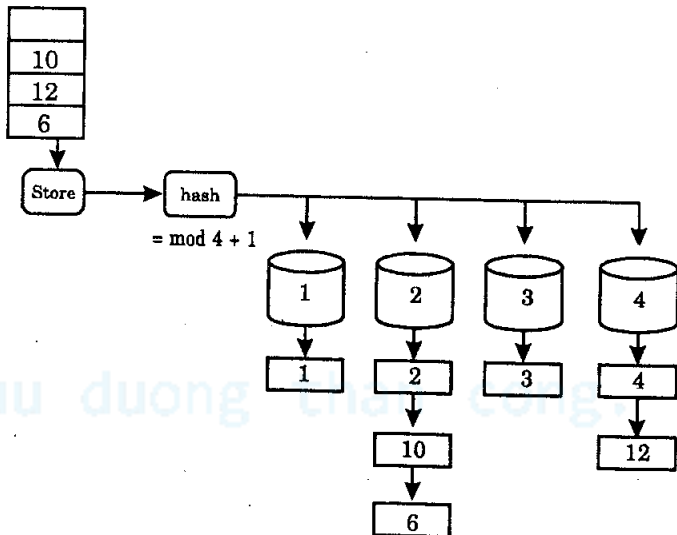
```
insert into test values (3,'John');
```

Giải thuật băm sẽ chia chỗ trống trên đĩa để lưu các dòng dữ liệu trong các vùng đã được tổ chức theo chỉ mục, các vùng này gọi là khối lưu trữ (buckets). Nếu một dòng có khóa chính phù hợp với các yêu cầu của khối buckets, nó sẽ được cất giữ trong vùng đó. Giải thuật để quyết định sử dụng buckets nào dựa vào hàm Hash. Giả sử, chúng ta có một hàm Hash đơn giản trong đó quy định chỉ số buckets bằng với giá trị khóa chính. Khi tạo chỉ mục, chúng ta sẽ xác định (các hệ DBMS sẽ quyết định bao nhiêu là đủ) cần tổ chức bao nhiêu khối buckets. Trong ví dụ này, giả sử chúng ta quyết định có 4 khối buckets. Việc đưa các dòng dữ liệu vào sẽ như sơ đồ hình 12-3.



Hình 12-3

Bây giờ, chúng ta có thể tìm thấy khóa Id 3 nhanh chóng và dễ dàng bởi việc khối mang chỉ mục 3 và đọc nội dung dữ liệu trong đó. Thế nhưng số khối buckets cấp phát sẵn đã đầy thì sao? Để thêm nhiều giá trị hơn chúng ta cần tái sử dụng lại các khối. Chúng ta cần một hàm Hash tốt hơn dựa vào phép chia cho 4 lấy phần dư (mod 4). Bây giờ, khối buckets 1 sẽ nắm giữ các số Id (1, 5, 9...), khối buckets 2 giữ (2, 6, 10...) ...



Hình 12-4

Giờ đây, một khối buckets có thể chứa nhiều dòng dữ liệu. Điều này có thể xảy ra va chạm vì khi một khóa xác định được buckets cần đến thì hệ thống cần phải thực hiện một công việc nữa là dò tìm trong buckets đó để tìm ra dòng dữ liệu có khóa chính xác như mong muốn hay không. Tuy

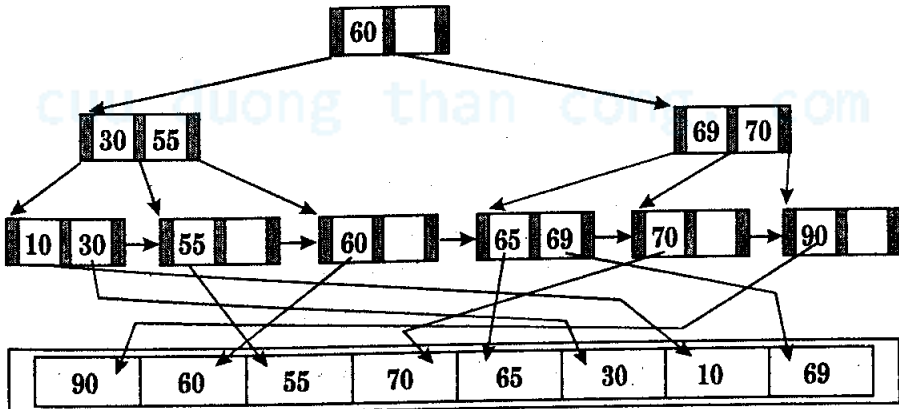
nhien, đây là công việc của hệ thống với cấp độ người dùng, bạn chỉ cần tạo chỉ mục là đủ (lệnh SQL CREATE INDEX), mọi thứ cài đặt hàm Hash bên trong đã được hệ DBMS lo liệu.

3.2. Cây nhị phân (Binary Tree)

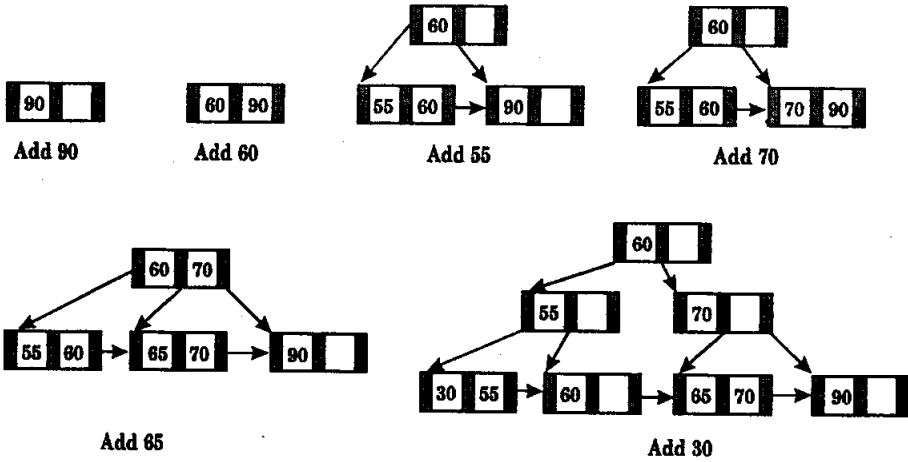
Cây nhị phân là cách tiếp cận mới nhất cung cấp cơ chế đánh chỉ số. Giải thuật này thông minh hơn sử dụng bảng Hash table, và cũng giải quyết vấn đề không biết trước là phải tạo bao nhiêu khối buckets như trong thuật giải Hash. Thuật giải cây nhị phân được áp dụng gọi là B+ Tree (Cây cân bằng). Với thuật giải B+ Tree, dữ liệu nguyên bản được lưu theo thứ tự tạo thành của nó. Điều này khiến nhiều chỉ số B+ Tree được lưu giữ lại cho cùng tập dữ liệu.

- ◊ Mức thấp nhất trong chỉ mục có một mục nhập cho mỗi mẫu tin dữ liệu.
- ◊ Chỉ mục được tạo động khi dữ liệu thêm vào file.
- ◊ Khi dữ liệu thêm vào chỉ mục được mở rộng sao cho mỗi mẫu tin có thể truy xuất đến cùng số mức của chỉ mục (và do đó cây “cân bằng”).
- ◊ Các mẫu tin có thể được truy nhập thông qua một chỉ mục hoặc số tuần tự.
- ◊ Mỗi chỉ số nút trong một B+ Tree có thể giữ một số khóa nhất định.

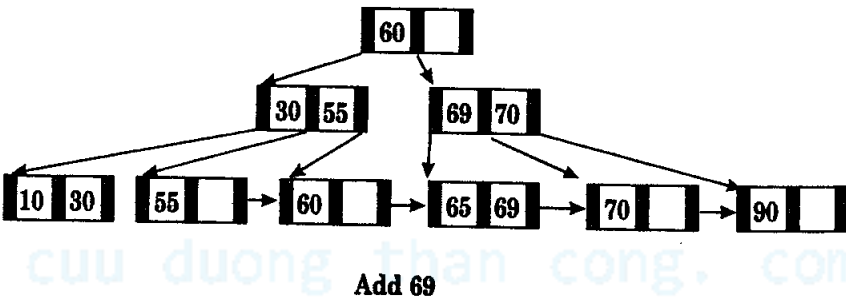
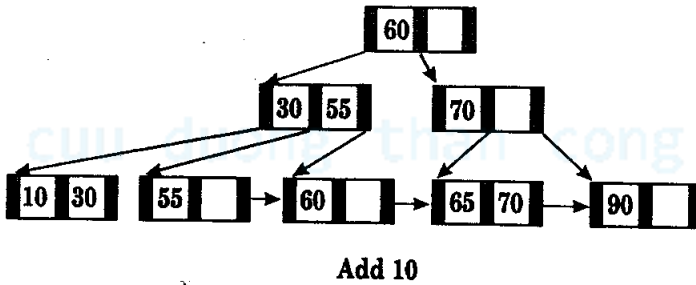
Dưới đây là sơ đồ ví dụ về cách dữ liệu mới được thêm vào cấu trúc B+ Tree.



Hình 12-5



Hình 12-6



Hình 12-7

- Mức đỉnh (gốc) của cây chỉ mục thông thường được giữ trong ký ức. Nó được đọc một lần từ đĩa và lưu lại trong suốt quá trình người dùng truy vấn.

3.3. Chi phí đánh chỉ mục và truy cập trên chỉ mục

- Tìm kiếm trực tiếp như trên khá tốn thời gian, trong khi tìm kiếm chỉ mục trong ký ức là rất nhanh.

- Chỉ phí đánh đổi lại là khi dữ liệu mới được thêm vào, DBMS phải mất một khoản thời gian cấu trúc và sắp xếp lại vị trí chỉ mục.
- Một hệ DBMS có thể sử dụng nhiều cách tổ chức file khác nhau cho mục đích lưu trữ chỉ mục của mình. Người dùng DBMS nói chung ít quan tâm đến nội dung kiểu file chỉ mục vì ít khi phải tiếp cận với file dữ liệu thô của cơ sở dữ liệu.
- Cấu trúc B+ Tree được tổ chức trên file khi có yêu cầu sử dụng chỉ mục cho bảng dữ liệu.
- Chỉ mục được phát sinh:
 - Cho các cột dữ liệu dùng làm khóa chính (được tạo với tùy chọn PRIMARY KEY hay UNIQUE trong lệnh SQL tạo bảng CREATE TABLE).
 - Cho các cột được xác định tường minh trong câu lệnh SQL như:

CREATE [UNIQUE] INDEX indexname ON tablename (col [,col]...);

- Chỉ mục của khóa chính phải là duy nhất (unique). Chỉ mục của khóa ngoại có thể trùng nhau.
- Trong câu lệnh truy vấn SQL có sử dụng mệnh đề WHERE, cột dữ liệu nào có đánh chỉ mục sẽ làm cho tốc độ câu truy vấn tăng rất nhanh, đặc biệt là trong các phép kết hợp dạng A.a1=B.b1.
- Tuy nhiên, không có sự cải tiến nào về tốc độ với những so sánh dạng IS [NOT] NULL.
- Đánh chỉ mục trên các cột có ít thông tin như dạng Y/N hay True/False có thể làm chậm tốc độ câu truy vấn.
- Có thể đánh chỉ mục trên cùng lúc nhiều cột, khi đó cột nào có nhiều thông tin giá trị nhất nên đặt trước.
- Nên tránh đánh chỉ mục trên các bảng ít dữ liệu, các bảng cập nhật thường xuyên hay các cột chứa giá trị chuỗi dài.
- Một bảng có thể có nhiều chỉ mục khác nhau.

- Đọc hoặc cập nhật một dòng mẫu tin có thể rất nhanh.
- Chèn một dòng dữ liệu có thể chậm hơn do bảng chỉ mục phải cập nhật lại (thực hiện bởi hệ thống).
- Tương tự thao tác chèn, xóa dòng dữ liệu cũng có thể chậm do bảng chỉ mục phải cập nhật lại.

cuu duong than cong. com

cuu duong than cong. com